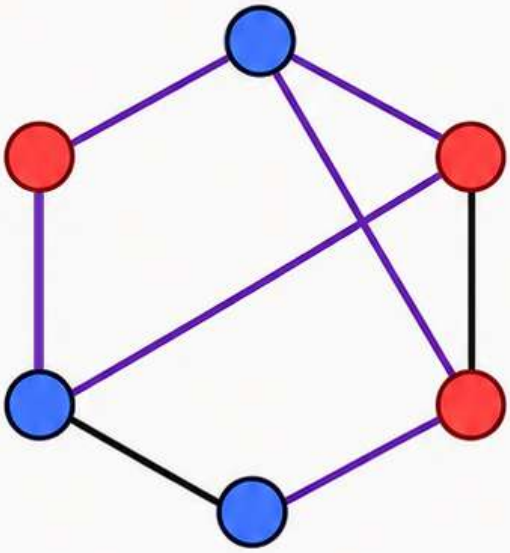
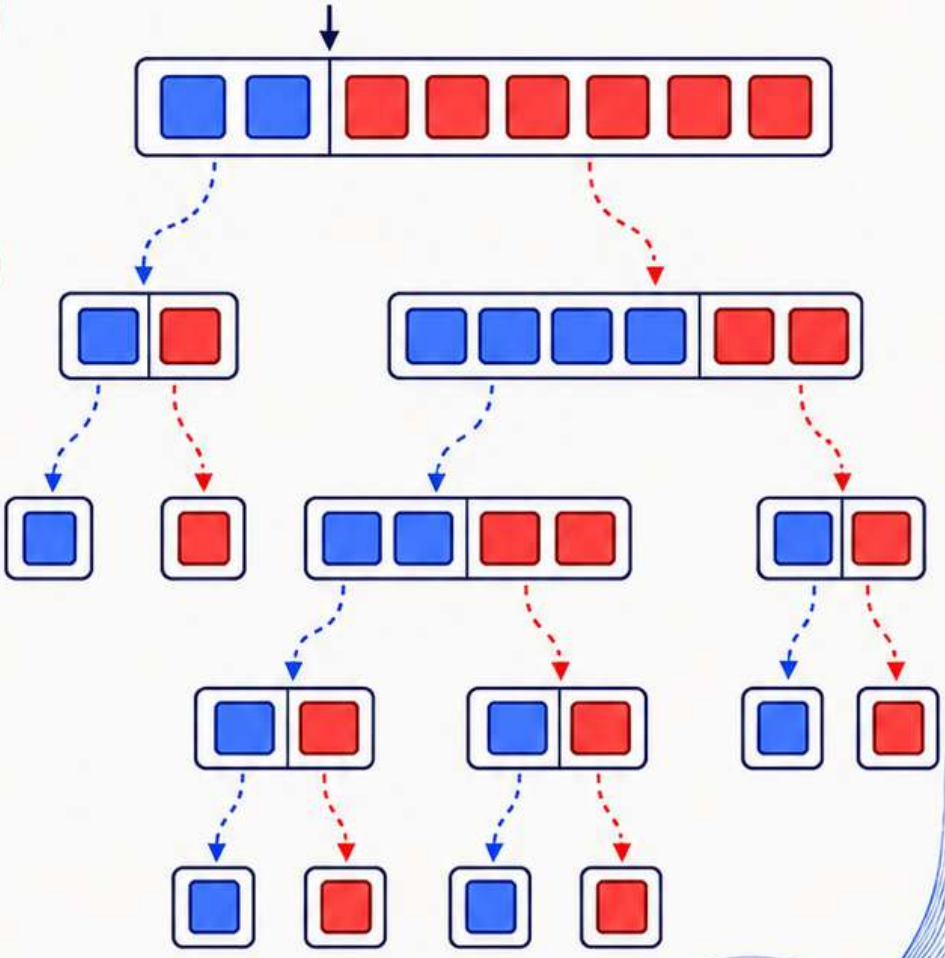
Randomised Algorithms

Sahil M Deo





Contents

Chapter 1

Introduction

1.1 What is a Randomised Algorithm?

A randomised algorithm is an algorithm that makes random choices during its execution. Unlike deterministic algorithms, whose behaviour is completely determined by the input, the behaviour of a randomised algorithm may vary between different executions on the same input.

Typically, the algorithm uses a random number generator (`rng`) to decide between multiple possible actions. As a result, quantities such as running time (Las Vegas algorithms), memory usage, or even correctness (Monte Carlo algorithms) can become random variables (though usually not all of them at the same time). The fact that correctness itself is not guaranteed can initially seem undesirable, especially in critical systems where absolute correctness is expected. However, in many practical situations, a sufficiently small error probability is considered acceptable. For example, if an algorithm can be shown to fail with probability at most 10^{-30}, then the chance of failure may already be smaller than the probability of random hardware faults such as memory bit flips caused by cosmic radiation, which programmers typically ignore during ordinary software development.

The performance of a randomised algorithm is therefore analysed using probability and expectation. If T_{run} denotes the runtime of the algorithm, we are usually interested in quantities such as $T_{algo} = \mathbb{E}[T_{run}]$ over all possible random choices[1] made by the algorithm.

1.2 Implementation of Randomness in C++

Let us make things more concrete with some actual code and implementation details! C++ provides pseudo-random[2] number generators through the `<random>` library. A commonly used generator is `mt19937`.

```
#include <bits/stdc++.h>
mt19937 rng(42);
```

[1] T_{algo} can still be a random variable over the input - so this may **not** be the same as average case performance of the algorithm T_{avg}, which is an expected value over all possible inputs. But for cleanliness of presentation, we will often ignore this distinction and use these terms interchangeably or simply use the variable T to mean either of them, depending on context.

[2] Since computers are deterministic, they rely on parameters like atmospheric noise and temperature for randomness. The details are mostly irrelevant for us... We will take pseudo random as true random for our purposes until the very last chapter in which the differences will be one of the main points of discussion...

This creates an object rng with seed[3] as 42 which will be passed as a parameter to other functions. It has the capability to produce "truly" random 32-bit integers.

Generating Random Integers

To generate a random integer uniformly from a range $[L, R]$, we use `uniform_int_distribution<int>`.

```
uniform_int_distribution<int>uid(L,R);

int x=uid(rng);
int y=uid(rng);
.
.
.
```

Each generation takes around constant time.

Generating a Random Permutation

A random permutation of an array (in-place modification) can be generated using `shuffle`:

```
shuffle(a.begin(),a.end(),rng);
```

Implementing Random Decisions

Sometimes, an algorithm must choose between multiple actions with specified probabilities. For example, let three actions X,Y,Z be such that they are executed with probabilities:

$$\Pr(X) = \frac{1}{5}, \qquad \Pr(Y) = \frac{3}{5}, \qquad \Pr(Z) = \frac{1}{5}$$

One simple way to implement this is to generate a random integer uniformly from 1 to 5 and partition the output space into sizes proportional to the probabilities we want to achieve.

```
uniform_int_distribution<int>uid(1,5);

int v=uid(rng);

if(v==1)
{
    //do X
}
else if(v<=4)
{
    //do Y
}
else
{
    //do Z
}
```

[3]Seed values are used in pseudo random generators as a way to recreate the generated random numbers. This can be used for debugging a randomised algorithm. If you don't care about that, set it to the current time in milliseconds or something similar and that should be good enough!

Since we have now dealt with the actual implementation details in a real programming language, we are well-equipped to write actual algorithms which use random decisions in their implementation. Sometimes we may write code with simplified or approximate syntax (a.k.a. pseudocode) which makes it easier to understand.

1.3 Problems

1 The following procedure is applied on an array a of n integers –

```
for(int i=n-1;i>0;i--)
{
    int j=random integer from [0,i];
    swap(a[i],a[j]);
}
```

(a) Show that this generates a valid permutation of the original array.

(b) Show that every permutation is equally likely to be generated (including the original array).

2 Suppose you have access only to a function `bool coin()` which returns 0 or 1 with equal probability. Let the phrase "generate a probability p" mean designing a function which returns 1 with probability p and 0 with probability $1-p$.

(a) Design a procedure to generate a probability $\frac{k}{2^m}$ using only finite number of calls to `coin()`.

(b) Design a procedure to generate any rational probability $\frac{p}{q}$ with **expected** finite number of calls to `coin()`
Hint: Think of the binary representation of a number.

(c) Generalise the above ideas to generate any real probability p with **expected** finite number of calls to `coin()`.

Chapter 2

Algorithmic Performance

For algorithms whose performance[1] depends on the input size or other input parameters, we usually estimate the number of elementary operations performed as a function of the input size n or the maximum value of input A_{max}, usually denoted by A. This gives a mathematical way to compare algorithms independent of hardware or implementation details.

A classical example is `BubbleSort`. The algorithm repeatedly traverses the array, comparing adjacent elements and swapping them whenever they are in the wrong order. After each pass, the largest remaining element moves to its correct position near the end of the array. With some thought it can be proved that the total number of swaps performed by `BubbleSort` is exactly equal to the number of inversions N_{inv} in the input array. (An inversion is a pair of indices (i, j) such that $i < j$ but $a_i > a_j$.) By taking swaps to be one operation taking constant time, we can estimate runtime of `BubbleSort` to be $T_{run} \propto N_{inv}$

2.1 Operations

In complexity analysis, we must know which operations are to be taken as a unit of time. This is sometimes a bit subjective since arithmetic operations ($+$,-,$\times$,$\div$) can be taken as constant time if we operate under the assumption that we are using fixed-length numbers like 32-bit integers.

Regardless, here is a list of the common operations taken as a unit of time in complexity analysis.

- Arithmetic operations ($+$,-,$\times$,$\div$)
- Comparisons ($<$,$>$,$\leq$,$\geq$)
- Boolean operations ($\wedge$,$\vee$,$\neg$)
- Bitwise operations (&,$\|$,$\neg$)
- Variable read/write
- Array read/write

2.2 Worst Case Analysis

In worst case analysis, we determine the maximum possible number of operations over all valid inputs of size n. For `BubbleSort`, the worst case occurs when the array is sorted in reverse order, where every pair is inverted.

[1] Although this section focuses primarily on running time, the same methods of analysis can also be applied to other resources such as memory usage.

$$N_{inv,max} = \binom{n}{2} = \frac{n(n-1)}{2}$$

Worst case analysis is useful because it guarantees an upper bound on the running time regardless of the input provided.

2.3 Average Case Analysis

In average case analysis, we compute the expected number of operations assuming that all valid inputs are equally likely. For `BubbleSort`, this corresponds to analysing the expected number of inversions in a random permutation of size n, where all $n!$ permutations are equally likely. Define the indicator[2] variable I_{ij} for each pair (i, j) as

$$I_{ij} = \begin{cases} 1, & \text{if } (i,j) \text{ is an inversion} \\ 0, & \text{otherwise} \end{cases}$$

The point of defining the indicator variable is to notice that the number of inversions is the sum of all I_{ij}. Formally we can write this as

$$N_{\text{inv}} = \sum_{1 \le i < j \le n} I_{ij}$$

For every pair of indices (i, j) with $i < j$, the probability that the pair forms an inversion is $\frac{1}{2}$ since for every permmutation with $a_i > a_j$, there is a permuation with $a_j > a_i$ and vice versa.

$$\Pr\big((i,j) \text{ is an inversion}\big) = \frac{1}{2} \implies \mathbb{E}[I_{ij}] = \frac{1}{2}$$

Using linearity of expectation[3],

$$\mathbb{E}[N_{\text{inv}}] = \mathbb{E}\left[\sum_{1 \le i < j \le n} I_{ij}\right] = \sum_{1 \le i < j \le n} \mathbb{E}[I_{ij}] = \sum_{1 \le i < j \le n} \frac{1}{2} = \frac{n(n-1)}{4}$$

A valuable strategy which we can deduce from this analysis is to randomise the input sequence. In case of an adversary[4], this can be useful because we can expect an improvement in time by factor of 2.

2.4 Time Complexity Analysis

Let us first define the asymptotic complexity of a function $f(n)$ in terms of n. My favourite way to think about complexity is that we try to find the "simplest" function $g(n)$ such that

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = k, \quad k \in \mathbb{R}_{>0}$$

This is compactly represented as $f(n) = \Theta(g(n))$. If via inequalities we are able to find $f(n) \le h(n)$ or $f(n) < h(n)$ and $h(n) = \Theta(g(n))$, then we can represent that by $f(n) \le \Theta(g(n))$. Similarly we define what it means to say that $f(n) \ge \Theta(g(n))$.

[2] Refer Appendix A.6

[3] Refer Appendix A.1

[4] An adversary is someone who deliberately gives inputs to make your algorithm perform badly. Randomising the input is one possible defense against adversaries.

By applying Sandwich theorem on the ratio $f(n)/g(n)$, we can conclude that if $f(n) \geq \Theta(g(n))$ and $f(n) \leq \Theta(g(n))$ then $f(n) = \Theta(g(n))$.

Since "simplest" function is not really well defined, it is best to look at some examples and gain intuition!

Function	Asymptotic Complexity
7	$\Theta(1)$
$n^2 + 100n + 1$	$\Theta(n^2)$
$n + \log n$	$\Theta(n)$
$\log(n!)$	$\Theta(n \log n)$
$\sum_{i=1}^{n} i$	$\Theta(n^2)$
$\sum_{i=1}^{n} \frac{1}{i}$	$\Theta(\log n)$
$\sum_{i=1}^{n} \log i$	$\Theta(n \log n)$

In the example of `BubbleSort`, the worst case runtime was $T_{run}(n) = \frac{n(n-1)}{2}$. So you will often come across statements like "`BubbleSort` has worst case time complexity $\Theta(n^2)$". What this means is that the complexity of the worst case runtime as a function of n is $\Theta(n^2)$.

A natural question which arises in comparing different algorithms for the same task is which one is faster? For example, consider two algorithms A and B, where A has runtime $T_{run}(n) = n^2$ and B has runtime $T_{run}(n) = 100n \log n$. To simplify our comparison, we just compare their complexities.

Comparing between the complexity functions $f(n)$ and $g(n)$ themselves is quite straight forward – we just evaluate the limits of the ratio $f(n)/g(n)$ as $n \to \infty$. If the first limit is 0, then $f(n) < g(n)$, if the limit is a non-zero constant (usually 1) then $f(n) = g(n)$, otherwise $f(n) > g(n)$. Basis these simple rules, we can have a total order of the common complexity functions as

$$\Theta(1) < \Theta(\log n) < \Theta(n^{\frac{1}{x}}) < \Theta(n) < \Theta(n \log n) < \Theta(n^2)$$

In the above comparison, $x > 1$. Algorithms having time complexity smaller than $\Theta(n)$ are called sub-linear time algorithms.

Finally all the above discussion allows us to say that A: $T_{run} = n^2$ is worse than B: $T_{run} = 100n \log n$ since A is $\Theta(n^2)$ and B is $\Theta(n \log n)$ even though for small n the first one may well be better, but for small n the performance doesn't really[5] matter anyway...

[5]Integer multiplication is an interesting case. The classic method of multiplying digit by digit as taught in school is $\Theta(n^2)$ time, but there are complicated methods which can do it in $\Theta(n \log n)$ time. But the benefit of using the complicated methods really doesn't show up until quite a large n because of high constant factor.

2.5 Problems

1 A `vector` in `C++` is an array with variable size. Once the memory allocated to the `vector` is exceeded, it is reallocated more memory somewhere else. The designers of the `vector class` must choose a sequence of sizes $s_0 = 0 < s_1 < s_2 < \ldots$ such that when the size of the `vector` becomes any s_i, we reallocate and assign memory of size s_{i+1} to the `vector`.

Thus each `push_back()` operation is one operation if space is available and s_i+1 otherwise. For the average case analysis take all sizes of vectors between 0 and $n-1$ (both inclusive) to be equally likely.

(a) If $s_i = i$, show that the time complexity for the average `push_back()` is $\Theta(n)$.

(b) Show that in general the time complexity for the average `push_back()`is $\Theta\left(1 + \frac{\left(\sum_{s_i<n} s_i\right)}{n}\right)$.

(c) Knowing the previous result, come up with a simple series of s_i that would give a time complexity of $\Theta(1)$ for the average `push_back()`.

2 Consider the algorithm

```
f(n)
{
    count = 0
    while(n>1)
    {
        if(n even)
            n = n/2
        else //n is odd
            n = n-1
        count = count + 1
    }
    return count
}
```

Here $f(n)$ counts the number of iterations required to reduce the number n to 1. We can think of $f(n)$ as a measure of the time taken by the algorithm (one iteration being one operation). Now consider $x \in \{2^{k+1} - 1 : k \in \mathbb{N}\}$ and n be the input constrained to be between 1 and x. Find the asymptotic[6] ratio of worst case time and average case time of the algorithm with respect to n. Equivalently, find

$$\lim_{x\to\infty} \frac{\max_{1\le n\le x} f(n)}{\left(\frac{1}{x}\sum_{n=1}^{x} f(n)\right)}$$

[6]We have seen this ratio being 2 in the case of `BubbleSort`.

Chapter 3

Complexity Classes

What is the best time complexity for a problem? When can we hope to come up with a *fast* algorithm? This is exactly the point of defining complexity classes. While the classes themselves are defined rigorously, there are some very important[1] statements which have not yet been proved – they remain open problems! In practice, however, we have conjectures which we believe to be true for now...

3.1 Decision Problems

Complexity classes are traditionally defined using *decision problems*. A decision problem is a computational problem whose answer is either `YES` or `NO`.

- Given a graph G, is G connected?
- Given an integer m, is m prime?
- Given a graph G and an integer k, does G contain a clique[2] of size at least k?

Although not all computational tasks are naturally decision problems, most can be reformulated as one. For example, the largest clique in a graph can be reformulated[3] as "is there a clique of size at least k in G?".

3.2 P

The class **P** consists of all decision problems that can be solved by a deterministic algorithm in at most polynomial time. Let Q be a decision problem and $T(Q)$ be the best time complexity[4] of a deterministic algorithm for Q. Then $T(Q) \leq \Theta(n^c)$ for some constant $c \geq 1$ implies $Q \in \mathbf{P}$.

$$\mathbf{P} = \bigcup_{c \geq 1} \{Q : T(Q) \leq \Theta(n^c)\}$$

Decision Problem(Q)	Algorithm	Input	Input Size	Runtime (T(Q))
Primality Test	AKS	Integer	$m = \Theta(\log_2 n)$	$\Theta(m^{12})$
Graph Connectivity	BFS	Adjacency Matrix	$n = \lvert V \rvert \times \lvert V \rvert$	$\Theta(n)$

[1] Such as the classic question of $\mathbf{P} \neq \mathbf{NP}$...

[2] Refer Appendix D.3.4

[3] Some of these reformulations are called *polynomial time reductions* which will be properly discussed later.

[4] This time complexity is in terms of the input length and not the input number itself – a number n when taken as input is represented by a bit string of size $m = \Theta(\log n)$. So an algorithm which takes $O(n)$ time will be considered as an exponential time algorithm since $\Theta(n) = \Theta(2^m)$. This is why the normal method of trying all numbers from 1 to $\sqrt{n}$ will not prove that primality testing of a number is in P.

3.3 NP

NP stands for non-deterministic polynomial time. What that really means is, if you perform the non-deterministic procedure called *guessing*, you can check your answer in polynomial time. **NP** does **not** stand for non-polynomial time. Formally, the class **NP** consists of all decision problems whose answer when `YES` can be verified in polynomial time. The algorithm which solved the problem can output a *certificate* along with `YES` – this certificate is information which can be used to verify in polynomial time that the answer is indeed `YES`.

Decision Problem (Q)	**Certificate**	**Verification Time**
Clique of size k	Set of k vertices	$\Theta(k^2)$
Graph Coloring of k colors	Vertex : Color mapping	$\Theta(V+E)$

An obvious conclusion from the definition of **NP** is that $\mathbf{P} \subseteq \mathbf{NP}$ because any problem that can be solved in polynomial time can also be verified in polynomial time by giving the problem itself as a certificate and the verification procedure is the solution itself.

$\mathbf{NP_{co}}$

The class $\mathbf{NP_{co}}$ contains complements of problems in **NP**. In this class of problems, the answer `NO` has a certificate verifiable in polynomial time. For example, the complement of the clique problem asks whether every clique in the graph has size at most $k-1$. The answer `NO` can be accompanied by a certificate which is the set of vertices forming a clique of size at least k.

An obvious conclusion is that any algorithm that solves an **NP** problem also immediately solves the corresponding $\mathbf{NP_{co}}$ problem by just flipping the answer from `YES` to `NO` and vice versa but the catch being it may not be able to produce a certificate for `NO` answer.

3.4 Polynomial-Time Reductions

To compare the difficulty of computational problems, we use the idea of *polynomial-time reductions*. A problem A is polynomial-time reducible to a problem B if we can convert A to B in polynomial time.

This is denoted by $A \leq_p B$. This means that if we can solve B in polynomial time, then we can also solve A in polynomial time. The notation suggests B is harder than A and one might find this confusing because we "reduced" A to B so shouldn't B be easier? Think of a reduction as a one-way procedure that converts A to a *particular* case of B. Solving the general case of a problem is at least as hard as solving the particular case. That is why B is at least as hard as A under polynomial time reduction.

The operator $\leq_p$ is a partial order[5]. We can also define $=_p$ as $A =_p B$ iff $A \leq_p B$ and $B \leq_p A$. This can be used to induce equivalence classes[6] within the complexity classes like $\mathbf{NP-complete}$ which is the "biggest" equivalence class in **NP**.

[5] Refer Appendix D.2.1
[6] Refer Appendix D.2.2

3.5 NP-Complete

NP – complete consists of those **NP** problems which are at least as hard as *every* problem in **NP**. For an L in **NP – complete**, every problem in **NP** can be polynomially reduced to L.

$$\textbf{NP} - \textbf{complete} \;=\; \{L \in \textbf{NP} : Q \leq_p L \quad \forall\, Q \in \textbf{NP}\}$$

3.6 NP-Hard

NP – hard consists of all those problems which are harder than *every* problem in **NP**. These problems need not even have polynomially verifiable certificates for `YES`. The problems need not even be decision problems – they could be any type of problem like optimisation or search problems.

$$\textbf{NP} - \textbf{hard} \;=\; \{L : Q \leq_p L \quad \forall\, Q \in \textbf{NP}\}$$

An obvious conclusion is that $\textbf{NP} - \textbf{complete} = \textbf{NP} \cap \textbf{NP} - \textbf{hard} \subseteq \textbf{NP} - \textbf{hard}$.

Example

The K-SAT decision problem is defined as follows – given a Boolean formula ϕ in the form $(k - CLAUSE_1)$ AND $(k - CLAUSE_2)$ AND ... (where a $k - CLAUSE$ is defined as k boolean variables joined using OR), determine whether there exists a satisfying assignment, that is, whether each variable in ϕ can be assigned a value to make $\phi = 1$. An example of a K-SAT expression with $k = 2$ is $\phi = (x \vee \neg z) \wedge (\neg x \vee y) \wedge (y \vee w)$

For $k = 2$, the problem 2-SAT can be polynomially reduced to the problem of SCC (Strongly Connected Graphs) by considering each variable as a node and each clause as inducing edges. SCC has been solved in polynomial time using `DFS` based approach. Now, given an instance of 2-SAT, replace each clause $a \vee b$ by $(a \vee b \vee z) \wedge (a \vee b \vee \neg z)$ where z is a fresh variable that does not appear elsewhere in the formula. These two expressions are logically equivalent, so this is a valid reduction. This reduction can clearly be performed in polynomial time.

$$\implies \text{2-SAT} \leq_p \text{SCC} \quad \text{and} \quad \text{2-SAT} \leq_p \text{3-SAT}$$

In fact, from any clause $(a_1 \vee a_2 \vee \ldots \vee a_n)$ we can go to $(a_1 \vee a_2 \vee \ldots \vee a_n \vee z) \wedge (\neg z \vee a_1 \vee a_2 \vee \ldots \vee a_n)$

$$\implies \text{2-SAT} \leq_p \text{3-SAT} \leq_p \cdots \iff \text{2-SAT} \leq_p \text{3-SAT} \leq_p \text{K-SAT} \;\; \forall\, k$$

All the K-SAT $\in$ **NP** because the satisfying assignment is checkable in polynomial time. Of course, 2-SAT $\in$ **P** as well as **NP** because of the reduction to SCC.

What about the bigger K-SATs? For a general N-SAT, $(a_1 \vee a_2 \vee \ldots \vee a_n)$ can be rewritten as $(a_1 \vee a_2 \vee b_1) \wedge (\neg b_1 \vee a_3 \vee b_2) \wedge (\neg b_2 \vee a_3 \vee b_3) \wedge \ldots (\neg b_{n-3} \vee a_{n-1} \vee a_n)$. This introduced $n - 3$ new variables $b_1, b_2, \ldots b_{n-3}$ and made it into a 3-SAT.

$$\text{K-SAT} \leq_p \text{3-SAT}$$

Since we proved both K-SAT $\leq_p$ 3-SAT and 3-SAT $\leq_p$ K-SAT, we have proved that K-SAT $=_p$ 3-SAT. With some more effort, it can be shown that all of **NP** is reducible to 3-SAT. Thus 3-SAT is **NP – complete** and with it, all K-SAT.

$$\underbrace{\text{SCC} = \text{2-SAT}}_{\mathbf{P}} \leq_p \underbrace{\text{3-SAT} = \text{4-SAT} = \text{5-SAT} = \cdots}_{\mathbf{NP-complete}}$$

P and **NP – complete** are quite close in difficulty, or are they? It is conjectured (and widely believed) that they are not equal. For more on this topic, see end-of-chapter problems!

3.7 RP

A decision problem belongs to **RP** if there exists a randomised polynomial-time algorithm with input size n satisfying the following properties:

- If the correct answer is `NO`, the algorithm always outputs `NO`.
- If the correct answer is `YES`, the algorithm outputs `YES` with probability at least $\frac{1}{n^c}$, $c > 0$.

Consider an algorithm as mentioned above – it outputs `YES` in the case where correct answer is `YES` with probability $p \geq \frac{1}{n^c}$. Run it k times and answer `YES` if it occurs even once in the k runs otherwise answer `NO`. This new algorithm has the same properties as the original algorithm except p is replaced by $1-(1-p)^k$. So if an algorithm has $p \geq \frac{1}{n^c}$, then for $k = n^c$ the probability will be more than $1 - \frac{1}{e}$.

$$(1 - \frac{1}{n^c})^{n^c} \leq \frac{1}{e} \implies 1 - (1 - \frac{1}{n^c})^{n^c} \geq 1 - \frac{1}{e}$$

As long as k is polynomial in the input size, the overall algorithm is also polynomial time. The point of having a lower bound $\frac{1}{n^c}$ is that we should be able to achieve any desired correctness probability by repeating the algorithm a polynomial number of times times – this may not be possible if our criterion is something like $\frac{1}{2^n}$. Once we get a lower bound of $1 - \frac{1}{e}$, we can do a constant number of repetitions (still keeping the algorithm overall polynomial) to achieve a probability which is as close to 1 as we desire.

$\mathbf{RP_{co}}$

A problem belongs to $\mathbf{RP_{co}}$ if there exists a randomised polynomial-time algorithm satisfying the following properties:

- If the correct answer is `YES`, the algorithm always outputs `YES`.
- If the correct answer is `NO`, the algorithm outputs `NO` with probability at least $\frac{1}{n^c}$, $c > 0$.

To clarify, when we say randomised polynomial time algorithm, we mean an algorithm which makes random choices but whose time complexity is polynomial in the input size. The relevant time complexity here is the actual worst-case time complexity and **not** the expected time complexity.

3.8 ZPP

A decision problem belongs to **ZPP** if there exists a randomized algorithm that has the following properties:

- It outputs the correct answer with probability 1.
- Expected time complexity is polynomial in the input size.

3.9 BPP

A decision problem belongs to **BPP** if there exists a randomized polynomial-time algorithm whose probability of producing the correct answer is at least $\frac{1}{n^c}$, where $c > 0$. Interestingly, it is believed that randomness does not truly add computation power and thus all the classes **P**, **RP**, **ZPP**, **BPP** are the same. This is conjecture, just like **P** $\neq$ **NP**.

3.10 Summary

Class	**Time Complexity**	**YES**	**NO**
P	Polynomial	Always correct	
NP	Verification in Polynomial	Certificate	Need not have certificate
RP	Polynomial	Pr(correct) $\geq \frac{1}{n^c}$	Always correct
ZPP	Expected polynomial	Always correct	
BPP	Polynomial	Overall Pr(correct) $\geq \frac{1}{n^c}$	

3.11 Problems

1 Solving an $\mathbf{NP-complete}$ problem in polynomial time is considered extremely difficult (if not impossible). Let C be the event that a single $\mathbf{NP-complete}$ problem is solved in polynomial time.

(a) Show that $C \implies \mathbf{P} = \mathbf{NP} = \mathbf{NP-complete}$

(b) Show that $\mathbf{P} \subseteq (\mathbf{NP_{co}} \cap \mathbf{NP}) \subseteq \mathbf{NP}$ and thus show $C \implies \mathbf{NP} = \mathbf{NP_{co}}$

2 Modern cryptography relies on the existence of one-way functions. One-way functions are functions which are computable in polynomial time but not invertible in polynomial time.

Consider the function $F : \{0,1\}^n \mapsto \{0,1\}^m$

(a) Show that if F is invertible then $m \geq n$.

(b) Using the above result, show that if $\mathbf{P} = \mathbf{NP}$ then F can not be a one-way function by reformulating the problem of finding the inverse as a decision problem. *Hint: Go bit-by-bit.*

3 Consider an algorithm `ALGO` which solves a decision problem $Q \in \mathbf{ZPP}$ with runtime T whose expected value is E such that $E \leq n^c$ where $c > 0$. Now consider the algorithm `TIMED_ALGO`:

```
TIMED_ALGO
{
    run ALGO with time limit 2*E
    if time limit exceeded
        return NO
    else
        return answer of ALGO
}
```

Let p be the probability that `TIMED_ALGO` gives the correct answer.

(a) Show that $p \geq Pr(T \leq 2E)$

(b) Use the markov inequality to show that $Pr(T > 2E) \leq \frac{1}{2}$

(c) Use the answers to **(a)** and **(b)** to show that the problem is solved by `TIMED_ALGO` with probability at least $\frac{1}{2}$.

(d) Using the result from **(c)**, show that $Q \in \mathbf{BPP}$. Show that this proof works for a general $Q \in \mathbf{ZPP}$, thus proving that $\mathbf{ZPP} \subseteq \mathbf{BPP}$

(e) Use a similar setup to show $\mathbf{ZPP} \subseteq \mathbf{RP} \cap \mathbf{RP_{co}}$

(f) For a general problem Z in $\mathbf{RP} \cap \mathbf{RP_{co}}$, use its $\mathbf{RP}$ solution and its $\mathbf{RP_{co}}$ solution to create a $\mathbf{ZPP}$ solution, thus showing that $\mathbf{RP} \cap \mathbf{RP_{co}} \subseteq \mathbf{ZPP}$. Use this along with **(e)** to prove that $\mathbf{RP} \cap \mathbf{RP_{co}} = \mathbf{ZPP}$

Chapter 4

The Classics

4.1 QuickSort

In general, to sort an input array of size n deterministically using only pairwise comparisons, it can be proved via a decision tree that we need at least $\approx n \log_2 n$ operations. From a time complexity point of view this is $\Theta(n \log n)$ time. The STL library of `C++` uses `QuickSort` in its `std::sort()` function not because it does better than this but because it is parallelisable at the processor level (the details of which we omit since it is not the focus of this book!)

4.1.1 The Algorithm

```
QuickSort(A)
{
    choose an element p from A as the "pivot"
    partition A into A<p, A=p, A>p
    return {QuickSort(A<p) : (A=p) : QuickSort(A>p)}
}
```

Note that `A<p` is our notation for the elements of `A` that are strictly less than `p`. Similarly we can define `A>p` and `A=p` and the final result is the concatenation of the sorted versions of these three sets. For simplicity of analysis, we will assume all elements in the array are distinct since that is the toughest case. So `A=p` is a single element.

4.1.2 Worst Case

A deterministic implementation may always choose the first or last element as pivot. Unfortunately, this can lead to very poor performance on adversarial inputs. For example, if the array is already sorted and we always choose the leftmost element as pivot, the recursion becomes extremely unbalanced, leading to a runtime of $\Theta(n^2)$.

$$T(n) = (n-1) + (n-2) + \cdots + 1 = \Theta(n^2)$$

Thus `QuickSort` has worst case complexity of $\Theta(n^2)$. The standard randomised version instead chooses the pivot uniformly at random. This modification dramatically changes the behaviour of the algorithm – it becomes very unlikely that we repeatedly choose extremely bad pivots.

4.1.3 Expected Time Complexity

Recurrence Relation

Let $T(n)$ be the random variable denoting the runtime of randomised `QuickSort` on an array of size n.

Suppose the pivot is at index k (0-indexed), where $0 \leq k \leq n-1$. Then the recursive subproblems have sizes k and $n-1-k$. Since partitioning itself takes n operations, we obtain a recurrence relation for $T(n)$.

$$T(n) \mid \text{pivot} = k = T(k) + T(n-1-k) + n$$

Because the pivot is chosen uniformly at random, every value of k is equally likely. Taking expectation over all random choices made by the algorithm gives

$$\mathbb{E}[T(n \mid \text{pivot} = k)] = n + \frac{1}{n}\sum_{k=0}^{n-1}(T(k) + T(n-1-k))$$

Taking expectation over all possible values of k and using the law[1] of total expectation gives us a recurrence relation for $\mathbb{E}[T(n)]$ which we denote as $Y(n)$.

$$\mathbb{E}[\mathbb{E}[T(n \mid \text{pivot} = k)]] = \mathbb{E}[T(n)] = n + \frac{1}{n}\sum_{k=0}^{n-1}\mathbb{E}[T(k)] + \mathbb{E}[T(n-1-k)]$$

$$\implies Y(n) = n + \frac{1}{n}\sum_{k=0}^{n-1}Y(k) + Y(n-1-k) = n + \frac{2}{n}\sum_{k=0}^{n-1}Y(k)$$

Changing the variable from $Y(n)$ to $S(n) = \sum_{k=0}^{n} Y(k)$ gives us a recurrence solvable by the method of telescoping summation.

$$\implies S(n) - S(n-1) = n + \frac{2}{n}S(n-1)$$

$$\implies S(n) = \left(1 + \frac{2}{n}\right)S(n-1) + n$$

$$\implies \frac{S(n)}{(n+1)(n+2)} = \frac{S(n-1)}{n(n+1)} + \frac{n}{(n+1)(n+2)}$$

$$\implies \frac{S(n)}{(n+1)(n+2)} - \frac{S(n-1)}{n(n+1)} = \frac{2}{n+2} - \frac{1}{n+1}$$

Summing this recurrence from 1 to n gives the final result as we desired.

$$\frac{S(n)}{(n+1)(n+2)} = \Theta(\log n) \implies S(n) = \Theta(n^2 \log n)$$

$$\implies Y(n) = S(n) - S(n-1) = \Theta(n \log n)$$

Hence the expected runtime of randomised `QuickSort` is $\Theta(n \log n)$

[1] See Appendix A.7

Indicator Variables

Here we explore an alternative method to arrive at the expected runtime of `QuickSort`. Observe that any pair of elements is compared at most once during the entire execution of `QuickSort`. Without loss of generality assume the array is a permutation of $\{1, 2, \ldots, n\}$. Consider the indicator variable I_{ij} indicating whether elements i and j are compared or not.

$$I_{ij} = \begin{cases} 1, & \text{if elements } i, j \text{ are compared} \\ 0, & \text{otherwise} \end{cases}$$

Then the total number of comparisons C is given by the sum of all I_{ij}.

$$C = \sum_{1 \le i < j \le n} I_{ij}$$

Two elements are compared only if one of them becomes the first pivot chosen among all elements lying between them in sorted order. Since all of them are equally likely, we have a 2 out of $j - i + 1$ chance of this happening.

$$\Pr(I_{ij} = 1) = \frac{2}{j - i + 1} \implies \mathbb{E}[C] = \mathbb{E}[\sum_{1 \le i < j \le n} I_{ij}] = \sum_{1 \le i < j \le n} \frac{2}{j - i + 1} = \Theta(n \log n)$$

4.2 QuickSelect

`QuickSort` is used to find the k-th smallest element[2] of an unsorted input array. An obvious method is to sort the array using `MergeSort`[3] or `QuickSort` and then access the k-th element in $\Theta(1)$. But since we only care about one element, it seems inefficient to sort the entire array.

4.2.1 The Algorithm

We pick a pivot p and partition the array as before. Basis the size of the partitions we can figure out which partition the k-th element is in. If it is in the `A=p` partition, then we already have our answer! Else we recurse into that partition and continue this process until we find the k-th element.

```
QuickSelect(A,k)
{
    choose an element p from A as the "pivot"
    partition A into A<p, A=p, A>p
    if size(A<p) >= k                   f// k-th element is in A<p
        return QuickSelect(A<p,k)
    else if size(A<p)+size(A=p) >= k    // k-th element is in A=p
        return p
    else                                // k-th element is in A>p
        return QuickSelect(A>p,k-size(A<p)-size(A=p))
}
```

Intuitively, this is similar to `QuickSort`. However, unlike `QuickSort`, we recurse into only one side – each step discarding a significant fraction of the array. This is a similar approach to Binary Search although not quite same since this does not assume a sorted array.

[2] This is commonly referred to as the k-th order statistic of the array.

[3] `MergeSort` is a divide and conquer deterministic sort algorithm which runs in $\Theta(n \log n)$ time.

4.2.2 Expected Time Complexity

Let X_i denote the size of the search space after i recursive calls of `QuickSelect`. ($X_0 = n$)

At each step, a pivot is chosen uniformly at random and the array is partitioned around it. Depending on the rank we are searching for, `QuickSelect` recurses into exactly one of the two partitions. To obtain a worst-case bound, we imagine that the algorithm always recurses into the larger partition and that all elements are distinct.

If the current search space has size x, and the pivot ends up at position k, then the larger partition has size equal to the maximum of $k-1$ and $x-k$.

Averaging over all pivot positions,

$$\mathbb{E}[X_{i+1} \mid X_i = x] = \frac{1}{x}\sum_{k=1}^{x} \max(k-1, x-k) \leq \frac{3}{4}(x-1) \leq \frac{3}{4}x$$

$$\implies \mathbb{E}[X_{i+1} \mid X_i] \leq \frac{3}{4}X_i$$

Taking expectations and applying the law of total expectation (along with the fact that $X_0 = \mathbb{E}[X_0] = n$) we get an upper bound for $\mathbb{E}[X_i]$.

$$\mathbb{E}[X_{i+1}] = \mathbb{E}[\mathbb{E}[X_{i+1} \mid X_i]] \leq \frac{3}{4}\mathbb{E}[X_i]$$

$$\implies \frac{\mathbb{E}[X_{i+1}]}{\mathbb{E}[X_i]} \leq \frac{3}{4} \implies \frac{\mathbb{E}[X_i]}{\mathbb{E}[X_0]} \leq \left(\frac{3}{4}\right)^i \implies \mathbb{E}[X_i] \leq n\left(\frac{3}{4}\right)^i$$

Let T be the random variable which denotes the number of recursive calls of `QuickSelect`. It is the first i such that $X_i \leq 1$.

Taking one comparison with the pivot as an operation of unit time, we get an upper bound on the expected time of `QuickSelect`.

$$E[T_{run}] \leq \sum_{i=1}^{T} n\left(\frac{3}{4}\right)^i \leq \sum_{i=1}^{\infty} n\left(\frac{3}{4}\right)^i = 3n$$

It is quite obvious that $E[T_{run}] \geq n$ since reading the input takes $\Theta(n)$ time. So the expected time complexity of `QuickSelect` is equal to $\Theta(n)$, which is a significant improvement over the sort-and-access method which takes $\Theta(n \log n)$

4.2.3 Expected Recursion Depth

Define H is the first i such that $X_i \leq 1$. This variable H is by definition the depth of the recursion tree of `QuickSelect`. Why does this matter? The analysis of this quantity will illustrate general methods which will be useful in later chapters.

Since $X_i \leq 1$ is equivalent to $H \leq i$, $H > i$ is equivalent to $X_i \geq 2$. Using Markov's Inequality[4],

$$\Pr(H > i) = \Pr(X_i \geq 2) \leq \frac{\mathbb{E}[X_i]}{2} \leq \frac{n}{2}\left(\frac{3}{4}\right)^i$$

We can also use the trivial upper bound 1 on $\Pr(H > i)$.

[4] See Appendix B.3

$$\Pr(H > i) \leq \min\left(1, \frac{n}{2}\left(\frac{3}{4}\right)^i\right)$$

Using the tail sum formula[5] we can bound the expected value of H.

$$\mathbb{E}[H] = \sum_{i=0}^{\infty} \Pr(H > i) \leq \sum_{i=0}^{\infty} \min\left(1, \frac{n}{2}\left(\frac{3}{4}\right)^i\right)$$

The Splitting Technique

We split the sum into two parts, and bound them or find them separately, thus finding our overall bound. Here we split according to when the min() changes from 1 to $\frac{n}{2}\left(\frac{3}{4}\right)^i$.

$$\mathbb{E}[H] \leq \sum_{i=0}^{m-1} 1 + \sum_{i=m}^{\infty} \frac{n}{2}\left(\frac{3}{4}\right)^i = m + \sum_{i=m}^{\infty} \frac{n}{2}\left(\frac{3}{4}\right)^i \quad \left(\text{where } m = \left\lceil \log_{\frac{4}{3}}\left(\frac{n}{2}\right)\right\rceil\right)$$

For the summation we can factor out the first term, which by our choice of m is less than or equal to 1. The summation left is a geometric series which evaluates to 4.

$$\sum_{i=m}^{\infty} \frac{n}{2}\left(\frac{3}{4}\right)^i = \left(\frac{n}{2}\left(\frac{3}{4}\right)^m\right) \cdot \left(\sum_{j=m}^{\infty}\left(\frac{3}{4}\right)^{j-m}\right) \leq (1) \cdot (4)$$

$$\implies \mathbb{E}[H] \leq m + 4 = \left\lceil \log_{4/3}\left(\frac{n}{2}\right)\right\rceil + 4 \implies \mathbb{E}[H] \leq \Theta(\log n)$$

4.2.4 Geometric Decline Theorem

For a general series of random variables $X_0 = n, X_1, X_2, \ldots$ and T being the first i such that $X_i \leq 1$, given $\frac{\mathbb{E}[X_{i+1}]}{\mathbb{E}[X_i]} \leq p$ with $0 < p < 1$, the expected value of T is at most $\Theta(\log n)$. The proof of this theorem is merely a generalisation of the proof for the recursion depth of `QuickSelect` and is left as an exercise to the reader.

[5] See Appendix A.2

4.3 Problems

1 **(a)** We concluded an upper bound for the expected recursion depth $\mathbb{E}[H] \leq \Theta(\log n)$ for `QuickSelect`. But to justify the expected complexity as $\Theta(\log n)$, show that $\mathbb{E}[H] \geq \Theta(\log n)$

(b) Prove the Geometric Decline Theorem as stated in the text.

2 Let C_n be the number of comparisons made by `QuickSort` on an array of size n. We already established the value of $\mathbb{E}[C_n]$ in the chapter itself.

$$\mathbb{E}[C_n] = \sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1}$$

(a) By grouping together all pairs with the same value of $j - i$, show that

$$\sum_{1 \leq i < j \leq n} \frac{2}{j - i + 1} = 2 \sum_{k=1}^{n-1} \frac{n - k}{k + 1}$$

(b) Let $H_n = \sum_{i=1}^{n} \frac{1}{i}$ be the n-th harmonic number. Using this and the above result show that

$$\mathbb{E}[C_n] = 2(n + 1)H_n - 4n = \Theta(n \log n)$$

Chapter 5

Hashing

5.1 Introduction

A recurring theme in randomized algorithms is the idea of replacing a large object by a much smaller *fingerprint* or *hash* which approximately preserves equality. Instead of comparing two complicated objects directly, we compare their hashes. If the hashes differ, the objects are certainly different. If the hashes are equal, the objects are *probably* equal.

The power of hashing comes from the fact that computing and comparing hashes is often significantly faster than comparing the original objects themselves. It may also be seen as a form of compression that preserves some information about the original object. In data structures like the `unordered set`, hashing helps to achieve constant time complexity (on average) within finite memory constraints.

Usually hashes are many-one functions, meaning that multiple different objects can have the same hash. Two inputs having the same hash is called a *collision*. A false conclusion of equality due to a collision is called a *false positive*. The probability of collision depends on the specific hash function used and the distribution of inputs. A good hash function should minimize the probability of collisions for typical inputs.

5.2 Universal Hash Families

In many applications, we want a low probability of a hash collision because the performance of certain data structures and algorithms depends on it. Since the whole point of a hash function is to go from a large set (the domain U whose size is n) to a smaller set (the range R whose size is m), we will be interested in the case $n \gg m$ where significant collisions are **forced** to occur.

Let U be the domain (whose elements are called keys) and let $[m] = \{0, 1, \ldots, m-1\}$ denote the range. A family of hash functions $\mathcal{H} = \{h : U \to [m]\}$ is said to be *universal* if for every pair of distinct keys $x, y \in U$, the probability of collision for a random $h \in \mathcal{H}$ is no greater than completely random assignment.

$$\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m}, \quad x \neq y$$

The hash functions themselves are completely deterministic, so $h(x) = h(y)$ is either true or false – its truth value is known if h is known. So the probability is coming from the random choice of the hash function h. The key idea is that even if an adversary knows the design of the hash functions in the family, they cannot predict which hash function will be used so they can

not force a collision with high probability.

For example, suppose p is a prime and $U = \{0, 1, \ldots, p-1\}$. For any integers $a \in \{1, \ldots, p-1\}$ and $b \in \{0, \ldots, p-1\}$, define $h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$. The family $\mathcal{H} = \{h_{a,b} : 1 \leq a \leq p-1,\ 0 \leq b < p\}$ forms a universal family.

5.3 Strongly Universal Hash Families

Universal hashing only bounds the probability of collisions. In many applications, we require a stronger guarantee that the hash values of distinct keys should behave as pairwise independent random variables. A family of hash functions $\mathcal{H} = \{h : U \to [m]\}$ is called *strongly universal* or *2-wise independent* if for every pair of distinct keys $x, y \in U$ and for every pair of hash values $i, j \in [m]$,

$$\Pr_{h \in \mathcal{H}}(h(x) = i \text{ and } h(y) = j) = \frac{1}{m^2}$$

In other words, for any two distinct keys, the hash values are independent and uniformly distributed over $[m]$. Strong universality immediately implies universality by putting $i = j$ in the definition since $\frac{1}{m}$ is exactly the upper bound on the probability of collision.

$$\Pr_{h \in \mathcal{H}}(h(x) = h(y)) = \sum_{i=0}^{m-1} \Pr_{h \in \mathcal{H}}(h(x) = i \text{ and } h(y) = i) = \sum_{i=0}^{m-1} \frac{1}{m^2} = \frac{1}{m}$$

5.4 k-Wise Independent Hash Families

A family of hash functions $\mathcal{H} = \{h : U \to [m]\}$ is said to be *k-wise independent* if for every collection of distinct keys $x_1, x_2, \ldots, x_k \in U$ and every choice of values $y_1, y_2, \ldots, y_k \in [m]$, we have

$$\Pr_{h \sim \mathcal{H}}(h(x_1) = y_1, \ldots, h(x_k) = y_k) = \frac{1}{m^k}$$

Equivalently, the random variables $h(x_1), h(x_2), \ldots, h(x_k)$ are jointly independent and each is uniformly distributed over $[m]$.

A standard construction of k-wise independent hash families uses polynomials over finite fields. Let $\mathbb{F}_q$ be a finite field with q elements. Choose coefficients $a_0, a_1, \ldots, a_{k-1} \in \mathbb{F}_q$ uniformly and independently at random, and define the hash function h as

$$h(x) = a_{k-1}x^{k-1} + \cdots + a_1 x + a_0$$

It can be proved that the random variables $h(x_1), h(x_2), \ldots, h(x_k)$ are uniformly and independently distributed over $\mathbb{F}_q$ for all $x_1, x_2, \ldots, x_k \in U$. Hence this construction yields the k-wise independent hash family

$$\mathcal{H} = \{h : h(x) = a_{k-1}x^{k-1} + \cdots + a_1 x + a_0, \text{ all } a_i \in \mathbb{F}_q\}$$

We now have enough theoretical understanding of hashing to see some applications which will hopefully make things concrete!

5.5 Polynomial Multiplication

Suppose we are given two polynomial expressions of degree k, $P(x) = \prod_{i=1}^{k}(a_i x + b_i)$ and $Q(x) = \sum_{j=0}^{k} c_j x^j$. For example, $P(x) = (x-1)(x-2)(x-3)$ and $Q(x) = x^3 - 6x^2 + 11x - 6$. We would like to design a procedure to check whether $P(x) \equiv Q(x)$ or not. Naively expanding the product takes $\Theta(k^2)$ time, and then comparing the resulting polynomials takes $\Theta(k)$ time, so the total verification time is $\Theta(k^2)$.

A more efficient approach is to design a hash function which is easily computable for both $P(x)$ and $Q(x)$. Consider the hash family $\mathcal{H} : \{h_a(P) : a \in Z_{64} = \{0, 1, \ldots, 2^{64} - 1\}\}$ be such that $h_a(P) = P(a)$. What this means is that we evaluate the polynomial $P(x)$ at a value from among the 2^{64} integers in Z_{64} and the result is our hash for the polynomial. From the perspective of the verification procedure, we return "equal" if $h(P) = h(Q)$ and "not equal" otherwise.

For example, let $P(x) = (x-1)(x-2)(x-3)$ and $Q(x) = x^3 - 6x^2 + 11x - 6$, and let us say we draw a random hash and get h_{10}.

$$P(10) = (10-1)(10-2)(10-3) = 504, \;\; Q(10) = 10^3 - 6(10^2) + 11(10) - 6 = 504$$

So we return "equal". Indeed, multiplying both polynomials out by hand proves that they are indeed equal polynomials.

5.5.1 Error Probability

The probability of a collision is

$$\Pr_{h \in \mathcal{H}}(h(P) = h(Q)) = \Pr_{a \in Z_{64}}(P(a) = Q(a))$$

Since $P - Q$ is a polynomial of degree at most k, the probability of a collision for a particular pair of polynomials is the probability of a random number in Z_{64} being a root of $P - Q$ which is at most $\frac{k}{2^{64}}$.

$$\Pr_{a \in Z_{64}}(P(a) = Q(a)) = \Pr_{a \in Z_{64}}(P(a) - Q(a) = 0) \leq \frac{k}{2^{64}}$$

For even large k such as 10^5, the probability of collision is at most 10^{-14} which is quite small. So as is usual for hashing methods, there are no false negatives, only false positives with probability at most 10^{-14}.

5.5.2 Time Complexity

Evaluating P at a random point is first calculating $a_i x + b_i$ for each bracket - this is three arithmetic operations per bracket. After that we calculate the product of all k brackets. The calculation of Q at that same random point is first the calculation of $x, x^2, \ldots, x^k$ which can be done in k multiplications. Coefficient muliplications are at most k multiplications. Addition of the $k+1$ terms is at most $k+1$ additions. Overall this takes at most $6k+1 = \Theta(k)$ operations.

Thus the total verification time is $\Theta(k)$, which is a significant improvement over the naive[1] $\Theta(k^2)$ time.

[1] There exists deterministic techniques to get this down to $\Theta(k \log k)$ time, but they have a large constant factor and also are very difficult to implement.

5.6 Matrix Multiplication

Suppose we are given matrices $A, B, C \in \mathbb{R}^{n \times n}$ and wish to verify whether

$$AB = C$$

Computing the product explicitly requires matrix multiplication. Standard matrix multiplication takes $\Theta(n^3)$ time, while the fastest known algorithms run in roughly $\Theta(n^{2.37})$ time, though they are highly complicated and rarely used in practice.

5.6.1 Freivalds' Algorithm

Choose a random vector $X \in \{0, 1, \ldots, p-1\}^n$, aka of form

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Instead of comparing the matrices themselves, we compute ABX and CX. Since matrix-vector multiplication takes only $\Theta(n^2)$ and we are doing 3 such multiplications - $B \cdot X$, then $A \cdot (BX)$, and $C \cdot X$. Then we compare the two products entry by entry in $\Theta(n)$ time. So the overall procedure is $\Theta(n^2)$ time.

The quantity MX may be viewed as a hash of the matrix M. If two matrices differ, their hashes are unlikely to coincide for a random choice of X.

5.6.2 Error Probability

Let $M = AB - C$. We want to bound the probability that $MX = 0$ when M is nonzero. Choose a prime p, and consider the finite field $\mathbb{Z}_p$.

$$\mathbb{Z}_p = (\{0, 1, 2, \ldots, p-1\}, +_p, \times_p)$$

To avoid making the entries of $M \equiv 0 \pmod{p}$ when it is nonzero over $\mathbb{Z}$ (and thus avoiding making it null in $\mathbb{Z}_p$), we use such a p which is strictly greater than every entry of M. We then choose the random vector $X \in \mathbb{Z}_p^n$ uniformly at random. $(Z_p)^n$ is a vector space[2] of dimension n over $\mathbb{Z}_p$. If M is not the null matrix, the dimension of its nullspace[3] is at most $n-1$.

In general, a d-dimensional vector space over $\mathbb{Z}_p$ contains exactly p^d vectors and here the nullspace has dimension at most $n-1$.

$$\implies |nullspace(M)| \leq p^{n-1}$$

Notice that whenever $MX = 0$ over the integers, we also have $MX = 0$ over $\mathbb{Z}_p$ since $0 \equiv 0 \pmod{p}$. So the set of false positives over $\mathbb{Z}$ is a subset of the set of false positives over $\mathbb{Z}_p$. Thus the probability of a false positive over $\mathbb{Z}$ is at most the probability of a false positive over $\mathbb{Z}_p$.

[2] It may be helpful to brush up on the theory of vector spaces, but you may choose to not bother with the finer details and take my word for it!

[3] Refer Appendix F.4

Since X is chosen uniformly from $(Z_p)^n$, which contains p^n vectors,

$$\Pr(MX = 0 \mid \mathbb{Z}) \leq \Pr(MX = 0 \mid \mathbb{Z}_p) \leq \frac{p^{n-1}}{p^n} = \frac{1}{p}$$

A suitable choice is $p = 2^{64} + 13$ if we are working with 64-bit integers, which gives an astronomically small error probability of about 5.4×10^{-20}.

5.7 A Matrix Problem

Given two $n \times m$ matrices A and B, determine if they are equivalent under row and column permutations. That is, can we permute the rows and columns of A to obtain B? Additionally, the sets of entries of A and B are both equal to $\{1, 2, \ldots, nm\}$.

First we reduce the problem[4] to a simpler one. Consider R(M) to be the set of all R_i where each R_i is the set of values in the i-th row of a matrix M, and similarly C(M) be the set of all C_i. Clearly, if A and B are equivalent under row and column permutations, then R(A) = R(B) and C(A) = C(B) since the row and column sets are preserved under these permutations. This proves that the set of all B such that R(B) = R(A) and C(B) = C(A) contains all matrices equivalent to A under row and column permutations (i.e. the set of all B such that R(B) = R(A) and C(B) = C(A) is a superset of the set of all B equivalent to A under row and column permutations).

Now notice that the number of row and column permutations on A are $n! \cdot m!$ (including the identity permutation). Each of these permutations gives a different matrix since the entries are all distinct. It can be easily shown that the number of matrices with given R(M) and C(M) is $n! \cdot m!$. Since the two sets have the same size and one contains the other, they must be equal! Hence the condition R(A) = R(B) and C(A) = C(B) is necessary as well as sufficient to conclude that A and B are equivalent under row and column permutations. So one way to check this condition is to create a set of sets of both matrices and check equality. But we can do better if we use hashing!

[4]This question was taken from Codeforces. The editorial indicates that the intended solution was deterministic, but where's the fun in that?

5.7.1 The Algorithm

```
pair_hash(x,y){return x*(2^32)+y;}

hash_matrix(mat, orientation)
{
        unordered_set s;
        n=mat.num_rows, m=mat.num_cols;
        if(orientation=="row")
        {
                loop(i from 0 to n-1)
                {
                    sum=0,squaresum=0;
                    loop(j from 0 to m-1)
                    {
                int x=first[i][j];
                sum+=x;
                squaresum+=x*x;
            }
            s.insert(pair_hash(sum,squaresum));
        }
        else
        {
            loop(i from 0 to m-1)
            {
                sum=0,squaresum=0;
                loop(j from 0 to n-1)
                {
                    int x=first[j][i];
                    sum+=x;
                    squaresum+=x*x;
                }
                s.insert(pair_hash(sum,squaresum));
            }
        }
        return s;
}
unordered_set RA=hash_matrix(A, "row");
unordered_set RB=hash_matrix(B, "row");
unordered_set CA=hash_matrix(A, "column");
unordered_set CB=hash_matrix(B, "column");

print((RA==RB)&&(CA==CB));
```

5.7.2 Error Probability

To obtain a more efficient solution, each row is hashed to a pair of values (sum, squaresum), where sum and squaresum denote the sum and sum of squares of elements in the row respectively.

$$h(R_i) = \left(\sum_{j=1}^{m} a_{ij}, \sum_{j=1}^{m} a_{ij}^2 \right)$$

We defined a custom hash for a pair (x, y) as $H(x, y) = x \cdot 2^{32} + y$ so we can combine the two values into a single hash value.

$$\mathbf{H}(R_i) = H(h(R_i))$$

where x and y are the computed row statistics. This choice of pair hash is popular for 32 bit integers, as it combines the two values into a single 64-bit integer guaranteeing uniqueness. We insert the hashes of all rows of matrix A into an unordered set and similarly for matrix B. If the two sets are equal, we conclude that the row sets of the two matrices are equal. The exact same procedure is repeated for the column sets of the two matrices. If both the row and column sets are equal, we conclude that the two matrices are equivalent under row and column permutations. While it is quite complicated to give a rigorous proof of correctness, the intuition is that the additional constraint on the entries of the matrices being distinct integers from 1 to nm makes a false positive extremely unlikely.

5.8 A Sliding Window Problem

Two integer arrays x and y, each of size n, are given along with an integer k ($1 \leq k \leq n$). For every contiguous subarray (window) of size k, determine whether the corresponding windows in x and y contain the same multiset of elements at every position. More precisely, for every index i such that $0 \leq i \leq n-k$, consider the multisets $\{x_i, x_{i+1}, \ldots, x_{i+k-1}\}$ and $\{y_i, y_{i+1}, \ldots, y_{i+k-1}\}$ and check whether they are identical for all i.

5.8.1 The Algorithm

```
bool ok=1;
sumX=0,sumY=0; //X is for array x, Y is for array y
sumX2=0,sumY2=0;
loop(i from 0 to k-1)
{
    sumX+=x[i];
    sumY+=y[i];
    sumX2+=x[i]*x[i];
    sumY2+=y[i]*y[i];
}
if(!((sumX==sumY)&&(sumX2==sumY2)))
        ok=0;
loop(i from k to n-1)
{
    if(!((sumX==sumY)&&(sumX2==sumY2)))
                ok=0;
    sumX+=x[i]-x[i-k];
    sumX2+=x[i]*x[i]-x[i-k]*x[i-k];
    sumY+=y[i]-y[i-k];
    sumY2+=y[i]*y[i]-y[i-k]*y[i-k];
}

print(ok);
```

5.8.2 Time Complexity

We used the hash $\left(\sum a_i, \sum a_i^2\right)$ for each window of size k in both arrays so we can compare the two windows in $\Theta(1)$ time. The additional benefit is that the change in the hash when the window slides by one position can also be computed in $\Theta(1)$ time[5]! Thus the overall time complexity is $\Theta(n)$, which is optimal since we have to at least read all the elements of both arrays.

5.8.3 Error Probability

It is quite difficult to show an upper bound on the probability of a false positive for this problem, but it is quite intuitive that for integer valued arrays it is very unlikely for both quantities to be same. Despite my best efforts, I was unable to find a counterexample for the overall solution of this problem...

[5]This kind of a hash is called a *rolling hash*.

5.9 Problems

1 Consider a general universal hash family $\mathcal{H}$ with all hash functions having size of domain $|U| = n$ and range $|R| = m$ with $n \gg m$.

(a) Let $x, y \in U$ and $h \in \mathcal{H}$. Define the indicator variable $I(h, x, y)$ as $I(h, x, y) = 1$ if $h(x) = h(y)$ and $I(h, x, y) = 0$ otherwise. Show that the overall collision probability $Q(\mathcal{H})$ for the hash family over all keys is

$$Q(\mathcal{H}) = \frac{\sum_{h \in \mathcal{H}} \left(\sum_{x \neq y} I(h, x, y) \right)}{\binom{n}{2} \cdot |\mathcal{H}|}$$

(b) For a particular $h \in \mathcal{H}$, let s_i be the number of keys mapped to bucket i. Show that the number of colliding pairs under h is

$$\left(\sum_{x \neq y} I(h, x, y) \right) = C(h) = \sum_{i=0}^{m-1} \binom{s_i}{2}$$

(c) Use $\sum_{i=0}^{m-1} s_i = n$ and Jensen's inequality suitably to show that

$$C(h) \geq \frac{n(n-m)}{2m}$$

(d) Use the conclusions from **(a)**, **(b)** and **(c)** to show that

$$Q(\mathcal{H}) = \frac{\sum_{h \in \mathcal{H}} C(h)}{\binom{n}{2} \cdot |\mathcal{H}|} \geq \frac{n-m}{m(n-1)}$$

(e) Show that the collision probability for a particular pair of keys x, y is

$$\Pr_{h \in \mathcal{H}}(h(x) = h(y)) = \frac{\sum_{h \in \mathcal{H}} I(h, x, y)}{|\mathcal{H}|}$$

(f) Use the result from **(e)** and rearrange the expression for $Q(\mathcal{H})$ from **(a)** to show that

$$Q(\mathcal{H}) = \frac{\sum_{x \neq y} \left(\Pr_{h \in \mathcal{H}}(h(x) = h(y)) \right)}{\binom{n}{2}} \geq \frac{n-m}{m(n-1)}$$

(g) Use the result from **(f)** and apply the pigeonhole principle to conclude that there exists a pair of keys x^*, y^* such that[6]

$$\Pr_{h \in \mathcal{H}}(h(x^*) = h(y^*)) \geq \frac{n-m}{m(n-1)} \approx \frac{1}{m}$$

2 For hashes of form $H(x) = (ax + b) \bmod c$, why is it preferred to have c as a prime number? *Hint: Think of the arithmetic progression (a very common occurrence in data) with common difference d - what happens when d is not coprime with c?*

[6] This conclusion shows why universal hash families operating on a large domain have the requirement of collision probability upper bound $\frac{1}{m}$, because the guarantee is optimal for $n \gg m$. Any guarantee smaller than that is impossible because the pigeonhole principle shows the existence of a pair of keys which violate any guarantee smaller than $\frac{1}{m}$.

3 An array is called *good* if its elements can be rearranged into consecutive integers. More precisely, an array b of length l is good if its elements can be rearranged into $x, x+1, \ldots, x+l-1$ for some integer x. Consider a pair of disjoint subarrays of equal length l of an array to be *noice* if they both are good and their concatenation is good. The task is to print the maximum value of l for a given array A.

For example, for the array $A = [2, 1, 5, 3, 4]$, the maximum value of l is 2 because the subarrays $[2, 1]$ and $[3, 4]$ are good and their concatenation $[2, 1, 3, 4]$ is also good and we can not have a bigger l because the total length is 5.

(a) Let $A = [a_0, a_1, \ldots, a_{l-1}]$ and $S(A) = \sum_{i=0}^{l-1} a_i$, $Q(A) = \sum_{i=0}^{l-1} a_i^2$. Show that A is good if

$$lQ(A) - S^2(A) = \frac{l^2(l^2-1)}{12}$$

(b) Write a loop which uses the above property to implement a rolling hash to detect all good subarrays of length l in an array of size n in overall $\Theta(n)$ time.

(c) In a noice pair of subarrays A_1 and A_2 of length l each, show that

$$|S(A_1) - S(A_2)| = l^2$$

(d) Consider that for a given l, we do a single linear pass over the array in order to detect at least one noice pair of subarrays of length l each if such a pair exists. Show that it suffices to store a mapping of $S(A)$ to the earliest occurence of that $S(A)$ seen among the good subarrays, and at each index i check whether $S(A) - l^2$ or $S(A) + l^2$ has been seen before. Think of how we can use the information of the earliest occurence to ensure that the two subarrays are disjoint.

(e) Finalise the above ideas into a complete algorithm of time complexity at most $\Theta(n^2 \log n)$ using ordered maps or $\Theta(n^2)$ using unordered maps for an input array of size n.

(f) Use the idea that variance is shift-invariant (adding a constant to all elements does not change the variance) to show the result from **(a)** without brute calculation.

$$l \cdot Q(A) - S^2(A) = l \cdot var(\{x, x+1, \ldots, x+l-1\})$$

Chapter 6

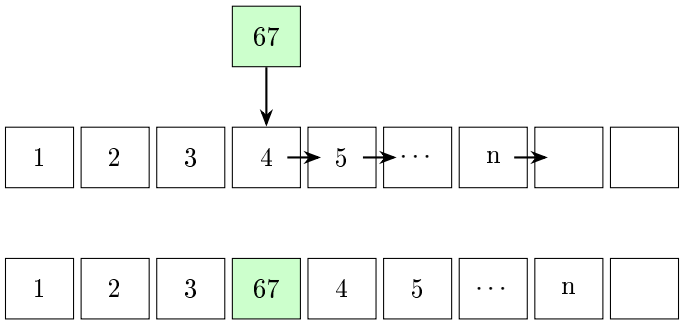
Data Structures

6.1 Skip Lists

Arrays

Suppose we store the following array in contiguous memory:

Suppose we wish to insert the element 67 between 3 and 4. Since arrays occupy contiguous memory, every element beginning from 4 must be shifted one position to the right in order to create space for the new element.

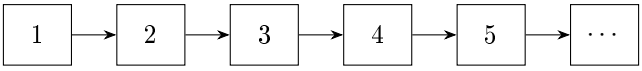


Insertion into an array requires shifting $\Theta(n)$ elements in the worst as well as average case.

How are Linked Lists any better?

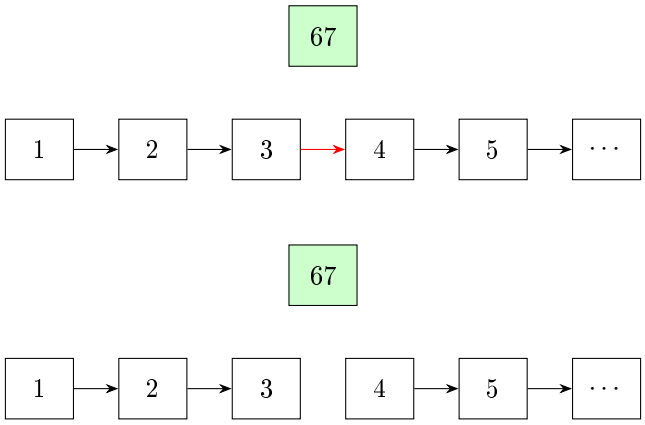
Unlike arrays, linked lists do not store elements contiguously in memory. Instead, each element stores `{Value,Next}`, where `Next` is a pointer to the next element.

So a linked list containing the elements $1, 2, 3, 4, 5, \ldots, n$ may be represented as

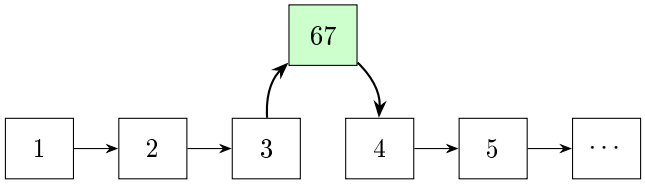


To insert an element 67 between say 3 and 4, we simply create a new node containing 67 and link it between 3 and 4.

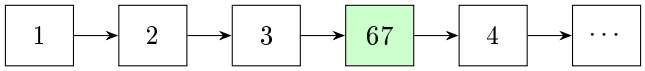
Step 1: Remove the old connection



Step 2: Create new connections



This linked list is exactly what we desired and can be equivalently represented as –



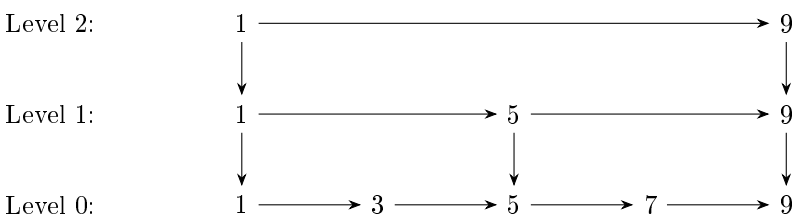
We used a constant number of operations to insert an element into the list. Thus insertion is $\Theta(1)$ time complexity.

Why Skip Lists?

Although insertion is $\Theta(1)$ time complexity, searching for an element in linked lists is $\Theta(n)$ time complexity because we have to traverse the whole list. Even if the linked list is "sorted", we can not binary search since that requires random access in constant time, which a linked list can not provide.

So the idea of skip lists is to bypass that limitation by adding “express lanes” to a linked list. Instead of traversing every element one-by-one, we maintain additional higher-level lists that skip (hence the name) over many elements at once.

Consider the following skip list –



Higher levels contain fewer elements and allow us to jump across large portions of the list quickly.

6.1.1 The Structure

The presence of an element in a level is determined by a random choice. An element occurs in level i with probability 2^{-i}. This means every element must occur in level 0.

Consequently, Pr(node reaches level k) $= \frac{1}{2^k}$. Thus, the expected number of nodes in level k is $\frac{n}{2^k}$.

Level	Expected Number of Nodes
0	n
1	$n/2$
2	$n/4$
3	$n/8$
$\vdots$	$\vdots$

6.1.2 Expected Height

Let H be the height of a skip list with n independent nodes, where each node is promoted to the next level with probability 1/2. Then it is easy to express the probability that height H is greater than or equal to h.

$$\Pr(H \geq h) = 1 - (1 - 2^{-h})^n$$

We use the tail-sum formula to find the expected value of H.

$$\mathbb{E}[H] = \sum_{h=1}^{\infty} \Pr(H \geq h) = \sum_{h=1}^{\infty} \left(1 - (1 - 2^{-h})^n\right)$$

Method 1: Splitting Method

Step 1: Split the expectation

Let $h_o = \lfloor \log_2 n \rfloor$ be the splitting point for the sum.

$$\mathbb{E}[H] = \sum_{h \leq h_0} \Pr(H \geq h) + \sum_{h > h_0} \Pr(H \geq h) = S_1 + S_2$$

Step 2: Bounding S_1

Consider the function $f(h) = \Pr(H \geq h) = 1 - (1 - 2^{-h})^n$. Since $f(h)$ is a decreasing function, we can bound it by $f(h_0)$ and $f(0)$.

$$f(h_o) \leq f(h) \leq f(0) \implies \sum_{h=1}^{h_o} f(h_o) \leq \sum_{h=1}^{h_o} f(h) \leq \sum_{h=1}^{h_o} f(0)$$

Since $f(0) = 1$, and $f(h_o) \geq 1 - (1 - \frac{1}{n})^n \geq 1 - \frac{1}{e}$ we get bounds for S_1.

$$h_o(1 - \frac{1}{e}) \leq S_1 \leq h_o$$

Step 3: Bounding S_2

Using Bernoulli's inequality[1], $(1-x)^n \geq 1-nx$ so we can bound $f(h)$.

$$1-(1-2^{-h})^n \leq n \cdot 2^{-h} \implies S_2 \leq \sum_{h>h_0} n \cdot 2^{-h} \leq 2$$

Step 4: Combining bounds

$$(1-e^{-1})h_0 + 2 \leq \mathbb{E}[H] \leq h_0 + 2$$

Since $h_0 \approx \log_2 n$, we get $\mathbb{E}[H] = \Theta(\log n)$

Method 2: Geometric Decline Theorem

Recall that using the markov inequality, we proved that the height of the recursion tree for `QuickSelect` is at most $\Theta(\log n)$

Using the same technique, we can show that if a sequence of nonnegative random variables satisfies $\frac{\mathbb{E}[X_{i+1}]}{\mathbb{E}[X_i]} \leq p$ $(0 < p < 1)$ and T denotes the first index for which $X_T \leq 1$, then

$$\mathbb{E}[T] \leq \Theta(\log(X_0))$$

Applying the Theorem

Let X_i denote the number of nodes present at level i of the skip list. Since every node is promoted independently with probability 1/2,

$$\mathbb{E}[X_i] = n\left(\frac{1}{2}\right)^i \implies \frac{\mathbb{E}[X_{i+1}]}{\mathbb{E}[X_i]} = \frac{1}{2}$$

The height of the skip list is precisely the highest level containing at least one node. Equivalently, if $H = \min\{i : X_i \leq 1\}$ then the height is H. Applying the Geometric-Decline theorem with $p = \frac{1}{2}$ gives us the desired upper bound.

$$\mathbb{E}[H] \leq \Theta(\log n)$$

This second method provokes a natural question – what was the point of the first method? In the second method we did not prove a lower bound! Although even in further analyses we won't need the lower bound, it is good to have established that it is exactly $\Theta(\log n)$. Also, we revised the splitting technique for finding the complexity of an expression, which is a useful tool in general.

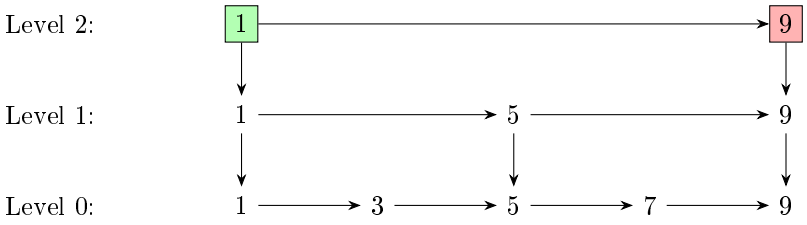
6.1.3 Search

Suppose we wish to search for the element 7. We begin at the top left level and move right while the next element is ≤ 7. Once that element is strictly greater than 7, we move down to the next level. Repeat until we reach the bottom level.

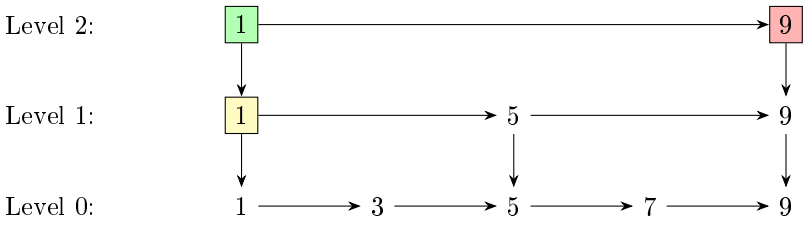
[1] Refer Appendix B.4

```
contains(x)
{
    current = top-left node
    while current exists
    {

        while current.right exists and is <= x
            current = current.right

        if current.value == x
            return FOUND

        current = current.down
    }

    return NOT FOUND
}
```

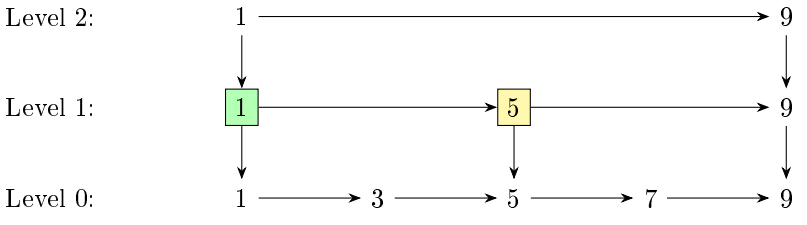
Step 1: Start at top, check right



Since on the right we have an element strictly greater than 7, we intend to move down.

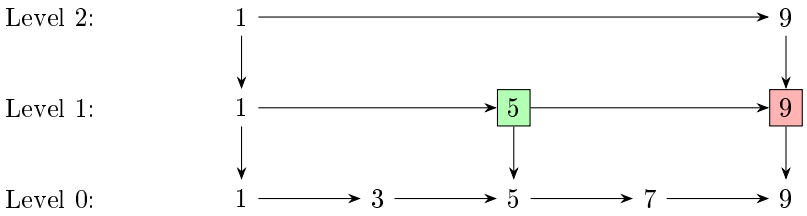


Step 2: Moved down, now check right

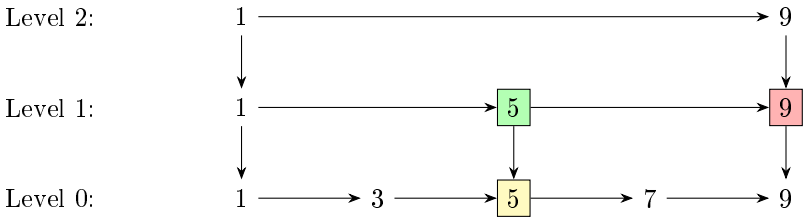


Since $5 \leq 7$, we intend to move right.

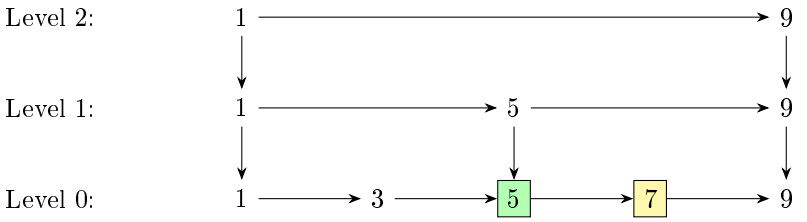
Step 3: Moved right, check right



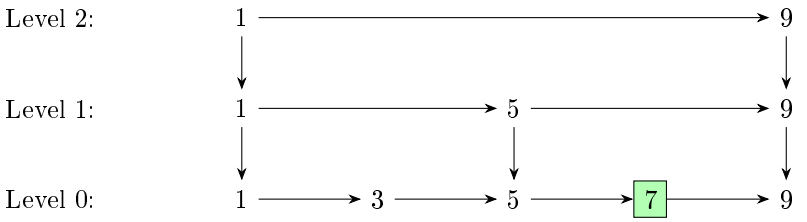
Since $9 > 7$, we intend to move down.



Step 4: Moved down, check right



Step 5: Moved right, found the target!



We move right to 7 and locate the desired element.

6.1.4 Insertion

One may wonder why skip lists use randomization at all. Could we not deterministically place

- every second element in level 1,
- every fourth element in level 2,
- every eighth element in level 3,
- and so on?

While this would indeed support logarithmic search complexity, insertions and deletions would become expensive. For example, inserting a new element near the beginning of the list changes which elements are the second, fourth, eighth, ... elements of the list.

Consequently, many levels may require rebuilding. Randomization avoids this problem by assigning heights independently to each node. Thus, updates are $\Theta(height) = \Theta(\log n)$ while the structure stays balanced in expectation.

The insertion operation in a skip list closely mirrors the search procedure. In fact, insertion is essentially a search followed by a series of pointer modifications. To insert a value x, we first perform the standard search starting from the topmost level. At each level, we move right until we either find x or encounter the first node with a key greater than x (or reach the end of the list).

If the key x is found during this traversal, then we do nothing. Otherwise, the search naturally terminates at the position where x should logically appear in the bottom level. This position is determined by the first "blocking" node whose key is greater than x, or by reaching a null pointer at the end of the list.

Once this position is identified, insertion proceeds as follows. We insert x immediately to the left of the blocking node at level 0. After placing it in the bottom layer, we decide randomly (typically with probability 1/2) whether the node should also appear in higher levels. If it is promoted, we move upward level by level, and at each level we again insert it in the same relative horizontal position, maintaining alignment with the structure discovered during the search phase.

Thus, the update process reuses the search path and only modifies pointers locally along that path. This is what ensures that skip list insertion remains logarithmic in expected time.

6.2 Treaps

Treaps are randomized balanced binary search trees. They combine two ideas:

1. The *binary search tree* property, which allows efficient searching.
2. The *heap* property, which is used to maintain balance.

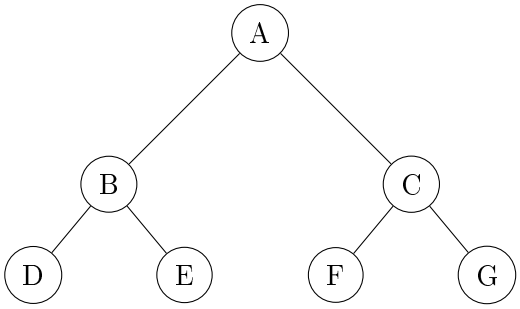
In a simple binary search tree, the insertion order matters. Certain orders can make the height $\Theta(n)$ while others can make it $\Theta(\log n)$. But if we average over all insertion orders, the height is $\Theta(\log n)$. So it seems to avoid adversarial inputs we should randomise the insertion order and the tree will be balanced in expectation! But the whole point of the data structure is we can insert elements at any time so we can not really manipulate the input order.

This brings us to the idea of assigning each element an index (called a priority) just before insertion and then whenever an element is inserted using the standard binary search tree way, we modify the existing tree to pretend as if it was inserted in increasing order of the priorities. To achieve this, we define an operation called *rotation*.

A *rotation* preserves the binary search property while "changing" the order of insertion. Thus we ensure a random insertion order and the treap remains balanced in expectation. This is a relatively simple implementation as compared to Red-Black Trees which stay balanced using rotations as well as recolorings.

6.2.1 Binary Trees

A binary tree is a rooted tree in which every node has at most two children, called the left child and the right child. For implementation purposes, the `struct Node` has three pieces of data – its own value, the pointer to its left child, and the pointer to its right child. If the child does not exist, the pointer is set to `nullptr`. We may show an empty node if there is no child for clarity in some figures.

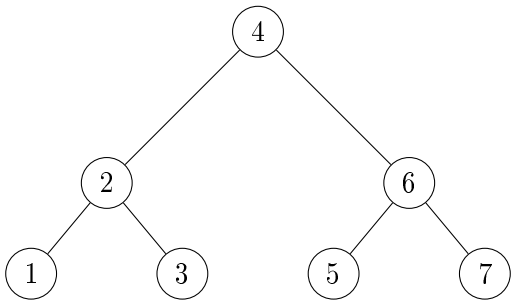


The topmost node is called the **root**. A node with no children is called a **leaf**. Every node together with all of its descendants forms a **subtree**.

6.2.2 Tree Traversal

A traversal specifies the order in which nodes are visited. The most important traversal for our purposes is the **In-Order Traversal**.

```
IOT(x)
{
    if (x is null)
        return

    IOT(x.left)
    print(x.value)
    IOT(x.right)
}

IOT(root) \\starting the recursive traversal from the root
```



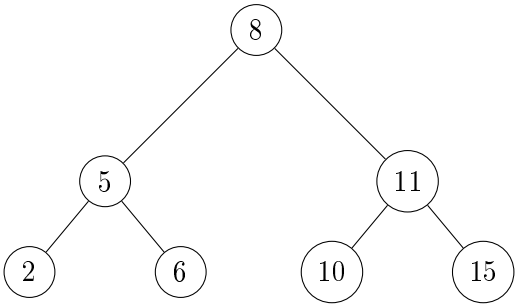
Convince yourself that the output for the above tree will be `1 2 3 4 5 6 7`. In-Order Traversal is most commonly seen in `Depth-First Search`, popularly known as `DFS`.

6.2.3 Binary Search Trees

The value stored at each node is referred to as its **key**. A binary search tree (BST) is a binary tree in which every non-leaf node satisfies the following properties:

- All keys in the left subtree are smaller than the node's key.
- All keys in the right subtree are larger than the node's key.

Any tree which has the above two properties is said to have the **BST** property.



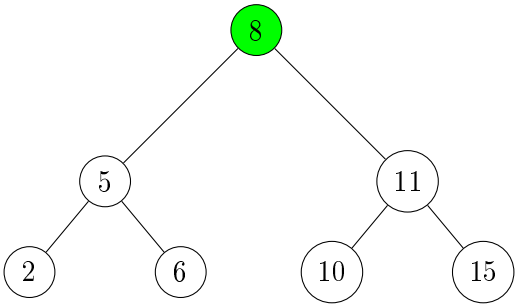
With some thought it can be proved that an In-Order Traversal of a BST lists all keys in sorted order.

Search

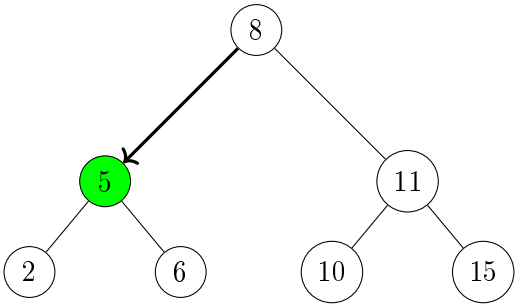
Searching for a key proceeds exactly as in binary search. At each node we compare the desired key with the current key and move either left or right. If the tree is balanced, the tree is the same as the recursion tree of a normal binary search on a sorted array ...

```
Search(key,node)
{
    if (node is null)
        return false

    if(key < node.value)
        return Search(key,node.left)
    if(node.value == key)
        return true
    if(key > node.value)
        return Search(key,node.right)
}

Search(x,root) \\Search the key x, starting from the root
```

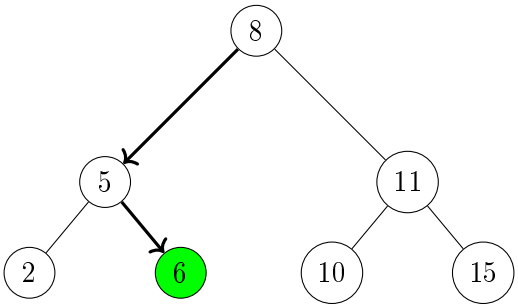
Let us say we are searching for 6 in the tree we saw earlier.



Since 8 is greater than 6, we move left.



Since 5 is less than 6, we move right.



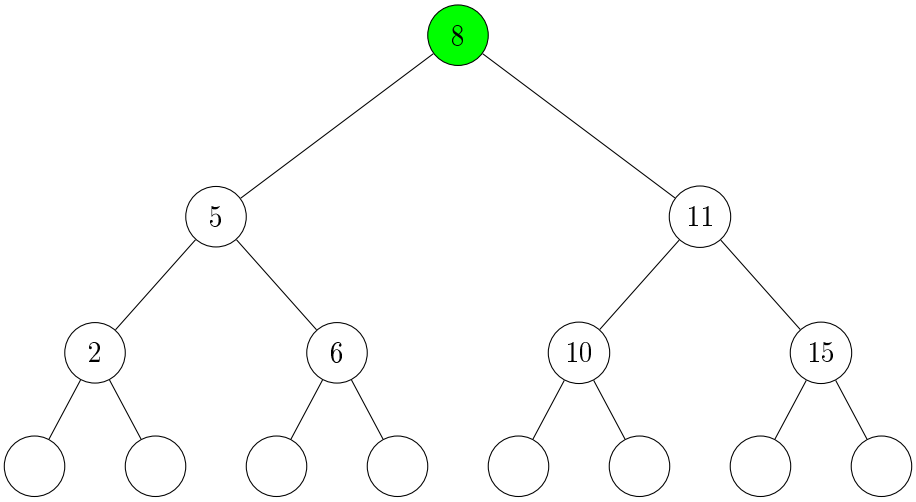
We found it! If we had instead been looking for 7, we would have taken the same path and ended up at the right child of leaf 6 (which is of course, an empty node) so we would have concluded that 7 is not in the tree.

Insertion

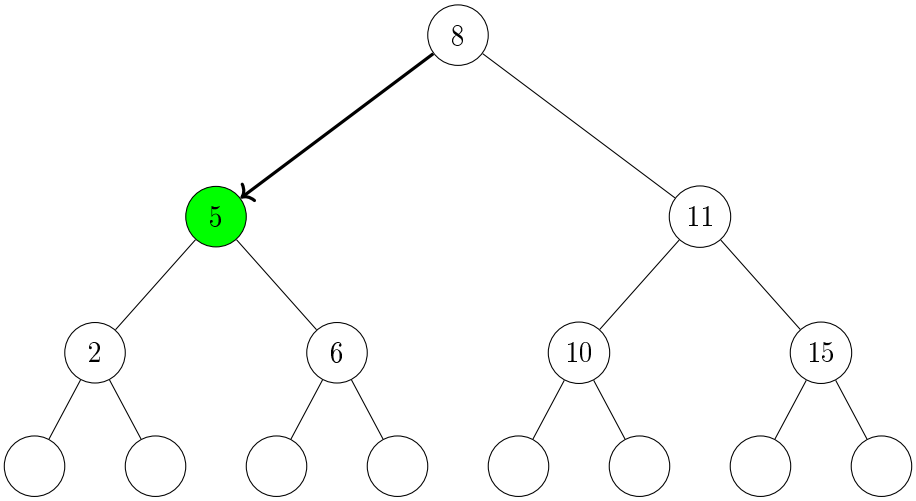
Insertion is similar to binary search. At each node we compare the key to be inserted with the current key and move either left or right. If we find the key in the tree, we do nothing. If we reach an empty node we insert the key there.

```
Insert(key,node)
{
    if (node is null)
    {
        node = new Node(value=key, left=null, right=null)
        return
    }

    if(key < node.value)
        Insert(key,node.left)

    if(node.value == key)
        return

    if(key > node.value)
        Insert(key,node.right)
}

Insert(x,root) \\Insert the key x, recurse starting from the root
```

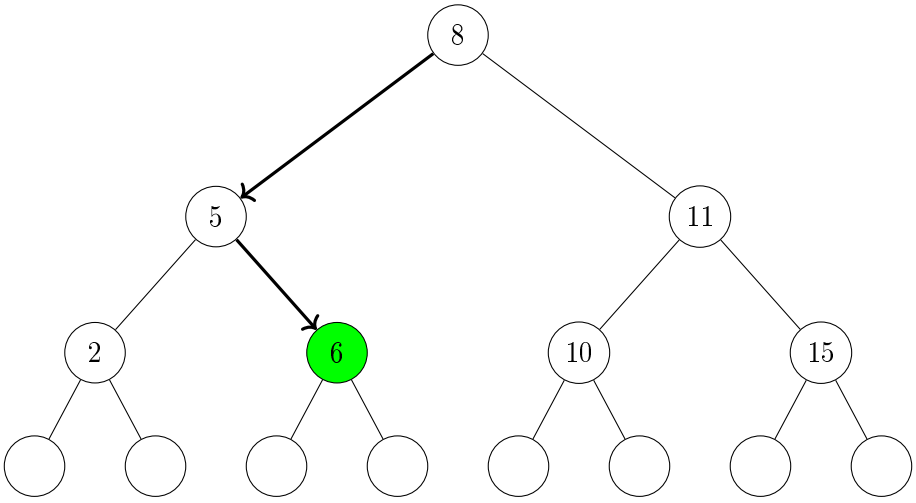
Let us say we want to insert 7 in the tree we saw earlier.



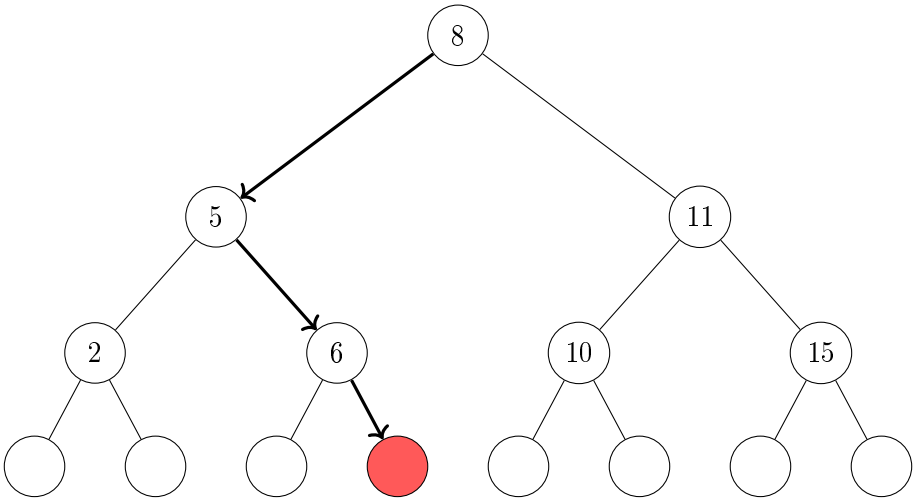
Since 8 is greater than 7, we move left.



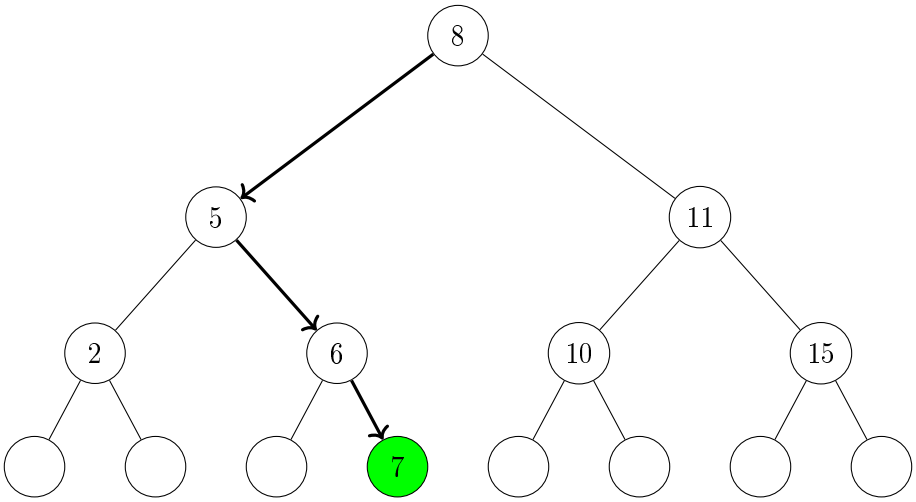
Since 5 is less than 7, we move right.



Since 6 is less than 7, we move right.

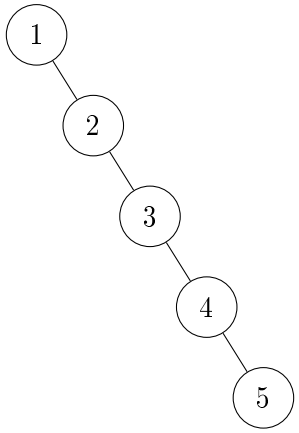


Since the node is empty, we create a new node and put 7 in it.



6.2.4 Balancing

The efficiency of a BST depends on its height. If the tree has height h, then search, insertion and deletion requires worst-case $\Theta(h)$ time. Ideally we would like $h = \Theta(\log n)$ where n is the number of stored keys. Unfortunately, a BST can become highly unbalanced. For example, inserting the keys $1, 2, 3, 4, \ldots n$ in this order produces a tree with height n.



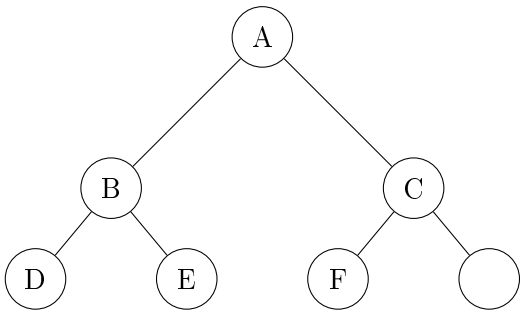
Example with n = 5

This tree is essentially a linked list. Its height is n and consequently all operations require $\Theta(n)$ time. To avoid this degeneration we need a mechanism for changing the shape of a tree while preserving the BST property.

6.2.5 Min-Heaps

A min-heap is a binary tree that satisfies two important properties:

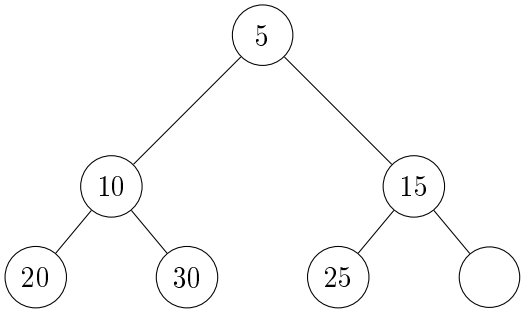
1. **Complete**: Every level is completely filled except possibly the last level, which is filled from left to right.



   The above figure shows a complete binary tree.

2. **Min-Heap Property**:

   - In a **Min-Heap**, every node is less than or equal to its children.



     The above figure shows a min-heap. It can be easily proved that the root has the smallest value in a min-heap.

A min-heap can support the `push` and `pop` operations.

Popping

The `pop()` function returns the root of the heap and removes it from the heap. To maintain the heap, we must then move some values up the tree. Logically, the smaller child should be moved up to the root. This creates a vacancy in the subtree in which the child was pushed up. That subtree now behaves like a smaller version of the original problem – a heap from which the root was removed. So we can recursively push the smaller child up until we hit empty node.

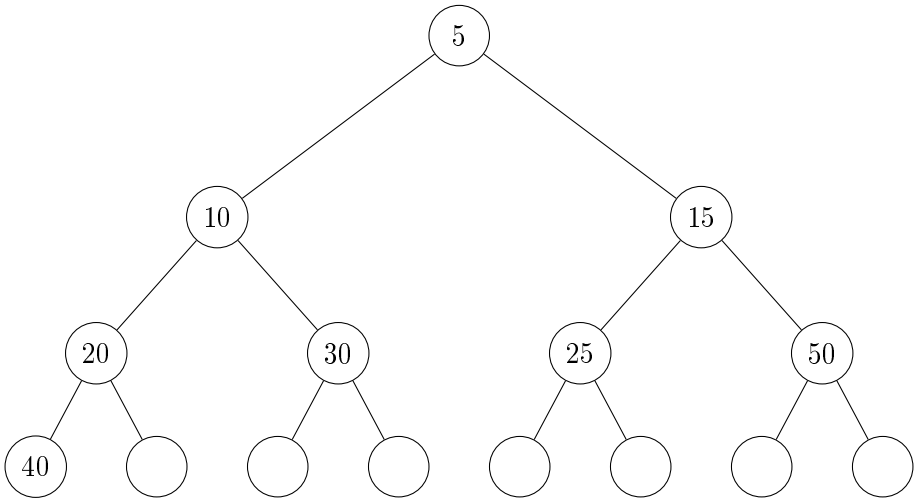
```
pop(root)
{
        ans=root.value
        root.value=null
        if(root.left < root.right)
                root.value=pop(root.leftChild)
        else
                pop(root.rightChild)
        return ans
}
```

Pushing

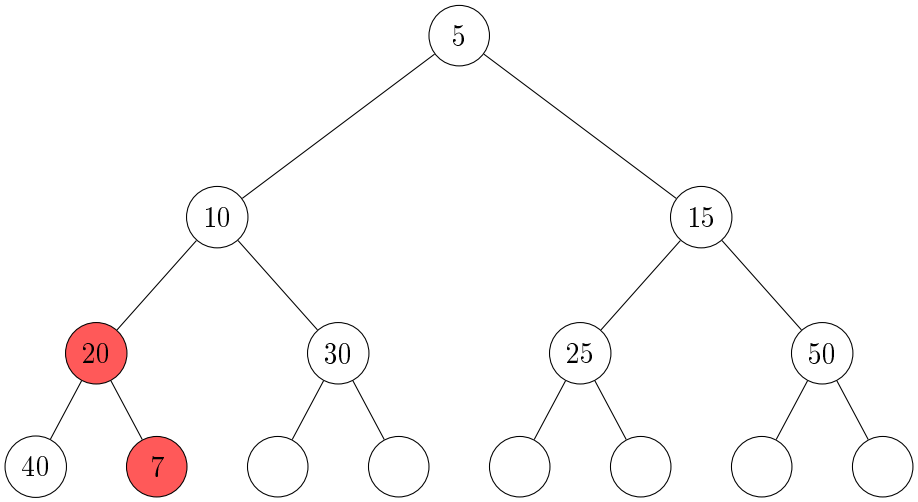
The `push()` function adds a new value to the heap by putting the new value at the bottom of the tree and then pushing it up until heap property is restored.

```
push(x)
{
        curr=bottomLeftMostNode
        curr.value=x

        while(curr.value<curr.parent.value)
        {
                swap(curr.value,curr.parent.value)
                curr=curr.parent
        }
}
```

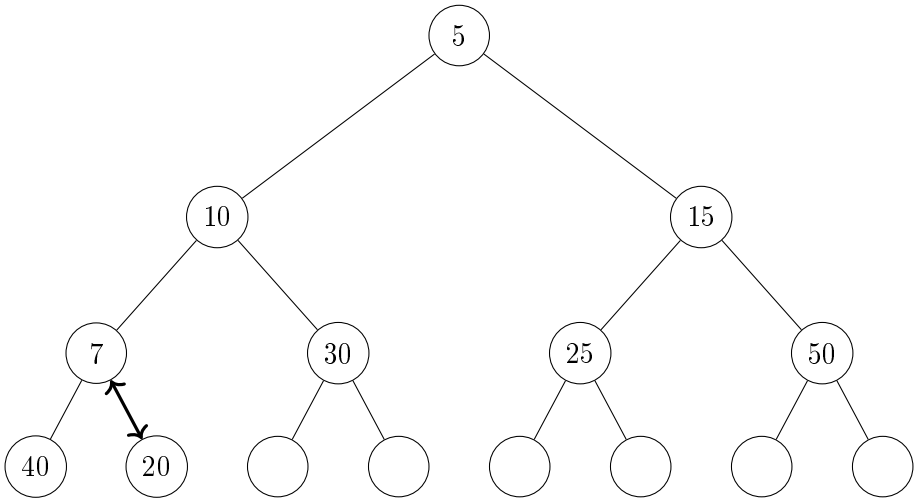
Let us insert 7 into the following min-heap.



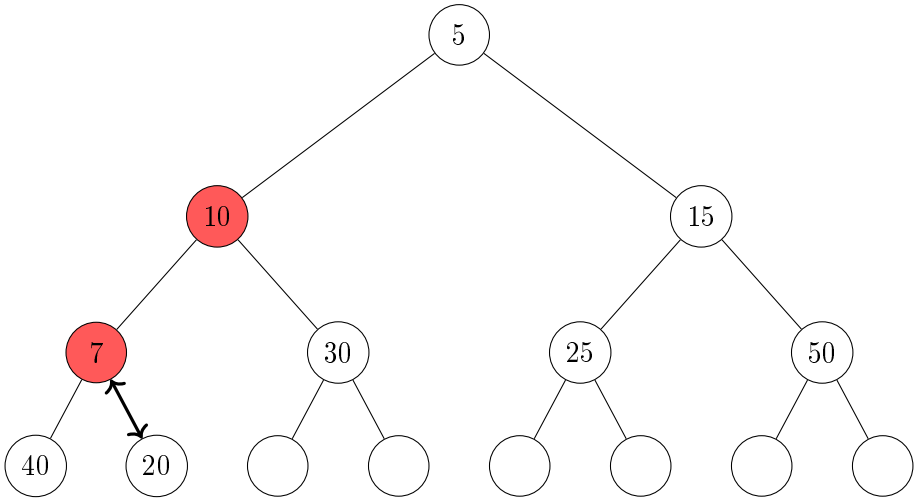
Insert 7 in bottom left most node. We detect the violation $20 > 7$.



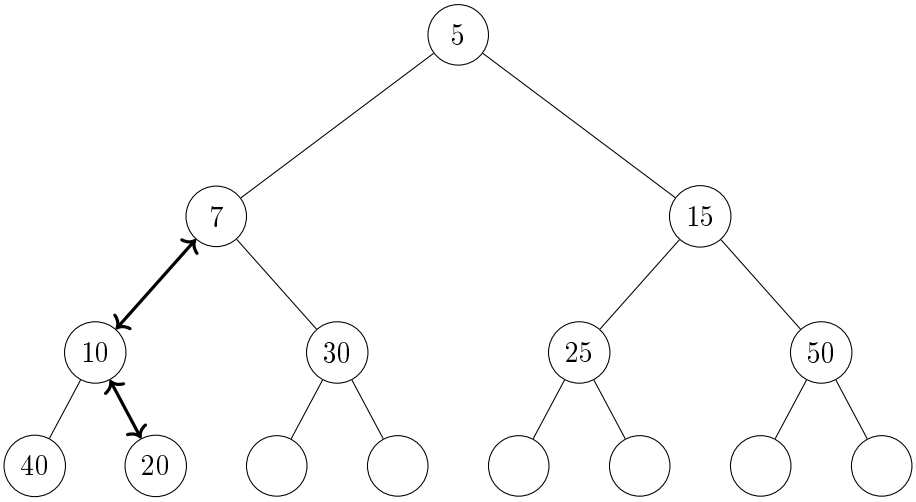
We swap them to solve the violation.



We detect the violation $10 > 7$.



We swap them to solve the violation.

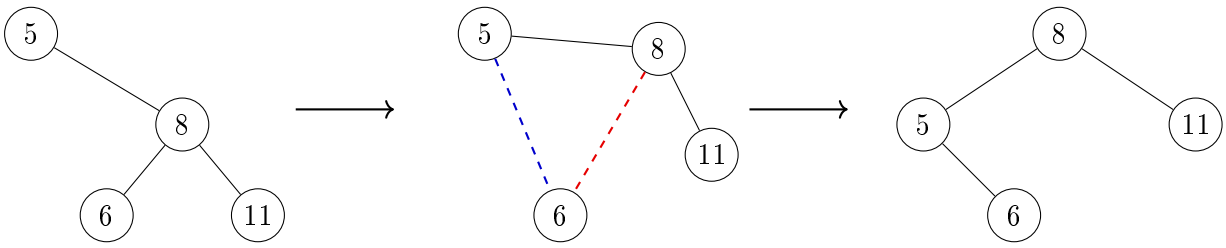


It can be proved that when there is no more violation involving the inserted value, there is no more violation in the tree. Thus the heap property is restored.

6.2.6 Rotations

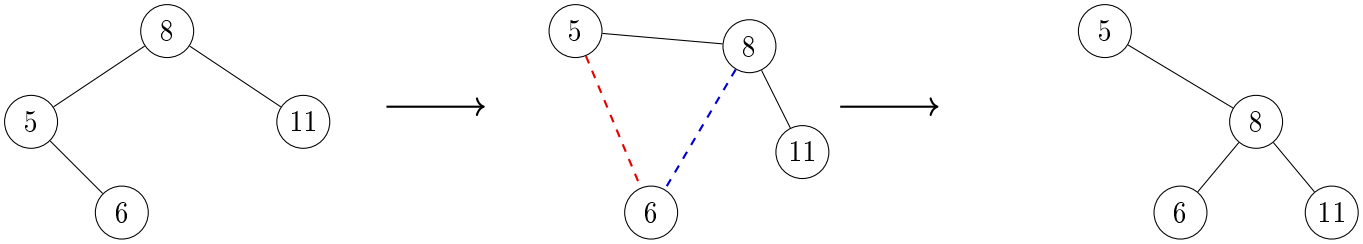
These are operations that transform a tree into another tree with the same inorder traversal. From an undirected graph perspective the only change that occurs is one edge is deleted and one is added. But from a rooted tree perspective 2 father-child relationships are changed. There are two types of rotations (left and right) which are inverse of each other.

Left Rotation



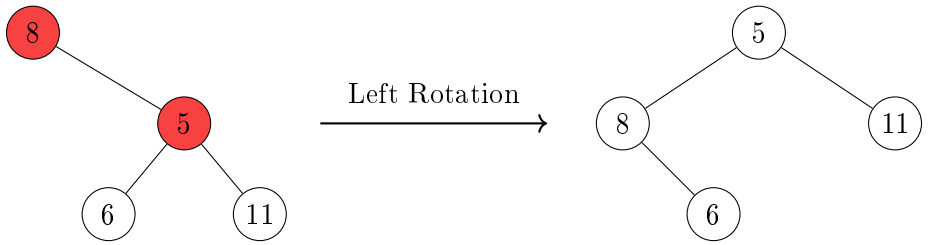
Before Intermediate After

Right Rotation

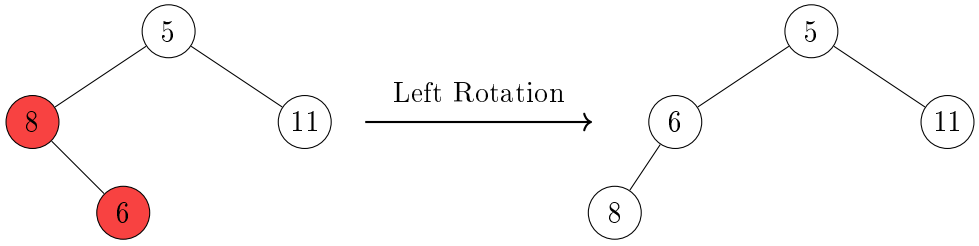


Before Intermediate After

Rotations can also fix heap violations the same way swaps did with the added benefit of preserving in-order traversal. A violation with right child and parent can be fixed with a left rotation at the parent and vice-versa.



Before After



Before After

Similar to Bubble Sort and the concept of inversions, we can look at the original tree as having 2 violations – 8 was above both 5 and 6 in the tree. Each rotation fixes exactly one of these violations just like each swap in Bubble Sort.

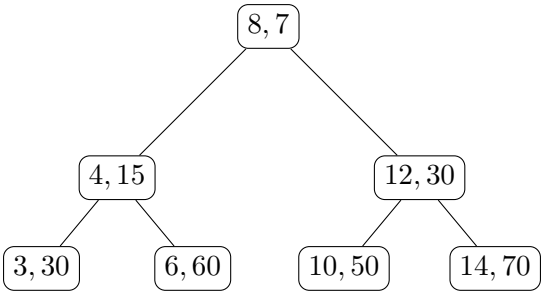
6.2.7 Treaps

Each node stores a key and along with it a random priority.

```
Node.value=(key,priority)
```

A treap simultaneously satisfies:

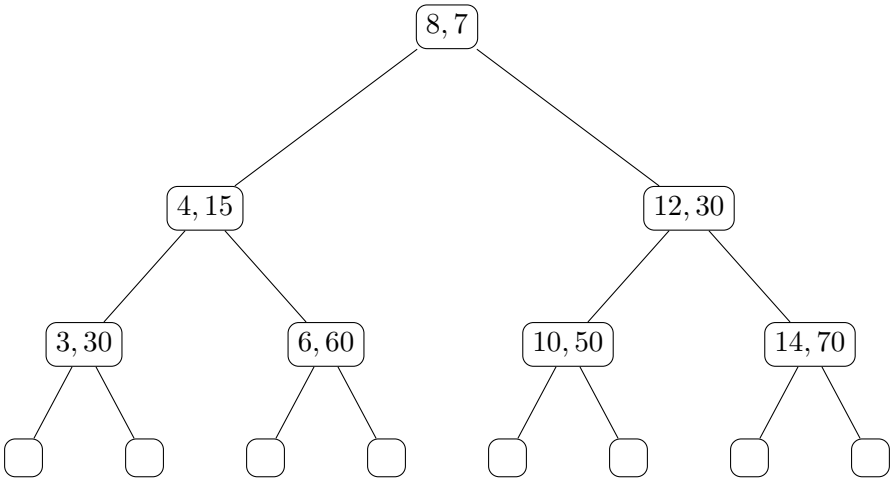
1. The **BST property** with respect to keys.
2. The **min-heap property** with respect to priorities.



The above figure shows a treap in which the first value is the key and the second is the priority.

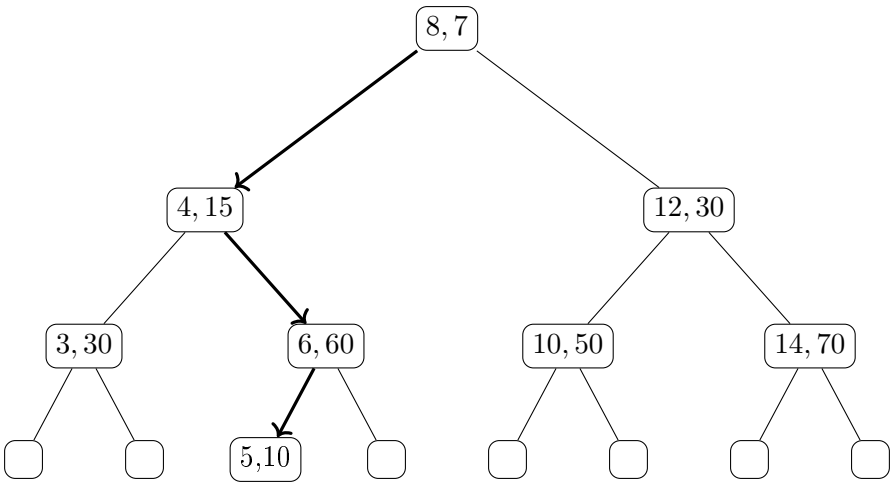
Insertion

Insertion in a treap proceeds in two phases. First, we insert the new key exactly as in a binary search tree. Then, if the heap property is violated, we restore it using rotations. Consider the following treap, where each node stores a pair (key, priority) and priorities satisfy the min-heap property.

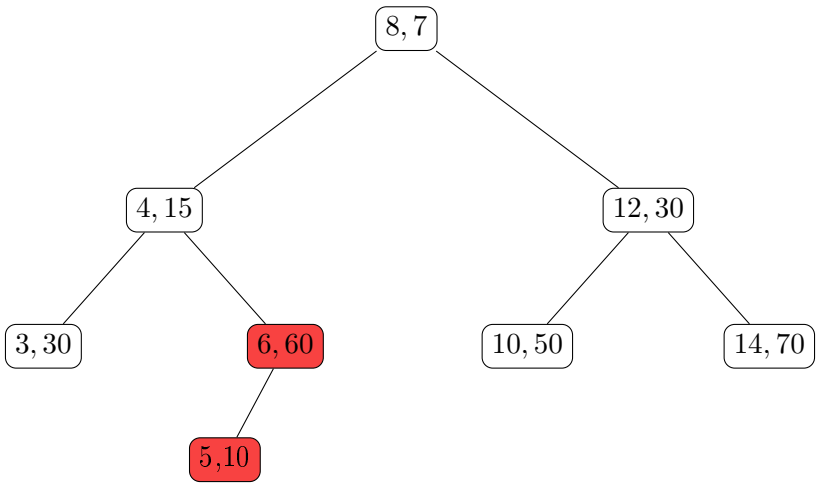


Suppose we wish to insert the node $(5, 10)$.

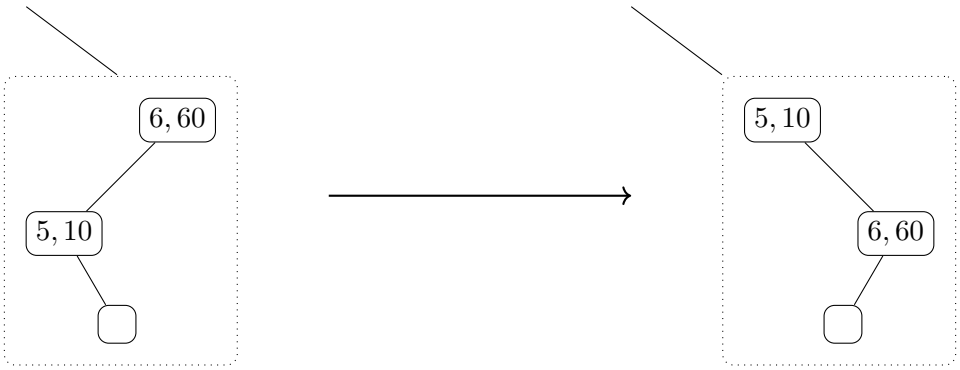
Step 1: BST Insertion Ignoring priorities, we first insert the key according to the BST property – the new node becomes the left child of $(6, 60)$.



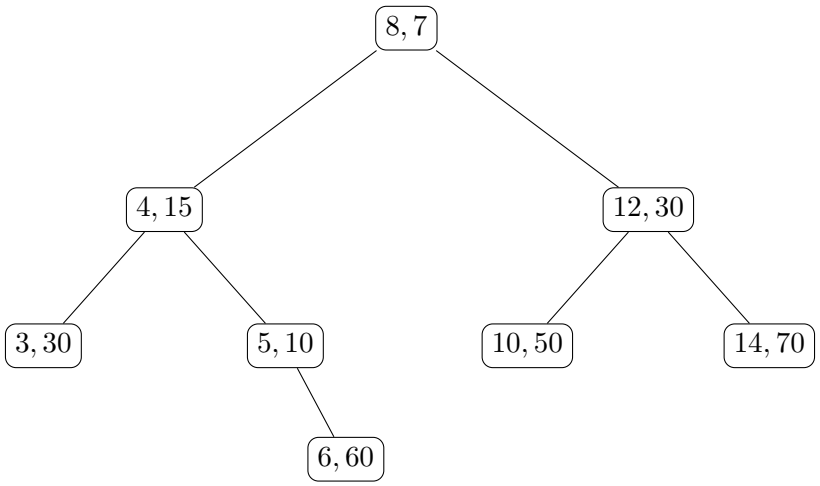
The BST property holds, but the min-heap property is violated since $10 < 60$.



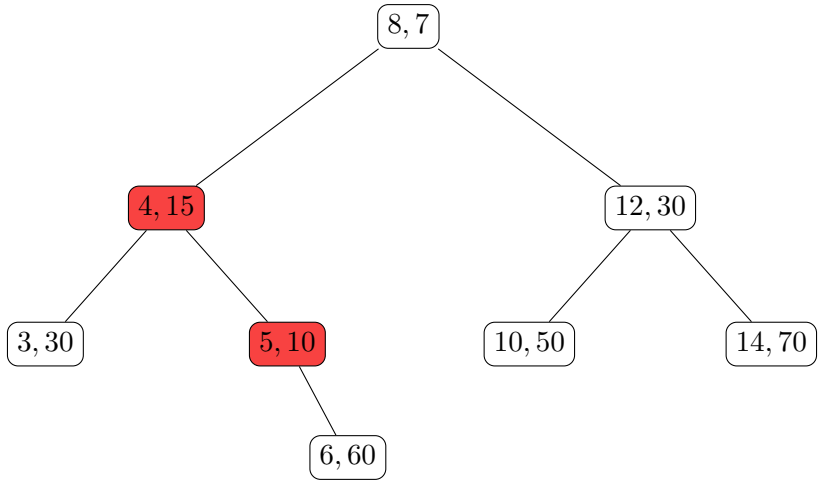
Step 2: Rotation at $(6, 60)$ Strictly speaking rotation involves three nodes, so we imagine the right child of $(5, 10)$ which is an empty node. We now perform a right rotation at $(6, 60)$.



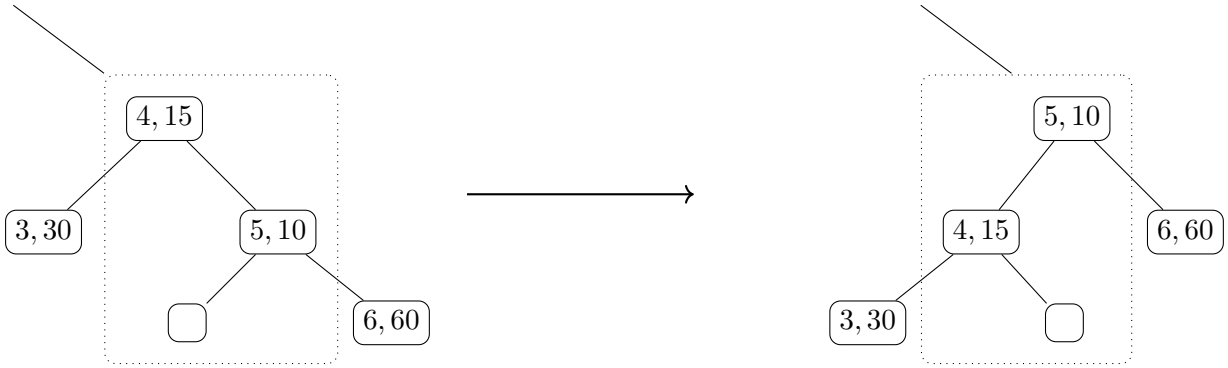
The treap now becomes



The heap property is still violated since $10 < 15$.

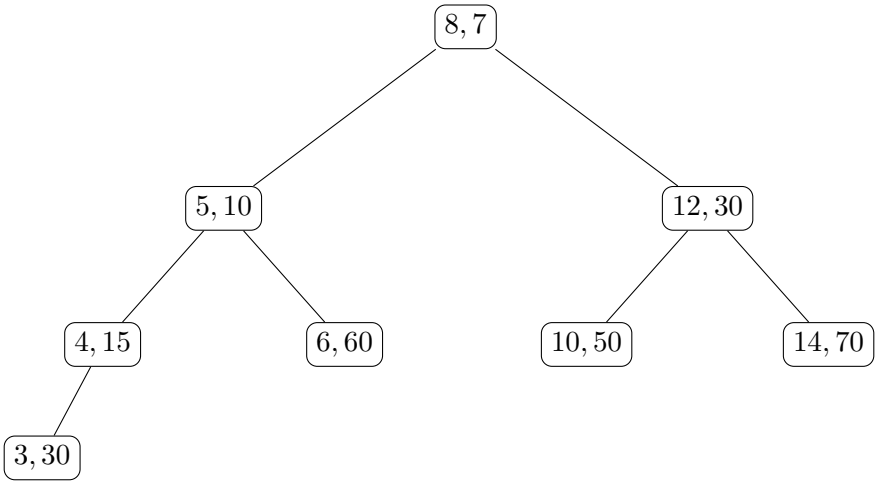


Step 3: Rotation at $(4, 15)$ We imagine the left child of $(5, 10)$ (which is an empty node) for the purposes of the rotation.



The heap property is now satisfied at every node:

$$7 < 10, \qquad 10 < 15, \qquad 10 < 60, \qquad 30 < 50, \qquad 30 < 70.$$



The heap property is now satisfied at every node. Notice that it takes at most $\Theta(H)$ time to fix a violation since the violation always involves the inserted element and in each rotation its height increases by exactly one.

Search

Searching in a treap is exactly the same as in a binary search tree – at most $\Theta(H)$ time.

Deletion

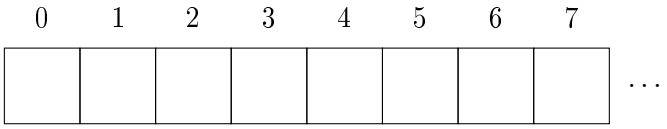
To delete an element, we must find it first. So we use the standard search to find it. Once it is found, we reverse the insert procedure by rotating *down* until it becomes a leaf. Then we can safely delete it.

6.3 Hash Tables

The need arose for a data structure to store a set of elements with efficient search, insertion and deletion operations – so we turn to hash tables!

6.3.1 The Structure

We begin with an empty array of size m. Since this is a plain old normal array of size m, we can access and modify any random element in constant time...

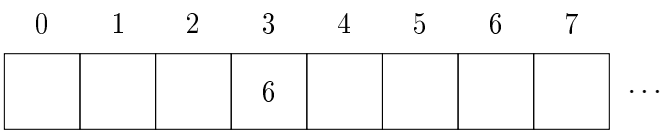


6.3.2 Insertion

We add an element x by computing $i = \mathbf{H}(x)$. For example, suppose $\mathbf{H}(6) = 3$. So we place 6 at index 3.

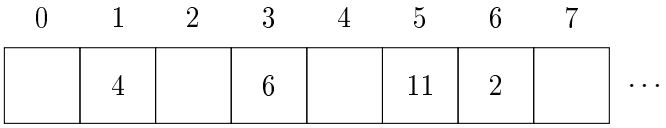
```
Insert(x)
{
    i = H(x)
    traverse Linked_List[i]
    {
        if x present
            return
    }
    add x to end of Linked_List[i]
}
```

First insertion



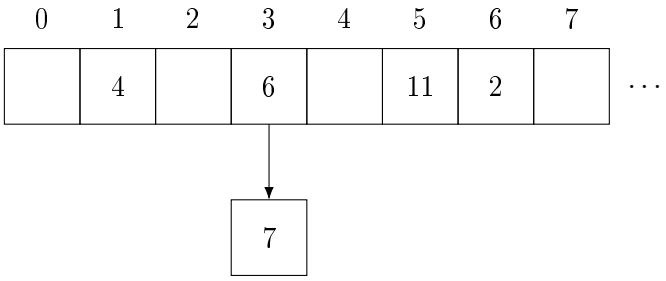
Similarly, we continue inserting elements in the same way as long as no two elements collide at the same index.

More insertions

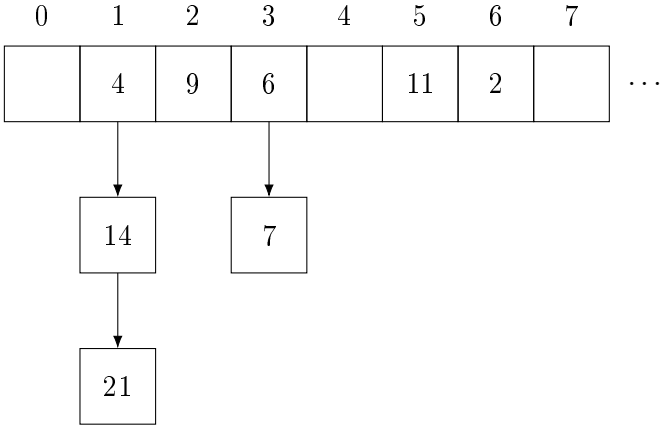


At this point, every index contains at most one element.

First *collision* Now we attempt to insert 7. Suppose $\mathbf{H}(7) = 3$. We try to place 7 at index 3, but we find 6 already present. So we extend that position into a linked list.



More collisions As more elements are inserted, collisions may occur at multiple indices, forming multiple linked lists.



Thus, each index i stores a linked list of all elements inserted till now who hash to i. Let the total number of elements inserted be n and the number of indices be m. Since on average each index has $\frac{n}{m}$ elements, the expected time of insertion is $\Theta(1 + \frac{n}{m})$.

6.3.3 Search

```
Search(x)
{
    i = H(x)
    traverse Linked_List[i]
    {
        if x found
            return true
    }
    return false
}
```

Since the expected number of elements per index is $\frac{n}{m}$, the expected time of search is $\Theta(\frac{n}{2m})$ since on average half the linked list is searched in a linear traversal.

6.3.4 Deletion

```
Delete(x)
{
    i = H(x)
    traverse Linked_List[i]
    if x found
        remove x from Linked_List[i]
}
```

The expected time of deletion is $\Theta(\frac{n}{2m})$ since on average half the linked list is searched in a linear traversal.

Overall, the three operations are generally considered expected $\Theta(1)$ since we keep $n \leq m$ and if n exceeds m we rehash with a larger m.

6.4 Bloom Filters

Suppose we wish to store a set of elements and support the operations `insert(x)` and `contains(x)`. A straightforward solution is to store all inserted elements in an `unordered_set`. This gives expected $\Theta(1)$ insertion and lookup time, but requires memory proportional to the number of stored elements. When the number of elements becomes extremely large, memory itself may become the primary constraint. Bloom filters can achieve this tradeoff.

6.4.1 The Structure

Instead of storing the elements themselves, we store only a hash of the set. We begin with a bit array of size m initialised to all zeros.

0	1	2	3	4	5	6	7	
0	0	0	0	0	0	0	0	$\cdots$

We then choose k hash functions

$$(H_1, H_2, \ldots, H_k)$$

each mapping elements to the set of indices $\{0, 1, \ldots, m-1\}$.

6.4.2 Insertion

To insert an element x, we compute

$$(H_1(x), H_2(x), \ldots, H_k(x))$$

and set all corresponding bits to 1. For example, if $k = 3$ and the hash functions are $H_1(x) = 3,\ H_2(x) = 7,\ H_3(x) = 10$ then we set positions $3, 7$, and 10 of the bit array to 1. If while inserting we see that a bit is already 1, then we do nothing.

This operation is $\Theta(k)$ time.

6.4.3 Search

To determine whether an element x is present, we again compute

$$(H_1(x), H_2(x), \ldots, H_k(x))$$

and inspect the corresponding bits.

- If at least one of these bits is 0, then x was certainly never inserted.
- If all of these bits are 1, then we report that x is present.

The second conclusion is of course subject to false positives! It is possible that the required bits were set earlier by completely different elements.

This operation is $\Theta(k)$ time.

6.4.4 Utility of Bloom Filters

The most important property of Bloom filters is that false negatives are impossible. If the filter reports that an element is absent, then that element was definitely never inserted. The only possible error is reporting that an element is present when it is actually absent.

Interestingly, Bloom filters (as the name suggests) are used as a filter. This means that incoming queries will go through the bloom filter – if the element is absent, we can trust the result. Otherwise we can perform a more expensive lookup in the database. So it is useful to reduce the time taken to process a membership query.

It can be shown[2] that the error probability of a Bloom filter with optimal k is approximately $2^{-\frac{m \ln 2}{n}}$. For this to be small, we need m to be large and n to be small. But one might question, wasn't the whole point of the bloom filter to reduce the memory usage?

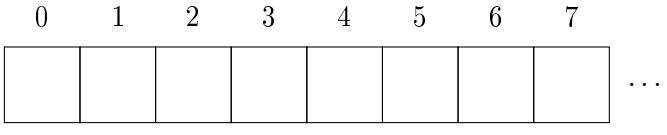
Let me clarify this with a practical example – lets say we have a database of URLs. Each URL takes 100 bytes to store. Now let us say we have 10^9 URLs – and the deterministic `set` structure has a constant factor of about 10 per node in memory. So overall that would require 10^{12} bytes of memory which is about 1 TB. In comparison, a Bloom filter of size $m = 10 \cdot n = 10^{10}$ bits would require only about 12.5 GB of memory.

6.5 Cuckoo Hashing

Recall that in ordinary hash tables, collisions are handled using linked lists. Cuckoo hashing takes a different approach. Instead of allowing multiple elements to occupy the same location, each element is given multiple possible locations and is stored in exactly one of them.

6.5.1 The Structure

We maintain an array of size m together with two hash functions (H_1, H_2).



[2] See Problems!

6.5.2 Insertion

Suppose we wish to insert an element x. Choose either of the hash functions H_1 or H_2 at random. Let this chosen hash be applied to x to obtain an index $H(x)$. If the array is empty at that index, we are done. Otherwise, let the occupant be y. We evict[3] y and place x in its position. The displaced element y must now be moved to its other possible location. If that position is occupied, another element is evicted, and the process continues.

We repeat this process until we either find an empty slot or find a cycle.

Cycles and Rehashing

In case we enter a cycle and insertion cannot be completed, choose new hash functions and rebuild the table from scratch.

Note that overall, as long as we keep n/m low, it can be shown that the expected time to insert an element is $\Theta(1)$ average time similar to hash tables.

6.5.3 Search

To determine whether an element x is present, we simply check the two possible locations $H_1(x)$ and $H_2(x)$. If x is found at either location, we report that it is present. Thus every lookup requires at most two array accesses, giving $\Theta(1)$ search time.

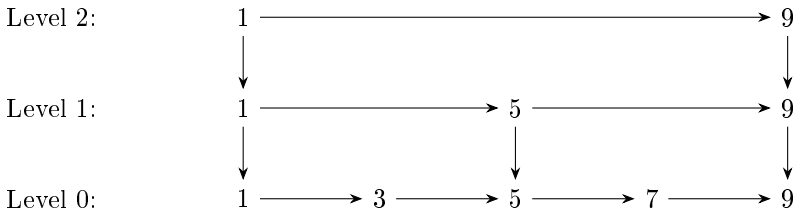
[3]This behaviour resembles the cuckoo bird which lays its eggs in another bird's nest and kicks out the occupying eggs. Hence the name ...

6.6 Problems

1 **(a)** Prove that the expected search time in a skip list is $\Theta(\log n)$. *Hint: downward moves are bounded by height, which is expected* $\Theta(\log n)$*.*

(b) Deduce similarly that insertion takes expected $\Theta(\log n)$ time.

(c) Try inserting 8 into the following skip list using the procedure we described earlier:



(d) Can you think of a $\Theta(\log n)$ deletion strategy for skip lists?

2 Consider n,m,k to be number of inserted elements, the size of bit array and the number of hash functions respectively in a Bloom filter. Assume the hash functions are independent.

(a) Show that the probability of a particular bit in the array being set is $\left(1-\left(1-\frac{1}{m}\right)^{kn}\right)$ and thus the probability of a false positive is $\left(1-\left(1-\frac{1}{m}\right)^{kn}\right)^{k}$

(b) Using suitable large m, n approximations, show that the optimal number of hash functions is $k=\frac{m}{n}\ln 2$

(c) Put the above results together to show that the probability of a false positive is approximately $2^{-\frac{m\ln 2}{n}}$.

3 Consider a Binary-Search Tree in which n distinct elements are inserted in a random order. Without loss of generality, consider the elements to be $1, 2, \ldots, n$. Let the random variable H_i be the height when i elements are inserted into the BST. Of course, $H_0 = 0$ and $H_1 = 1$.

(a) Prove that $H_n \geq \log_2(n+1) - 1$

(b) Use the definition of BST on the root to show that

$$(H_n|\ root = i) = 1 + max(H_{i-1}, H_{n-i})$$

(c) Use the substitution $Y_i = 2^{H_i}$ and take expectation both sides to show that

$$\mathbb{E}[Y_n|\ root = i] = 2 \cdot \mathbb{E}[max(Y_{i-1}, Y_{n-i})]$$

(d) Use the inequality $max(a, b) \leq a + b$ and the law of total expectation to show that

$$\mathbb{E}[Y_n] \leq \frac{4}{n} \cdot \sum_{i=1}^{n} \mathbb{E}[Y_{i-1}]$$

(e) Substitute S_n as the sum of $\mathbb{E}[Y_i]$ from 0 to n and show that

$$S_n \leq S_{n-1}\left(1 + \frac{4}{n}\right)$$

(f) Use Telescoping Product and the value of $S_0 = \mathbb{E}[Y_0] = \mathbb{E}[2^{H_0}] = 1$ to show that

$$S_n \leq \binom{n}{4}$$

(g) Use the expectation form of Jensen's inequality and the result from **(f)** to show that

$$\mathbb{E}[H_n] \leq \log_2 \left(\binom{n-1}{3} \right)$$

(h) Use the result from **(a)** and **(g)** to show that

$$\mathbb{E}[H_n] = \Theta(\log n)$$

Chapter 7

Number-Theoretic Algorithms

7.1 Introduction

Prime numbers have fascinated mathematicians for many years. They appear deceptively simple to define – a prime is a positive integer greater than one whose only divisors are 1 and itself. Yet many of the deepest questions in mathematics revolve around the distribution and properties of primes.

Beyond their theoretical significance, prime numbers occupy a central position in modern computing. Public-key cryptographic systems such as `RSA` rely on arithmetic involving very large primes, often hundreds or thousands of bits long. As a result, computers routinely face questions such as

- Is a given integer prime?
- How can a large prime be generated efficiently?
- If an integer is composite, can its factors be found quickly?

At first glance, these problems appear daunting. Consider an integer with several hundred decimal digits. Testing divisibility by every smaller integer is clearly infeasible, and even more sophisticated deterministic approaches may require substantial computational effort.

Surprisingly, randomness provides an elegant alternative. Instead of attempting to prove directly that a number is prime, randomized algorithms often search for evidence that it is composite. A carefully chosen random experiment can reveal such evidence with high probability.

In this chapter, we will not be treating multiplication and division as constant time operations. Since these algorithms frequently work with numbers of the order 10^100 or even larger, we will assume that multiplication and division are $\Theta(\log^2 n)$ time, since a number n has $\Theta(\log n)$ digits.

7.2 Fermat Test

The simplest algorithm to test the primality of an integer n is to check for divisibility by every integer from 2 up to $\sqrt{n}$. Using the fact that divison takes $\Theta(\log^2 n)$, the whole algorithm runs in $\Theta(\sqrt{n}\log^2 n)$ time. Any algorithm we come up with should be at least as efficient as this!

The central result underlying our first algorithm is Fermat's Little Theorem. Let the set of primes be denoted by $\mathbb{P}$ and the set $\{0,1,\dots\}$ by $\mathbb{N}$.

7.2.1 Fermat's Little Theorem

$$p \in \mathbb{P}, a \in \mathbb{N} \implies a^p \equiv a \pmod{p}$$

A useful equivalent form is obtained if $p \nmid a$, in which case we can divide both sides by a to get

$$a^{p-1} \equiv 1 \pmod{p}$$

This observation immediately suggests a test for primality. Given an integer n, choose some value a, compute $a^{n-1} \bmod n$, and check whether the congruence holds. If the congruence fails, then n cannot be prime.

7.2.2 The Algorithm

```
exp(a, x, n) // computes a^x modulo n in O(log x)
{
    if x == 0
        return 1
    y = exp(a, x/2, n)
    if x even
        return (y*y) mod n
    else
        return (a*y*y) mod n
}

PrimalityTest(n) // for n>=4
{
    if(n even)
        return "composite"

    choose a random a in {2, ..., n-2}

    /*We avoid a=1 and a=n-1 since they trivially satisfy
    the congruence even when n is composite */

    if exp(a, n-1, n) != 1
        return "composite"
    else
        return "probably prime"
}
```

Note the rather beautiful technique used in the function[1] `exp(a, x, n)`. Another way of looking at it is that we find the binary expansion of x and compute the powers of a corresponding to the bits of x using repeated squaring.

7.2.3 Error Probability

The natural question is –

"How often can a composite number fool the test?"

[1] This is sometimes called the binary exponentiation algorithm.

For many composite integers the probability is quite small. Unfortunately, there exists a remarkable family of numbers that pass Fermat's test for *every* admissible choice of a.

These numbers are known as Carmichael[2] numbers, and they expose a fundamental weakness of the algorithm.

7.3 Miller–Rabin Test

7.3.1 A Key Property

Let p be an odd prime and let d such that

$$p - 1 = 2^s d, \qquad d \text{ is odd}$$

$$\text{Define } x_i = a^{2^i d} \pmod{p}, i \in \mathbb{N} \iff x_0 = a^d, \quad x_1 = a^{2d}, \quad x_2 = a^{4d}, \quad \ldots, \quad x_s = a^{2^s d}$$

By Fermat's Little Theorem, we have $x_s = a^{p-1} \equiv 1 \pmod{p}$.

Clearly, each term is the square of the previous one modulo p. That is,

$$x_{i+1} \equiv x_i^2 \pmod{p}$$

A simple fact about prime moduli –

$$p \in \mathbb{P},\ x \in \mathbb{N},\ x^2 \equiv 1 \pmod{p} \implies x \equiv 1 \pmod{p} \text{ or } x \equiv -1 \pmod{p}.$$

So either $x_0 = 1 \pmod{p}$ or there is some index $i \geq 0$ such that $x_i = -1 \pmod{p}$.

[2]The smallest Carmichael number is 561. It has been proven that there are infinitely many Carmichael numbers.

7.3.2 The Algorithm

```
exp(a, x, n)
{
    if x == 0
        return 1
    y = exp(a, x/2, n)
    if x even
        return (y*y) mod n
    else
        return (a*y*y) mod n
}

isPrime(n) // for n>=4
{
    if(n even)
        return "composite"

    s=0
    d=n-1
    while d is even
        increment s by 1, divide d by 2

    //here we have decomposed n-1 into 2^s and d, where d is odd

    Choose a random a in {2, ..., n-2}
    x = exp(a, d, n)

    if x == 1 or x == n-1
        return "probably prime"

    loop i=1 to s-1
    {
        x = (x*x) mod n
        if x == n-1
            return "probably prime"
        if x == 1
        {
            return "composite"
            /*since hitting 1 mod n early means we have found a
            non-trivial square root of 1 mod n, so n is composite*/
        }
    }
    return "composite"
}
```

7.3.3 Error Probability

It can be shown that if n is composite, then the probability that the Miller–Rabin test returns "probably prime" is at most 1/4.

The proof[3] that the Miller–Rabin test has error probability at most 1/4 is quite technical and relies on some non-trivial number theory and group theory. Let $T = \{2, \ldots, n-2\}$, the set of all possible a's. The key and final step is to show that one can partition T into disjoint subsets of size at least 4 such that each subset contains at most one false positive. Thus,

$$|F| \leq |T|/4 \implies \frac{|F|}{|T|} \leq \frac{1}{4}$$

7.3.4 Performance

Since there are $\Theta(\log n)$ multiplications involved in binary exponentiation, and each multiplication takes $\Theta(\log^2 n)$ time, the overall total time complexity is $\Theta(\log^3 n)$.

Repeating the test k times will take $\Theta(k \log^3 n)$ time. A reasonable choice of k (e.g., $k = 34$) makes the error probability at most 10^{-20}.

7.4 Pollard's Rho Algorithm

The Problem – Given a composite integer n, find a non-trivial[4] factor of n.

The Solution – We can create a pseudo-random sequence of integers $x_0, x_1, \ldots, x_i, \ldots$ such that $x_i \equiv x_{i-1}^2 + c \pmod{n}$ for some c.

If at any point $\gcd(x_i - x_j, n)$ is not 1 or n, we can take that to be a factor of n by definition[5] of the gcd!

Why do we use the weird sequence $x_i \equiv x_{i-1}^2 + c \pmod{n}$? Why not generate a sequence of random integers and do the same thing? The reason behind using a deterministic sequence is that in this iterative process, we can use Floyd's cycle-finding algorithm to detect collision which takes time linear in the cycle size. In the other case, we would have to store and check with all previous values of x_j for every new x_i generated. That is quadratic in the length of the cycle! Secondly, choosing x_0 and c randomly acts as a seed for the pseudo-random process. The sequence is thus overall truly random for our analysis purposes yet is compatible with Floyd's algorithm.

[3] In case you are interested, see https://codeforces.com/blog/entry/98102

[4] In this case, that would be a factor other than 1 and n

[5] Conventionally, gcd(0,n) is taken to be n.

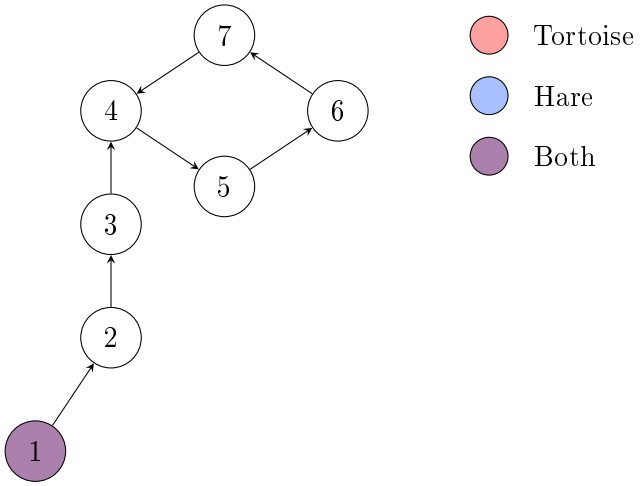
7.4.1 Floyd's Cycle-Detection Algorithm

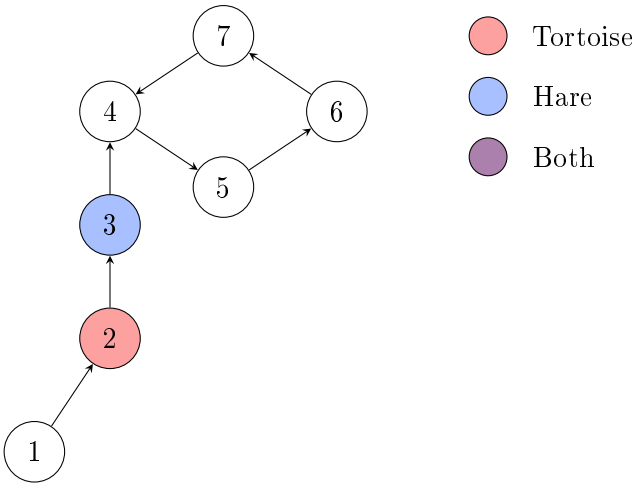
```
FloydCycleDetection(f,x)
{
    tortoise = f(x)
    hare = f(f(x))

    while tortoise != hare:
    {
        tortoise = f(tortoise)
        hare = f(f(hare))
    }

    return
}
```

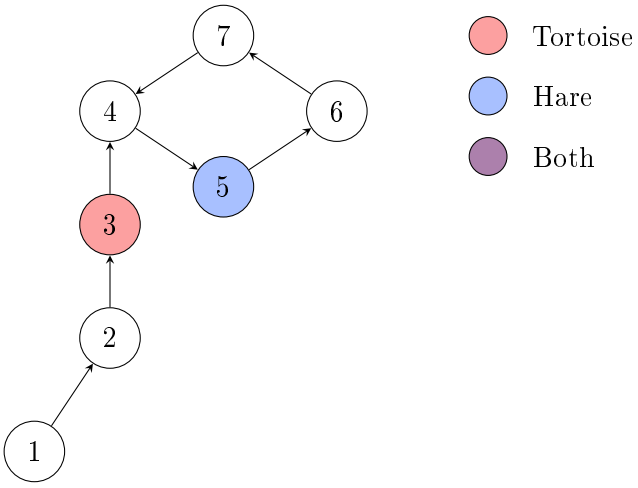
Example

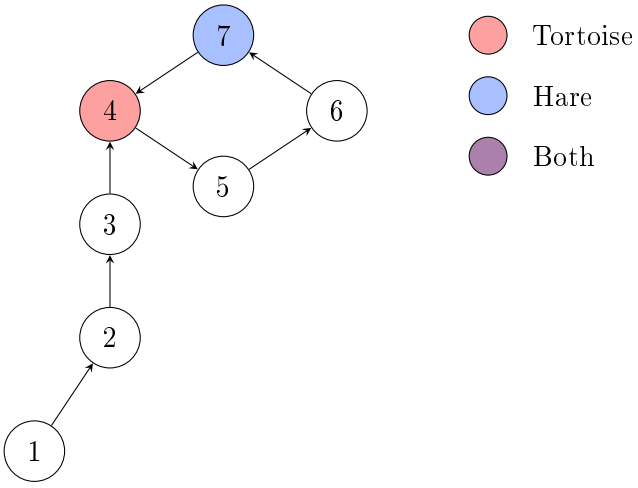
We can represent the sequence with a directed graph as follows – there is an edge between each x and $f(x)$ or equivalently each x_i and x_{i+1}. The starting node is x_0 where both the hare and tortoise start. It can be proven that Floyd's algorithm works with some effort, but the point of this example is to give a visual intuition rather than write a rigorous proof. Also, I like colorful graphs!

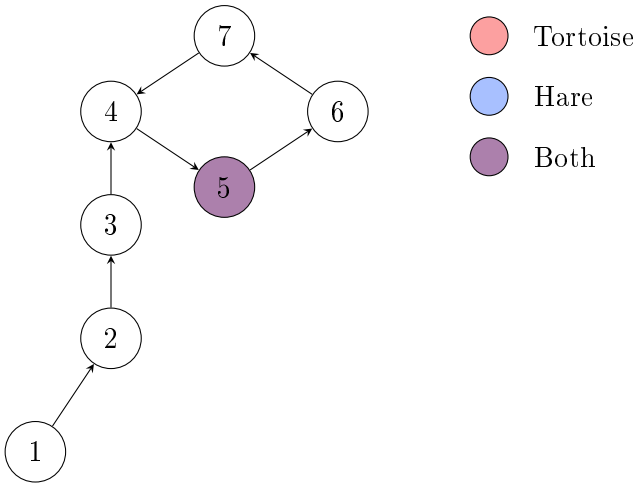
The shape of the graph is deliberately chosen to be like the greek letter ρ, which is where the name `Pollard's Rho` comes from. It is the general shape of the sequence which the algorithm considers, because the output space of $f(x)$ is finite so it must go into a cycle by pigeonhole principle.




7
4
6
5
3
2
1
Tortoise
Hare
Both


7
4
6
5
3
2
1
Tortoise
Hare
Both


7
4
6
5
3
2
1
Tortoise
Hare
Both



And thus when both coincide, we can report we have found the cycle! Try this on a cycle with a different (perhaps odd) number of nodes to really convince yourself that it works.

7.4.2 The Algorithm

```
f(t)
{
    return (t*t + c) mod n
}

PollardRho(n)
{
    if n is even:
        return 2

    choose random c in [1, n-1]
    choose random x in [0, n-1]

    y = x
    d = 1

    while d == 1:
    {
        x = f(x)
        y = f(f(y))

        d = gcd(|x - y|, n)
    }

    if d == n:
        restart algorithm with new x, c

    return d
}
```

7.4.3 Performance

Collision Model

Our sequence $x_0, x_1, x_2, \ldots$ is generated by iterating a polynomial function modulo n. Since our choice of x_0 and c is random,the sequence can be treated as independent samples from a uniform distribution over $\{0, 1, \ldots, n-1\}$, until repetition occurs. And we are looking for a hidden number p, the modulo over which the sequence repeats. Since p divides n, the sequence when taken modulo p will thus act like independent samples from a uniform distribution over $\{0, 1, \ldots, p-1\}$.

Equivalently it can be said that the distribution Uniform(0,n-1) modulo p is equivalent to the distribution Uniform(0,p-1).

This naturally leads us to the birthday paradox[6] model, where we are interested in the expected time until a collision occurs in a sequence of random samples from a finite set.

Suppose we define the collision step S as the smallest index such that a repetition occurs,

$$\iff S = \min\{j \mid x_i = x_j \text{ for some } i < j\}$$

Probability of No Collision

Let P_k denote the probability that the first k samples are all distinct. This is equivalent to $Pr(S > k)$. Then using the fact that the samples are uniform and independent, we can express P_k as

$$P_k = Pr(S > k) = \prod_{i=0}^{k-1} \left(1 - \frac{i}{p}\right)$$

Expected Collision Step

The expected collision time can be expressed as

$$\mathbb{E}[S] = \sum_{k=0}^{\infty} Pr(S > k) = \sum_{k=0}^{\infty} P_k = 1 + \left(1 - \frac{1}{p}\right) + \sum_{k=2}^{\infty} P_k = 2 - \frac{1}{p} + \sum_{k=2}^{\infty} P_k$$

We will use $2 - \frac{1}{p} \leq 2$ and the inequality $1 - x \leq e^{-x}$ to bound this.

$$P_k = \prod_{i=0}^{k-1} \left(1 - \frac{i}{p}\right) \leq \prod_{i=0}^{k-1} e^{-i/p} = e^{-\sum_{i=0}^{k-1} i/p} = e^{-k(k-1)/(2p)}$$

$$\implies \mathbb{E}[T] \leq 2 + \sum_{k=2}^{\infty} e^{-k(k-1)/(2p)}$$

For $k \geq 2$,

$$k(k-1) \geq \frac{k^2}{2} \implies \frac{k(k-1)}{2p} \geq \frac{k^2}{4p} \implies e^{-k(k-1)/(2p)} \leq e^{-k^2/(4p)}$$

$$\implies \mathbb{E}[S] \leq 2 + \sum_{k=2}^{\infty} e^{-k^2/(4p)}$$

[6] The probability that in a group of 23 people we find two with the same birthday is approximately 1/2. This is of course, not a true *paradox* but simply a surprising result from an intuition standpoint.

Since the function $f(x) = e^{-x^2/(4p)}$ is monotonically decreasing on $[0, \infty)$, we can compare the sum with an integral!

$$\sum_{k=2}^{\infty} e^{-k^2/(4p)} \leq \int_0^{\infty} e^{-x^2/(4p)}\, dx.$$

Using the substitution $x = 2\sqrt{p}\, t,\ dx = 2\sqrt{p}\, dt$,

$$\implies \int_0^{\infty} e^{-x^2/(4p)}\, dx = 2\sqrt{p} \int_0^{\infty} e^{-t^2}\, dt = 2\sqrt{p} \cdot \frac{\sqrt{\pi}}{2} = \sqrt{\pi p}.$$

$$\implies \mathbb{E}[S] \leq 2 + \sqrt{\pi p} \implies \mathbb{E}[S] \leq \Theta(\sqrt{p})$$

Collision Detection

Once a collision occurs modulo p, say

$$x_i \equiv x_j \pmod{p}, \quad i \neq j,$$

we know p must divide $x_i - x_j$. But how do we check that if we don't know p? Knowing we want to find a p which divides $x_i - x_j$ and n, the logical step is to find the $\gcd(x_i - x_j, n)$, because if that is 1 then p is definitely[7] 1. And as soon as $\gcd(x_i - x_j, n)$ becomes not 1 (and not n), we can take that as the divisor. We know all p for which the collision happened are definitely[8] smaller than this number so we could not have done better!

Due to Floyd's cycle detection algorithm, we don't need to keep track of all pairs of indices i, j. We can simply keep track of the pairs (x_i, x_{2i}) for $i = 1, 2, \ldots$ and check for collisions between these pairs. This is what the algorithm does by maintaining two pointers x and y that move at different speeds.

Cost Per Step

Each step of the algorithm performs a constant number of modular multiplications and one greatest common divisor computation. Using the Euclidean algorithm, $\gcd(a, b)$ can be computed in at most $\Theta(\log^3 max(a, b)))$ time.

Hence each iteration costs at most $\Theta(\log^3 n)$ time since n is the largest number we are working with for all the involved operations.

Total Expected Complexity

Since cost per step is at most $\Theta(\log^3 n)$, and the expected number of steps is at most $\Theta(\sqrt{p})$, the total expected complexity is at most $\Theta(\sqrt{p} \log^3 n)$ for a particular prime p.

Let $n = \prod_{i=1}^{k} p_i^{e_i}$ where $p_1 < p_2 < \cdots < p_k$ are the prime factors of n. Let T_i denote the random time until a collision occurs modulo p_i, and let T be the running time of `Pollard's Rho`, i.e.

$$T = \min_{1 \leq i \leq k} T_i$$

Since $T \leq T_i$ for every i, taking expectations gives

$$\mathbb{E}[T] \leq \mathbb{E}[T_i] \quad \forall i \in \{1, 2, \ldots, k\} \implies \mathbb{E}[T] \leq \min_{1 \leq i \leq k} \mathbb{E}[T_i]$$

[7] This is concluded using the lemma – all common divisors of a and b divide the gcd(a,b)

[8] By definition of the gcd, it must be the *greatest* common divisor!

$$\implies \mathbb{E}[T] \leq \Theta(\sqrt{p_1}\log^3 n), \quad \text{where } p_1 \text{ is the smallest prime factor of } n$$

$$p_1 <= \sqrt{n} \implies \mathbb{E}[T] \leq \Theta(\sqrt{p}_1 \log^3 n) \leq \Theta(n^{\frac{1}{4}} \log^3 n)$$

What did we achieve?

We can find a factor of n in expected time $\Theta(n^{\frac{1}{4}} \log^3 n)$, a vast improvement over the naive $\Theta(n^{\frac{1}{2}} \log^2 n)$ method.

To factorise a large number n fully, we can use the Pollard-Rho algorithm now! Since every factor we find is at least 2, after dividing n by 2 and repeating the factor-finding process, the overall procedure terminates in at most $\Theta(\log_2 n)$ steps. Thus we can factorise a number n (no matter how large), in expected time at most $\Theta(n^{\frac{1}{4}} \log^4 n)$.

Note that we have used a lot of worst case upper bounds in the analysis. The average performance in practice is much better than the complexity suggests!

7.5 Problems

1 During RSA encryption, one computes a secret integer $x = \phi(n)$. A valid public exponent e must satisfy

$$\gcd(e, x) = 1.$$

Since we do not want to reveal x to the public, we can not take e to be something deterministic like $x - 1$ because an adversary can crack our encryption if he gets to know the algorithm. One possible strategy is to repeatedly choose e uniformly at random from $\{1, 2, \ldots, x\}$ until a suitable value is found.

(a) Let

$$x = \prod_{i=1}^{k} p_i^{e_i}$$

be the prime factorization of x. Show that the probability that a randomly chosen $e \in \{1, 2, \ldots, x\}$ satisfies $\gcd(e, x) = 1$ is

$$\frac{\varphi(x)}{x} = \prod_{i=1}^{k} \left(1 - \frac{1}{p_i}\right).$$

(b) Show that if x can be any number from 1 to n where n is very large, the probability of the randomly chosen e being correct tends to $\frac{6}{\pi^2}$ as n becomes larger.
Hint: Use $\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$

Chapter 8

Probabilistic Method

8.1 Introduction

The probabilistic method refers to the general method of proving the occurrence of an event or make guarantees about the quantity of an object by setting up a random experiment. A common method is finding the probability of existence of the object and using its non zero value to justify there must exist at least one such case where the object exists. Another way is to find the expected value of the quantity of the object and then concluding there must exist a case with at least the expected value.

This method simultaneously proves the existence of the object and gives a randomised algorithm for finding a case – simply perform the random experiment repeatedly until the object is found. Knowing the probability or useful bounds on it will help us obtain the expected time complexity of the algorithm.

8.2 Array Construction

Construct an array of $4n$ integers in which each of the values $1, 2, \ldots, n$ appears exactly four times, subject to the following condition – for a value x, let $p_{x,1} < p_{x,2} < p_{x,3} < p_{x,4}$ be the positions of its four occurrences. Then for every x the three consecutive gaps

$$g_1 = p_{x,2} - p_{x,1}, \qquad g_2 = p_{x,3} - p_{x,2}, \qquad g_3 = p_{x,4} - p_{x,3}$$

must be pairwise distinct. For instance, with $n = 3$ the array $[1, 1, 2, 1, 2, 3, 1, 3, 2, 2, 3, 3]$ works because the gaps are $(1, 2, 3)$ for 1, $(2, 4, 1)$ for 2 and $(2, 3, 1)$ for 3, each consisting of three different numbers.

8.2.1 Random Experiment

Consider the random experiment of just constructing a random permutation of the $4n$ integers. Let the four positions of x, in increasing order, be $a_1 < a_2 < a_3 < a_4$, and define the gaps g_1, g_2 and g_3.

$$g_1 = a_2 - a_1, \qquad g_2 = a_3 - a_2, \qquad g_3 = a_4 - a_3,$$

These gaps have a span $s = g_1 + g_2 + g_3 = a_4 - a_1$. Once the gaps are fixed, the exact subset is determined by the starting position a_1 alone, which must satisfy $a_1 \geq 1$ and $a_1 + s = a_4 \leq N$.

There are therefore exactly $N-s$ admissible subsets for each tuple of gaps with $s \leq N-1$. Also note that the probability of two gaps being equal is the same regardless of which pair of gaps we are referring to (by symmetry).

$$\Pr(g_1 = g_2) = \Pr(g_2 = g_3) = \Pr(g_1 = g_3)$$

8.2.2 Success Probability

Two Equal Gaps

When $g_1 = g_2 = g$ and $g_3 = h$, $s = 2g + h$. So the number of such subsets are $N - (2g + h)$. Double summation over all possible g and h will yield all valid subsets and thus the exact probability.

$$\Pr(g_1 = g_2) = \frac{1}{\binom{4n}{4}} \sum_{g=1}^{2n-1} \left(\sum_{h=1}^{4n-2g-1} (4n - 2g - h) \right) = \frac{1}{\binom{4n}{4}} \sum_{g=1}^{2n-1} \binom{4n-2g}{2} = \frac{n(2n-1)(8n-5)}{3 \cdot \binom{4n}{4}}$$

$$\implies \Pr(g_1 = g_2) = \frac{n(2n-1)(8n-5)}{n(2n-1)(4n-1)(4n-3)} = \frac{8n-5}{(4n-1)(4n-3)} \approx \frac{1}{2n} + \Theta(\frac{1}{n^2})$$

All Equal Gaps

Here $g_1 = g_2 = g_3 = g$, so $s = 3g$ and the number of subsets is $4n - 3g$. The summation over g will go from 1 to $G = \lfloor \frac{4n-1}{3} \rfloor$

$$\sum_{g=1}^{G} (4n - 3g) = 4nG - \frac{3G(G+1)}{2} = \frac{G\,(8n - 3G - 3)}{2}$$

$$\implies \Pr(g_1 = g_2 = g_3) = \frac{G\,(8n - 3G - 3)}{2\binom{4n}{4}} = \frac{3\,G\,(8n - 3G - 3)}{(2n)(2n-1)(4n-1)(4n-3)}$$

Taking large n approximations and $G \approx \frac{4n}{3}$ gives $\approx \frac{1}{4n^2}$

$$\Pr(g_1 = g_2 = g_3) \approx \frac{4n\,(8n - 4n)}{(2n)(2n)(4n)(4n)} \approx \frac{1}{4n^2}$$

Inclusion-Exclusion

$$\Pr(\text{bad value}) = \Pr((g_1 = g_2) \cup (g_2 = g_3) \cup (g_1 = g_3))$$

$$= \Pr(g_1 = g_2) + \Pr(g_2 = g_3) + \Pr(g_1 = g_3) - 3 \cdot \Pr(g_1 = g_2 = g_3) + \Pr(g_1 = g_2 = g_3)$$

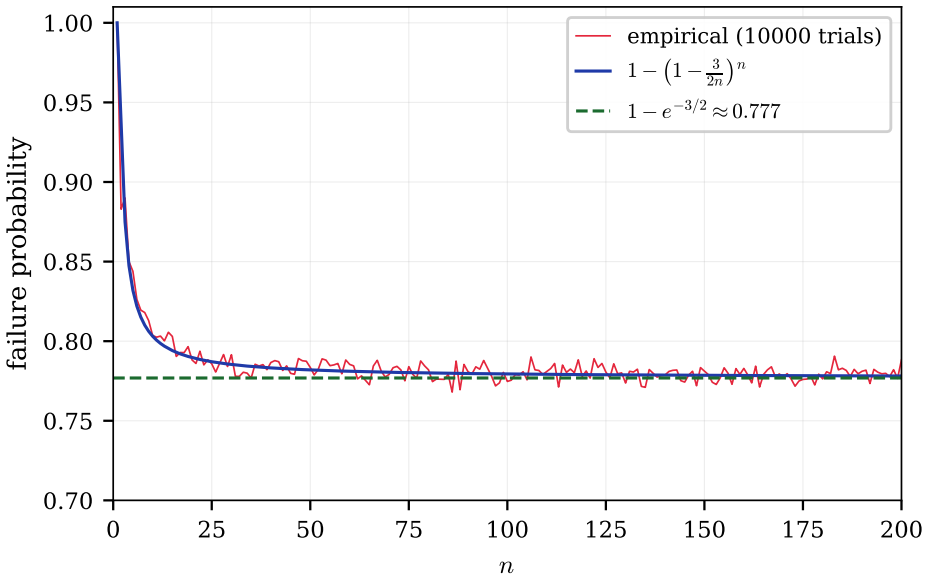
$$= 3\Pr(g_1 = g_2) - 2 \cdot \Pr(g_1 = g_2 = g_3) \approx 3 \cdot (\frac{1}{2n}) - 2 \cdot (\frac{1}{4n^2}) + \Theta(\frac{1}{n^2}) \approx \frac{3}{2n}$$

$$\implies \Pr[\text{bad value}] \approx \frac{3}{2n}$$

Independence

The arrangement fails exactly when the gaps of at least one value is bad. The union bound[1] gives $Pr(\text{failure}) \leq \sum_1^n \frac{3}{2n} = \frac{3}{2}$ as the upper bound on overall failure which is useless. To find anything useful, we need to know something about how they are dependent on each other. We will skip the exact analysis of this interdependence and notice that the placement of one value affects the gaps of other value by at most 4. Since this won't affect the equality of the gaps of most arrangements (because in most arrangements the gaps differ significantly), they are almost[2] independent.

$$\Pr[\text{success}] \approx (1 - \Pr(\text{bad value}))^n \approx \left(1 - \frac{3}{2n}\right)^n$$



Interestingly, the method used to generate this graph is a randomised algorithm which uses the method of sampling. We used the idea of law of large numbers by saying that the number of failures divided by total number of runs (10000) should be a good approximation to the failure probability. Of course, 10000 was chosen because it was large as compared to the n under consideration.

8.2.3 The Algorithm

So the probabilistic method tells us there is at least 0.2 probability of a valid construction for n greater than 25. Thus this gives rise to a polynomial time algorithm – create a random array of $4n$ integers and simply verify whether the properties hold or not. If doesn't hold, try again!

[1] Refer Appendix A.3

[2] Yes, this is an approximation but the graph of empirical data should convince you how good it is.

```
construct()
{
    vector A;
    loop(i from 1 to n)
    {
        A.append({i,i,i,i});
    }
    // A is now {1,1,1,1,2,2,2,2,...,n,n,n,n}

    while(true)
    {
        shuffle(A);
        CHK=check_gaps(A); // can be done in O(n)
        if(CHK)
            return A
    }
}
```

8.2.4 Expected Time Complexity

Each `while loop` is essentially a run of the random experiment with each run taking $\Theta(n)$ time. Since the success probability is at least 0.2, the expected number of runs is at most $\frac{1}{0.2} = 5$.

$$\mathbb{E}[N] \leq 5$$

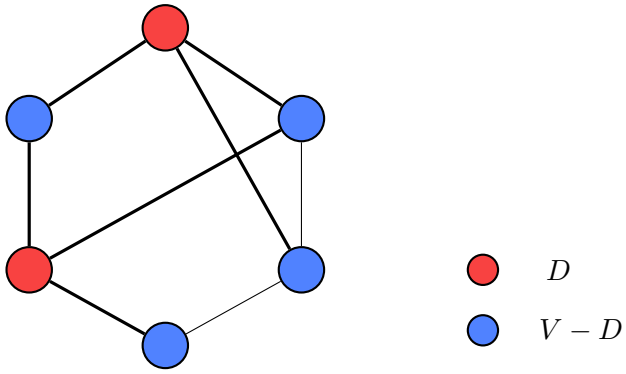
Let $T_1, T_2, \ldots, T_N$ be the runtimes of the corresponding loops and T be the runtime of the whole algorithm. Each T_i is i.i.d. so using Wald's equation[3] we get that the expected runtime of the whole algorithm is $\Theta(n)$.

$$T = \sum_{i=1}^{N} T_i \implies \mathbb{E}[T] = \mathbb{E}[T_i] \cdot \mathbb{E}[N] \leq \Theta(n) \cdot 5 = \Theta(n)$$

8.3 Dominating Set

We say a vertex u dominates a vertex v if $u = v$ or $(u, v) \in E$. A *dominating set* is a set of vertices D such that each vertex in the graph is dominated by at least one other vertex. Since finding the smallest dominating set is $\mathbf{NP - hard}$, we are interested in using the probabilistic method, to come up with a simple algorithm which has a guarantee on the size of the dominating set it produces.

[3] Refer Appendix A.8



8.3.1 Random Experiment

Suppose the graph has minimum degree d. We choose k vertices randomly without replacement and check whether they form a dominating set.

8.3.2 Success Probability

Fix a vertex $v \in V$. It has at least d neighbours, so at most $(n-d-1)$ vertices do not dominate v. So the probability that v remains undominated is at most $\binom{n-d-1}{k}/\binom{n}{k}$

$$\Pr[v \text{ remains undominated}] \leq \frac{\binom{n-d-1}{k}}{\binom{n}{k}} = \prod_{i=0}^{k-1} \frac{n-d-1-i}{n-i} = \prod_{i=0}^{k-1} \left(1 - \frac{d+1}{n-i}\right)$$

$$1 - \frac{d+1}{n-i} \leq 1 - \frac{d+1}{n} \implies \Pr\left(\text{undominated } V_i\right) \leq \prod_{i=0}^{k-1} \left(1 - \frac{d+1}{n}\right) = \left(1 - \frac{d+1}{n}\right)^k$$

The algorithm fails if at least one vertex remains undominated. Applying the union bound, we get an upper bound for overall failure probability.

$$\Pr(\text{failure}) = \Pr\left(\bigcup_{i=1}^{n} \text{undominated } V_i\right) \leq \sum_{1}^{n} \Pr(\text{undominated } V_i) \leq n \cdot e^{-k(d+1)/n}$$

To guarantee existence, probability of failure must be proved strictly less than 1 so that probability of success is non-zero.

$$ne^{-k(d+1)/n} < 1 \implies k > \frac{n \ln n}{d+1}$$

8.3.3 The Algorithm

```
Dominate(G,k)
{
    while(true)
    {
        n=V.size()

        shuffle(V)
        D = V[1 to k]
        dominated = set(D)

        for u in D
        {
            for v in neighbours[u]
                dominated.insert(v)
        }

        if(dominated.size()==n)
            return D
    }
}
```

8.3.4 Expected Time Complexity

For $k > \frac{n \ln n}{d+1}$, we guaranteed the existence of a dominating set of size k. A suitable choice for k is $\lceil n \ln n/(d+1) \rceil + n/(d+1)$, because in that case the success probability is at least $1 - 1/e$.

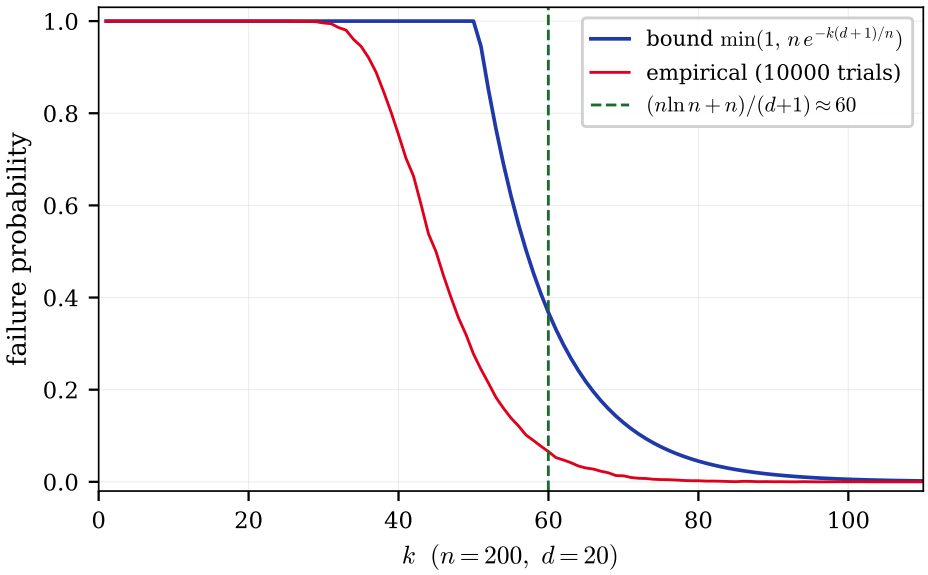
$$\Pr(\text{failure}) \leq ne^{-k(d+1)/n} \leq ne^{-(\frac{n \ln n}{d+1} + \frac{n}{d+1})(d+1)/n} = \frac{1}{e} \implies \Pr(\text{success}) \geq 1 - \frac{1}{e}$$

If the implementation of the algorithm is done using a boolean array for storing whether a vertex was dominated or not, then one iteration takes $\Theta(n+m)$ time. The expected number of iterations (N) is at most $\frac{1}{1-1/e} = \frac{e}{e-1}$

Let $T_1, T_2, \ldots, T_N$ be the runtimes of the corresponding loops and T be the runtime of the whole algorithm. Each T_i is i.i.d. so using Wald's equation we get that the expected runtime of the whole algorithm is $\Theta(n+m)$.

$$T = \sum_{i=1}^{N} T_i \implies \mathbb{E}[T] = \mathbb{E}[T_i] \cdot \mathbb{E}[N] \leq \Theta(n+m) \cdot \frac{e}{e-1} = \Theta(n+m)$$

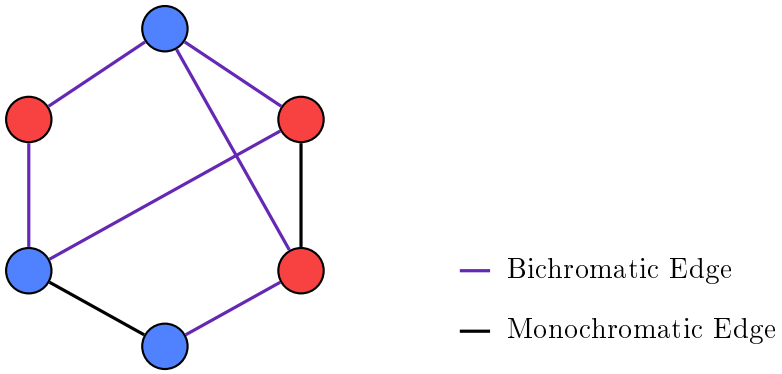
Thus in expected polynomial time we can find a dominating set of size $\lceil n \ln n/(d+1) \rceil + n/(d+1) \approx (n(\ln n + 1))/(d+1)$. This will be useful as long as d is not smaller than $\ln n$, otherwise the dominating set will just be the whole set of vertices.



The union bound while theoretically looks loose is practically quite a good bound in this case.

8.4 Bipartite Subgraph

Given an undirected graph $G = (V, E)$ with $|E| = m$, find the largest subgraph[4] of G which is bipartite. Another way of thinking about this problem[5] is to color all the vertices with one of two colors (say red and blue) and maximise the number of bichromatic edges. The actual maximisation problem is **NP − hard** so we will use probabilistic method to make an algorithm along with a guarantee on the number of bichromatic edges it produces.



8.4.1 Random Experiment

Color each vertex red or blue with equal probability and then count the number of bichromatic edges. Let X be the random variable representing the number of bichromatic edges. Define the indicator variable I_{ij} to be 1 if the edge (i, j) is bichromatic and 0 otherwise.

$$I_{ij} = \begin{cases} 1 & \text{if color}(i) \neq \text{color}(j) \\ 0 & \text{otherwise} \end{cases}$$

[4] Refer Appendix D.3.3
[5] This problem is commonly called MAX-CUT

By definition of I_{ij}, it is clear that X is the sum of all the I_{ij}.

$$X = \sum_{(i,j)\in E} I_{ij}$$

For every edge (i, j), the probability of i and j being different colors is $\frac{1}{2}$ because in exactly half the cases they are of different color, where each case is equally likely.

$$\Pr(I_{ij} = 1) = \frac{|\{(R,B),(B,R)\}|}{|\{(R,R),(R,B),(B,R),(B,B)\}|} = \frac{2}{4} = \frac{1}{2}$$

Now the expected value of X is the sum of expected values of I_{ij}. Let $m = |E|$ be the number of edges in G.

$$\mathbb{E}[X] = \mathbb{E}[\sum_{(i,j)\in E} I_{ij}] = \sum_{(i,j)\in E} \mathbb{E}[I_{ij}] = \sum_{(i,j)\in E} \frac{1}{2} = \frac{m}{2}$$

Using this result we can conclude that there exists a bipartite subgraph of G which has at least $m/2$ bichromatic edges, since in at least one case $X \geq \mathbb{E}[X]$.

8.4.2 The Algorithm

```
Color(G,n)
{
    count=0
    m=E.size()
    while(count < m/n)
    {
        for i in V
            color[i] = random(0,1)

        for (i,j) in E
        {
            if (color[i] != color[j])
                count++
        }
    }
}
```

8.4.3 Success Probability

For $n > 2$, Markov's inequality on the non-negative random variable $m - X$ shows that the algorithm will produce a subgraph with at least $\frac{m}{n}$ bichromatic edges with non-zero probability.

$$Pr(m - X > m - \frac{m}{n}) \leq \frac{\mathbb{E}[m - X]}{m - m/n} = \frac{n}{2(n-1)} \implies Pr(X < \frac{m}{n}) \leq \frac{n}{2(n-1)}$$

$$\implies Pr(X \geq \frac{m}{n}) \geq 1 - \frac{n}{2(n-1)}$$

For example, producing $\frac{m}{3}$ bichromatic edges has at least $1 - \frac{3}{2(2)} = \frac{1}{4}$ probability. For any $n > 2$ we will be able to produce a useful lower bound this way. But what about $n = 2$? For

that, we will exploit the fact that X is a discrete[6] random variable.

Case 1: $m = 2k$ **is even**

Consider all distributions with the same p such that $p = Pr(X \geq \frac{m}{2}) = Pr(X \geq k)$. The maximum $\mathbb{E}[X]$ among all such distributions would be achieved when all the probability mass of the event $X < k$ (which is $1 - p$) is pushed to $k - 1$ and all the probability mass of the event $X \geq k$ (which is p) is pushed to $2k$.

$$\sum_{i=0}^{k-1} \Pr(X = i) \cdot i \leq \sum_{i=0}^{k-1} \Pr(X = i) \cdot (k-1) = (1-p)(k-1)$$

$$\sum_{i=k}^{2k} \Pr(X = i) \cdot i \leq \sum_{i=k}^{2k} \Pr(X = i) \cdot (2k) = (p)(2k)$$

$$\mathbb{E}[X] = \sum_{i=0}^{2k} \Pr(X = i) \cdot i = \sum_{i=0}^{k-1} \Pr(X = i) \cdot i + \sum_{i=k}^{2k} \Pr(X = i) \cdot i \leq (1-p)(k-1) + (p)(2k)$$

Putting $\mathbb{E}[X] = k$, we get the desired lower bound for the success probability.

$$k \leq (1-p)(k-1) + 2kp \implies k \leq k - 1 + p(k+1)$$

$$1 \leq p(k+1) \implies p \geq \frac{1}{k+1} = \frac{2}{m+2} \iff \Pr\Big(X \geq \frac{m}{2}\Big) \geq \frac{2}{m+2}$$

Case 2: $m = 2k + 1$ **is odd**

Similar to the previous case, let $p = Pr(X \geq \frac{m}{2}) = \Pr(X \geq k + 1)$. Using similar arguments as before and $\mathbb{E}[X] = k + \frac{1}{2}$, we get the desired lower bound for the success probability.

$$\sum_{i=0}^{k} \Pr(X = i) \cdot i \leq \sum_{i=0}^{k} \Pr(X = i) \cdot (k) = (1-p)(k)$$

$$\sum_{i=k+1}^{2k+1} \Pr(X = i) \cdot i \leq \sum_{i=k+1}^{2k+1} \Pr(X = i) \cdot (2k+1) = (p)(2k+1)$$

$$\mathbb{E}[X] = \sum_{i=0}^{2k+1} \Pr(X = i) \cdot i = \sum_{i=0}^{k} \Pr(X = i) \cdot i + \sum_{i=k+1}^{2k+1} \Pr(X = i) \cdot i \leq (1-p)(k) + (p)(2k+1)$$

$$k + \frac{1}{2} \leq (1-p)k + p(2k+1) \implies \frac{1}{2} \leq p(k+1) \implies p \geq \frac{1}{2(k+1)}$$

$$\implies p \geq \frac{1}{m+1} \iff Pr(X \geq \frac{m}{2}) \geq \frac{1}{m+1}$$

Combining the two cases we can say that $Pr(X \geq \frac{m}{2})$ is at least the minimum of those two lower bounds.

$$Pr(X \geq \frac{m}{2}) \geq \min\left(\frac{2}{m+2}, \frac{1}{m+1}\right) = \frac{1}{m+1}$$

[6] If X was a continous random variable, it would have been quite difficult to produce a useful lower bound without extracting more information about X from the problem.

8.4.4 Expected Time Complexity

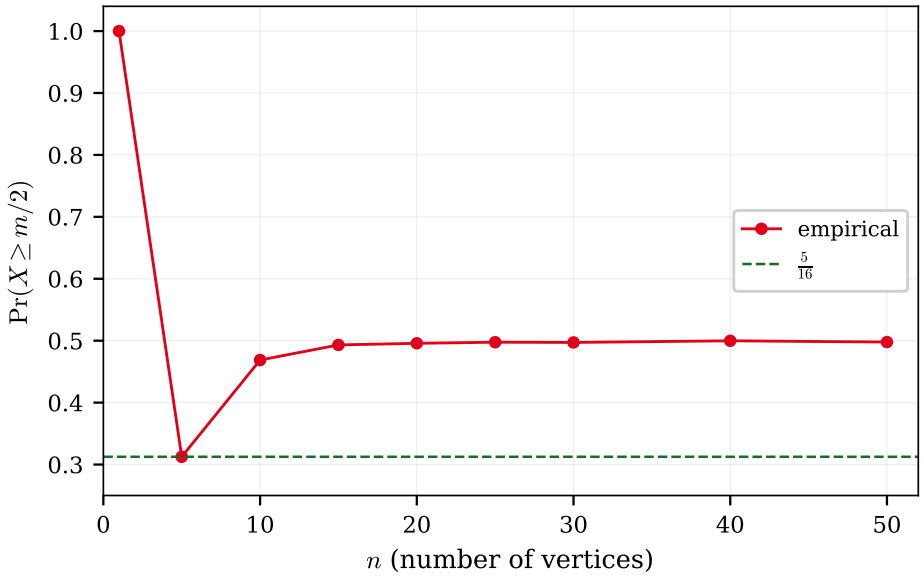
Each `while loop` is essentially a run of the random experiment with each run taking $\Theta(n+m)$ time. Since the success probability is at least $\frac{1}{m+1}$, the expected number of runs is at most $\frac{1}{1/(m+1)} = m+1$

Let $T_1, T_2, \ldots, T_N$ be the runtimes of the corresponding loops and T be the runtime of the whole algorithm. Each T_i is i.i.d. so using Wald's equation we get that the expected runtime of the whole algorithm is at most $\Theta(nm)$.

$$T = \sum_{i=1}^{N} T_i \implies \mathbb{E}[T] = \mathbb{E}[T_i] \cdot \mathbb{E}[N] \leq \Theta(n+m) \cdot (m+1) = \Theta(nm)$$

8.4.5 Conjecture

What is the worst graph for this algorithm? Upon running a large simulation, it is my conjecture that the worst case is the 5-cycle (a pentagon). In that case brute forcing all possiblities gives 5/16 probability of success. For all $n \geq 5$, we can have $n-5$ isolated vertices and a 5-cycle to achieve 5/16 so if the bound is correct, it is tight. So it seems the algorithm is much better than we expected and should run in expected $\Theta(n+m)$ time! In fact, it seems for large n that the success probability is about half, so we expect to need only 2 runs.



For each n, random graphs were generated worst case was taken. For small n all colorings were brute-forced where as for large n sampling was used.

Chapter 9

Online Algorithms

9.1 Introduction

An online algorithm is an algorithm that processes its input piece-by-piece in a serial fashion, without having the entire input available from the start. In some problems, this doesn't matter – the obvious solution is online since it requires a single pass over the array only. But in some problems, the online requirement makes the problem much harder and forces us to make decisions without knowing the future.

Online decision algorithms have randomness involved in their analysis for non-adversarial[1] inputs. Further, the goal of the algorithm usually is not deterministic optimality, but rather to maximise the probability of success or to achieve a good competitive ratio[2].

9.2 Dating

The Problem

Consider the problem of finding the best partner. Assume that the number of people you will eventually date is fixed, i.e. some known value n. After dating a person, you must immediately decide whether to commit to them forever or reject them permanently and continue searching. Figure out a strategy that maximises the probability of ending up with the best partner.[3] We assume that while dating person i, we only know how they compare relative to the people we have already dated. So we could sort the partners till now in a total order if needed.

The Solution

Most course materials, internet reels I have encountered this problem in directly jump to the strategy

> "Reject the first r people, then accept the first person afterwards who is better than everyone seen so far."

However, they often omit the proof of why an optimal strategy must necessarily have this threshold form. We first prove this fact.

[1] They are often called oblivious adversaries

[2] Competitive ratio is a measure of how well an online algorithm performs compared to an optimal offline algorithm that has access to the entire input from the start. The term n-competitive is used when an algorithm achieves at least $\frac{1}{n}$ times the "performance" of an optimal offline algorithm.

[3] This is a different problem from maximising the expected "value" of your partner. Here the objective is specifically to maximise the probability of selecting the single best partner among all n people.

Optimality Proof

Suppose we are currently considering the k-th person.

If the current person is *not* the best among the first k people seen so far, then accepting them can never succeed: This is because there already exists an earlier person who is strictly better, and that earlier person has already been rejected. Hence an optimal strategy should only ever consider accepting people who are the best seen so far.

Now suppose the current person *is* the best among the first k people seen so far. Let $\boldsymbol{q_k}$ be the optimal probability of eventually selecting the globally best person if we reject the k-th person and continue optimally afterwards.

Q] If we instead accept the current person immediately, what is the probability of success?

A] Since the ordering of the n people is uniformly random, each of the first k positions is equally likely to contain the globally best person. Conditioned on the fact that the current person is the best among the first k, the probability that they are actually the globally best person equals $\frac{k}{n}$.

Therefore, accepting gives success probability $\frac{k}{n}$, rejecting gives success probability q_k. The optimal decision is to accept iff $\frac{k}{n} \geq q_k$[4].

Now observe that $\frac{k}{n}$ is **increasing** with k.

On the other hand, q_k is **decreasing** with k. To see this, notice that from time k, one valid strategy is:

> "Reject one more person no matter what, and then behave optimally from time $k+1$ onwards."

This strategy achieves success probability exactly q_{k+1}. Since q_k is the *optimal* achievable success probability starting from time k, we must have

$$\mathbf{q_k \geq q_{k+1}}$$

Since $\frac{k}{n}$ is increasing and q_k is decreasing, the optimal decision changes at most once from rejecting to accepting as k increases. Therefore there exists some threshold $0 \leq r \leq n$ such that

- for $k \leq r$, rejecting is optimal,
- for $k > r$, accepting the first "best-so-far" person is optimal.

Hence every optimal strategy must be a threshold strategy. Note that $r = 0$ corresponds to accepting the first person, and $r = n$ corresponds to rejecting everyone and ending up with no partner. Both of these are valid threshold strategies, though they are not very good ones[5].

[4] This is a common theme in online algorithms - the optimal decision at each step is determined by comparing the expected "value" of actions and choosing the one with maximum value. Here our "value" is the success probability. In game theory this is called "utility" instead.

[5] Though it may be argued that having no partner is the best choice - this is beyond the scope of this text and is left as an exercise for the reader.

Finding the Optimal Threshold

We now analyse the threshold strategy –

> "Reject the first r people, then accept the first person afterwards who is better than everyone seen so far."

Suppose the globally best person appears at position k. For our strategy to succeed,

1. We must have $k > r$
2. Among the first $k-1$ people, the best person must appear within the first r positions.

The first condition ensures that we do not automatically reject the best person. The second condition ensures that no earlier candidate after position r triggers acceptance before the globally best person appears.

$$\Pr[\text{best person appears at position } k] = \frac{1}{n}$$

Further, among the first $k-1$ people, each position is equally likely to contain their maximum.

$$\implies \Pr[\text{maximum among first } k-1 \text{ lies in first } r] = \frac{r}{k-1}$$

$$\implies \Pr[\text{success}] = \sum_{k=r+1}^{n} \frac{1}{n} \cdot \frac{r}{k-1} = \frac{r}{n} \sum_{k=r}^{n-1} \frac{1}{k} = \frac{r}{n} \left(H(n-1) - H(r-1)\right)$$

Using the approximation $H(x) \approx \ln(x)$,

$$\frac{r}{n} \left(H(n-1) - H(r-1)\right) \approx \frac{r}{n} \ln\left(\frac{n-1}{r-1}\right) \approx \frac{r}{n} \ln\left(\frac{n}{r}\right)$$

$$x = \frac{r}{n} \implies P(x) = x \ln\left(\frac{1}{x}\right)$$

$$\implies P'(x) = \ln\left(\frac{1}{x}\right) - 1$$

$$P'(x_o) = 0 \implies \ln\left(\frac{1}{x_o}\right) = 1 \implies x_o = \frac{1}{e}$$

Since $\frac{r_o}{n} = \frac{1}{e} \implies r_o = \frac{n}{e}$, the optimal strategy is –

> "Reject approximately the first $\frac{n}{e}$ people, then accept the first person afterwards who is better than everyone seen so far."

The corresponding success probability is $\frac{1}{e} \ln e = \frac{1}{e}$ which is approximately **36.8**%. Not bad at all!

Strictly speaking, we should also verify that this critical point indeed corresponds to a maximum rather than a minimum. One way is via the second derivative test –

$$P''(x) = -\frac{1}{x} < 0, \ \forall\, x \in \mathbb{R}^{>0}$$

so $P(x)$ is concave and hence the critical point is necessarily a global maximum.

Alternatively, we can avoid calculus entirely by inspecting the behaviour near the boundaries.

$$\because P(x) = x \ln\left(\frac{1}{x}\right)$$

As $x \to 0, P(x) \to 0$ since x decays faster than $\ln(1/x)$ grows.

Similarly, as $x \to 1, P(x) \to 0$ because $\ln(1) = 0$.

Since $P(x)$ is positive for $0 < x < 1$, rises away from 0, and eventually falls back to 0, the unique critical point at $x = \frac{1}{e}$ must correspond to the global maximum.

Behaviour for Small n

We used some limiting n approximations in finding the optimal threshold (I hope you noticed!). In case you were not planning on dating a tending-to-infinite number of people, you might be interested in the optimal strategy for small values of n. For finite n, the exact success probability for threshold r is

$$\max_{0 \le r \le n} \left(\frac{r}{n} \sum_{k=r}^{n-1} \frac{1}{k} \right)$$

Computationally we can now easily find max for each n

n	2	3	4	5	6	7
n/e	0.74	1.10	1.47	1.84	2.21	2.58
round(n/e)	1	1	1	2	2	3
Actual optimal r	**1**	**1**	**1**	**2**	**2**	**3**

The corresponding optimal success probabilities are

n	2	3	4	5	6	7
Pr[success]	**0.500**	**0.500**	**0.458**	**0.433**	**0.428**	**0.414**

We observe that the optimal threshold follows the value suggested by $\frac{n}{e}$ even for relatively small values of n, and it will get closer and closer as n increases. The success probability is already around 41.4% for $n = 7$, and it will approach the limiting value of $\frac{1}{e} \approx 36.8\%$ as n grows.

9.3 Load Balancing

Consider a setting with m identical machines processing incoming tasks. The tasks arrive one after another, and the arrival times are sufficiently close that we may assume they arrive within negligible time of each other.

Each incoming task must immediately be assigned to one of the m machines. Once a decision is made, it cannot be reversed. Thus, the algorithm has no knowledge of future tasks while making its decision.

The tasks assigned to a machine are processed sequentially and hence form a queue at that machine. Suppose the incoming tasks have processing times $T_1, T_2, \ldots, T_n$ where n itself is not

known in advance.

The objective is to minimize the *makespan*, the finishing time of the busiest machine, equivalently $\max_{1 \le i \le m} T_i$ where T_i denotes the total processing time assigned to machine i. This is a natural objective since the overall processing time is determined by the machine that finishes last[6].

Greedy Algorithm

A natural strategy is the following greedy algorithm.

```
Assign(Task T)
{
    find machine with min load
    assign T to this machine
}
```

Performance Analysis

Interestingly, we do not actually have a polynomial time algorithm to find the optimal *offline* solution to this problem.

We will therefore use lower bounds on OPT (the optimal offline makespan) to derive an upper bound on the competitive ratio of the greedy algorithm.

Let ALG denote the makespan produced by the greedy algorithm. Consider the task that finishes last in the greedy schedule. This may or may not be the last *assigned* task. Let its processing time be **p**.

Immediately before assigning this job, suppose the minimum machine load was L. By our hypothesis that this machine will be the one that finishes last, the greedy makespan is

$$ALG = L + p$$

Hence the total load already assigned before this final job was at least **mL**. Including the final job, the total processing load is therefore at least $\mathbf{mL + p}$.

By Pigeonhole Principle, at least one machine must have load at least the average load, so –

$$OPT \ge \frac{mL + p}{m} = L + \frac{p}{m}$$

Since any valid schedule must process the job of size p,

$$OPT \ge p \implies OPT(1 - \frac{1}{m}) \ge p\left(1 - \frac{1}{m}\right)$$

Adding the two inequalities gives:

$$OPT(2 - \frac{1}{m}) \ge L + p = ALG \implies \boxed{\frac{ALG}{OPT} \le 2 - \frac{1}{m}}$$

Therefore the greedy algorithm is $\left(2 - \frac{1}{m}\right)$-competitive, sometimes written as 2-competitive since the difference is negligible for large m.

[6] This concept is similar to the Rate Determining Step (RDS) in the field of Chemical Kinetics

Tightness of the Bound

The previous analysis showed that the greedy algorithm is $\left(2 - \frac{1}{m}\right)$-competitive. We now construct an example showing that the analysis cannot, in general, be improved beyond a factor of $\left(2 - \frac{1}{m}\right)$.

Consider $m = 4$ machines and the following sequence of incoming jobs:

$$1, 1, 1, \ldots, 1 \text{ (12 times)}, 4$$

The greedy algorithm will assign the first 12 jobs as follows:

Machine i	1	2	3	4
Jobs assigned	1, 1, 1	1, 1, 1	1, 1, 1	1, 1, 1

After processing these 12 jobs, each machine has load 3. The last job of size 4 will be assigned to any machine (say machine 1), resulting in a final makespan of 7. On the other hand, an optimal offline algorithm could have assigned the first 12 jobs as follows:

Machine i	1	2	3	4
Jobs assigned	1, 1, 1, 1	1, 1, 1, 1	1, 1, 1, 1	4

After processing these 13 jobs, each machine has load 4. Thus the optimal makespan is 4, and the competitive ratio is $\frac{7}{4}$, which is indeed our bound of $2 - \frac{1}{4}$.

9.4 Distinct Elements

We consider a stream of elements

$$a_1, a_2, \ldots, a_m$$

drawn from a the finite set U. The goal is to write an online algorithm that estimates the number of distinct elements (denoted by D) in the stream. We have the constraint that both m and D are large (say 2^{100}) so we can not use $\Theta(D)$ space like using a RBT[7]. In fact, we would really like to use a small amount of space as compared to D.

9.4.1 Flajolet–Martin Method

We take a hash function $\mathbf{H} : U \to \{0,1\}^N$, where we ensure[8] $2^N \approx m^2$. This hash function is not being used in the traditional sense of reducing memory required to store the element but rather a way to convert an element to a random[9] bitstring. For each element x, define $\rho(x)$ as the number of leading zeros in $\mathbf{H}(x)$.

We then maintain ρ_{max} as we go through the input stream. At the end of the stream, our estimate for D is $D = \alpha \cdot 2^{\rho_{max}}$

Why ρ_{max}?

The statistic max is multiplicity-insensitive, i.e. repeated occurences of an element in the input do not change the value of the max whereas something like "the number of elements with $\rho = 3$" will be skewed by multiple occurences of an element.

[7] This is shorthand for a Red-Black Tree which is used in the implementation of the `set` in `C++`

[8] This choice of N ensures a small collision probability as is required for this algorithm.

[9] As always, we will take pseudo-random as true random for our purposes

Why $\alpha \cdot 2^{\rho_{max}}$?

For a random bitstring, $P(\rho(x) \geq k) = 2^{-k}$. For each level $k \geq 0$ and index of a distinct element $e \in \{1, \ldots, D\}$, define the indicator variable

$$I_{k,e} = \begin{cases} 1 & \text{if } \rho(e) \geq k \\ 0 & \text{otherwise} \end{cases}$$

The point of defining the indicator variable was to define

$$D_k = \sum_{z=1}^{D} I_{k,z}$$

so that D_k counts the number of distinct elements whose hash values have at least k leading zeros.

Since a uniformly random hash has at least k leading zeros with probability 2^{-k},

$$E[I_{k,z}] = P(\rho(x) \geq k) = 2^{-k}$$

Since the variables $I_{k,1}, I_{k,2}, \ldots, I_{k,D}$ are i.i.d.[10], the Strong Law of Large Numbers gives

$$\frac{D_k}{D} = \frac{1}{D} \sum_{z=1}^{D} (I_{k,z}) \xrightarrow{\text{a.s.}} E[I_{k,z}] = 2^{-k}$$

$$\implies D_k \approx D \cdot 2^{-k}$$

So we expect ρ_{max} to be when $D_{\rho_{max}}$ drops to 1,

$$D_{\rho_{max}} \approx 1 \implies D \cdot 2^{-\rho_{max}} \approx 1 \implies D \approx 2^{\rho_{max}}$$

What is α?

If we do multiple parallel runs of the algorithm (essentially using different hash functions and maintaining ρ_{max} separately for each) and take simple average over them (given enough number of runs the Strong Law of Large Numbers will hold), then $\alpha = \frac{E[2^{\rho_{max}}]}{D}$, which we can find since we know the distribution of the random variable ρ_{max} with respect to a fixed D.

The cumulative distribution function of ρ_{max} is

$$Pr(\rho_{max} \geq k) = 1 - (1 - 2^{-k})^D$$

Technically, α is a function of D but as D goes to infinity, α approaches a constant value. The calculus to find $\lim_{D \to \infty} \alpha$ is quite involved so we just skip to stating $\alpha \approx 1.29281$ for large D.

Modification

The average of the random variable $2^{\rho_{max}}$ takes quite a large number of samples to converge to the mean because it has a long right tail (which means the larger values skew the mean a lot). This can also be observed by the large variance of the random variable $2^{\rho_{max}}$.

[10] This is shorthand for independent and identically distributed

Let $R_1, R_2, \ldots, R_m$ be the values obtained from m independent runs and compute the average of 2^{-R_i} rather than the usual 2^{R_i} or equivalently use the harmonic mean of 2^{R_i}. The modified estimate is then proportional to harmonic mean of 2^{R_i} with β_m as constant factor.

$$D = \beta_m \cdot HM(2^{R_i}) = \frac{\beta_m}{Avg(2^{-R_i})}$$

The random variable 2^{-R_i} has a smaller variance hence will help us converge to the correct answer faster.

9.4.2 HyperLogLog Method

Instead of designing multiple independent hashes as in Flajolet–Martin, we can achieve the same effect using a single hash function and storing multiple max statistics of the data. Let $\mathbf{H}(x) \in \{0,1\}^N$ exactly as defined before. We now split the hash into two parts,

$$\mathbf{H}(x) = B(x) \,\|\, S(x)$$

where the first p bits $B(x)$ determine a bucket. A particular bucket is defined as all those elements whose hash starts with a particular bitstring. Thus there are $b = 2^p$ different buckets – while reading the input stream, we classify each element into one of these buckets.

Define $\rho(x)$ as the number of leading zeros in the bitstring $S(x)$. For each bucket we maintain its ρ_{max}.

What we have done is stored b different i.i.d. max statistics of the input stream and we can take the harmonic to get an estimate for number of elements in each bucket. Intuitively, each bucket receives approximately $\frac{D}{b}$ distinct elements and each bucket behaves like a smaller run of the Flajolet–Martin Method.

As before, HyperLogLog introduces a constant factor β_b and estimates number of distinct elements in one bucket using the harmonic mean of 2^{R_i} as

$$\beta_b \cdot HM(2^{R_i}) = \frac{\beta_b}{Avg(2^{-R_i})}$$

So D is estimated as b times the above estimate so

$$D = \frac{\beta_b \cdot b}{Avg(2^{-R_i})}$$

Again, finding the value of the constant β_b is a bit complicated so we skip to stating

$$\beta_b = \left(m \int_0^\infty \left(\log_2 \left(\frac{2+u}{1+u} \right) \right)^m du \right)^{-1} \approx \frac{0.7213}{1 + \frac{1.079}{b}}$$

Memory Usage

Each bucket needs a variable which stores the maximum number of leading zeros observed in its bucket. The maximum number of leading zeros is atmost N since that is the length of the hash. To store the number N we need $\log_2 N$ bits. Since chose N such that $2^N \approx m^2$, we have a space requirement of

$$\log_2(2 \log_2 m) = \Theta(\log(\log(m)))$$

With b buckets, the total memory requirement is

$$\Theta(b \log \log m)$$

In practice b is chosen to be a fixed constant such as $2^{10} = 1024$ so for $m = 2^{100}$, the memory requirement is about $2^{10} \cdot \log_2(2 \log_2(2^{100})) \approx 8000$ bits which is very small.

Chapter 10

Heuristic Algorithms

10.1 Introduction

Many computational problems of practical interest have a statement like "given a graph find a set of vertices of size at most x satisfying property P."

Now one way to guarantee a solution is to find the minimum sized set of vertices satisfying P, but the optimisation problem may be **NP − hard**. So we are often forced to settle for a solution that is not necessarily optimal but can be found efficiently. Such algorithms are *heuristic algorithms* – algorithms designed to quickly find solutions that are often good in practice, even if they do not always provide provable optimality guarantees or even any guarantees at all.

A famous example of a heuristic algorithm is the

10.2 Dominating Set Revisited

Recall the dominating set problem: given a graph $G = (V, E)$, find a smallest possible subset $D \subseteq V$ such that every vertex in the graph is either contained in D or adjacent to some vertex in D.

A Greedy Heuristic

A very natural approach is the following greedy algorithm:

> Repeatedly select the vertex that dominates the largest number of currently undominated vertices.

More precisely:

1. Initially, all vertices are marked undominated.
2. At each step, choose a vertex whose closed neighbourhood $N[v]$ contains the largest number of undominated vertices.
3. Add this vertex to the dominating set.
4. Mark all vertices in $N[v]$ as dominated.
5. Repeat until every vertex becomes dominated.

At every step we greedily maximise immediate progress.

Analysing the Greedy Heuristic

Let OPT denote the size of a minimum dominating set.

Suppose that at some stage of the algorithm there remain r undominated vertices.

Since the optimal solution uses only OPT vertices to dominate the entire graph, those same OPT vertices must also collectively dominate these remaining r vertices.

Therefore, by the pigeonhole principle, at least one of those optimal vertices must dominate at least $\frac{r}{\text{OPT}}$ of the currently undominated vertices.

But the greedy algorithm always chooses a vertex dominating as many undominated vertices as possible. Hence the greedy step must dominate at least $\frac{r}{\text{OPT}}$ new vertices.

Thus after one greedy step, the number of remaining undominated vertices becomes at most

$$r - \frac{r}{\text{OPT}} = r\left(1 - \frac{1}{\text{OPT}}\right)$$

If r_t denotes the number of undominated vertices after t greedy steps, then

$$r_{t+1} \leq r_t \left(1 - \frac{1}{\text{OPT}}\right)$$

Since initially $r_0 = n$, we obtain

$$r_t \leq n\left(1 - \frac{1}{\text{OPT}}\right)^t$$

We would like all vertices to become dominated, i.e. $r_t < 1$.

Since

$$r_t \leq n\left(1 - \frac{1}{\text{OPT}}\right)^t$$

it suffices to find the smallest t such that

$$n\left(1 - \frac{1}{\text{OPT}}\right)^t < 1$$

Rearranging:

$$\left(1 - \frac{1}{\text{OPT}}\right)^t < \frac{1}{n}$$

Taking logarithms:

$$t \ln\left(1 - \frac{1}{\text{OPT}}\right) < -\ln n$$

Since $\ln(1 - \frac{1}{\text{OPT}}) < 0$, dividing reverses the inequality:

$$t > \frac{\ln n}{-\ln\left(1 - \frac{1}{\text{OPT}}\right)}$$

Let S denote the size of the dominating set produced by the greedy algorithm. Then S is the smallest integer t satisfying the above inequality. Thus:

$$S = \left\lceil \frac{\ln n}{-\ln\left(1 - \frac{1}{\text{OPT}}\right)} \right\rceil \leq \frac{\ln n}{-\ln\left(1 - \frac{1}{\text{OPT}}\right)} + 1$$

Now using the inequality

$$-\ln(1 - x) \geq x$$

with $x = \frac{1}{\mathrm{OPT}}$, we obtain

$$-\ln\left(1 - \frac{1}{\mathrm{OPT}}\right) \geq \frac{1}{\mathrm{OPT}}$$

Taking reciprocals reverses the inequality:

$$\frac{1}{-\ln\left(1 - \frac{1}{\mathrm{OPT}}\right)} \leq \mathrm{OPT}$$

$$\implies S \leq \ln n \cdot \frac{1}{-\ln\left(1 - \frac{1}{\mathrm{OPT}}\right)} + 1$$

$$\implies S \leq \mathrm{OPT} \ln n + 1$$

Hence the greedy algorithm produces a dominating set of size at most

$$\boxed{\mathrm{OPT} \ln n + 1}$$

10.3 Independent Set

The Problem We are given a set of exams and a set of students. Each student is enrolled in a subset of the exams. We wish to schedule as many exams as possible in the first time slot[1], subject to the constraint that no student should have two exams scheduled at the same time[2].

The goal is to find a largest possible subset of exams that can be scheduled simultaneously without any conflict.

Reduction

We model the problem using a graph $G = (V, E)$ where:

- each vertex represents an exam,
- an edge exists between two vertices if there is at least one student enrolled in both corresponding exams.

In this formulation, a feasible schedule corresponds to a set of vertices with no edges between them. Such a set is called an *independent set*. Thus, the problem reduces to finding a maximum independent set in a graph.

10.3.1 Random Permutation

V is the set of all vertices. Maintain a set - the independent set we are building (S). Pick a random vertex from V – delete it from V and if none of its neighbours are in S, put it in S. Keep doing this until V is empty. This guarantees forming a valid independent set.

Note that the order in which we pick vertices is a uniformly random permutation of the vertices.

[1] this is different from the problem of the overall scheduling problem which is equivalent to finding the Chromatic Number, i.e. the largest number of colors needed to colour a graph which is equivalent to the minimum number of slots required to conduct the whole exam schedule. This is a simpler problem where we only care about the first time slot.

[2] Or you could just give them a Time Turner and hope for the best :)

Probability a Vertex is Selected

Fix a vertex v. Consider the set consisting of v and its neighbours $N(v)$. There are $d(v) + 1$ vertices in total.

Since all relative orderings are equally likely, the probability that v is the picked earliest among these vertices is:

$$\Pr[v \in S] = \frac{1}{d(v) + 1}$$

Expected Size of the Independent Set

Let I_v be the indicator random variable for the event $v \in S$. Then:

$$|S| = \sum_{v \in V} I_v$$

Taking expectation and using linearity:

$$\mathbb{E}[|S|] = \sum_{v \in V} \mathbb{E}[I_v] = \sum_{v \in V} \frac{1}{d(v) + 1}$$

Therefore, there exists an independent set of size at least:

$$\sum_{v \in V} \frac{1}{d(v) + 1}$$

Denoting the maximum independent set size by $\alpha(G)$, we have:

$$\alpha(G) \geq \sum_{v \in V} \frac{1}{d(v) + 1}$$

Using the Jensen's[3] inequality on 1/(x+1), we have:

$$\sum_{v \in V} \frac{1}{d(v) + 1} \geq \frac{n}{d + 1}$$

where d is the average degree of the graph. Hence:

$$\alpha(G) \geq \frac{n}{d + 1}$$

10.3.2 Local Decision

We now describe an alternative randomized construction that produces a large independent set via local random choices.

Each vertex independently chooses a random value:

$$X_v = \begin{cases} 1 & \text{with probability } p \\ 0 & \text{with probability } 1 - p \end{cases}$$

We construct a set S by including a vertex v if:

- $X_v = 1$, and
- for every neighbour $u \in N(v)$, $X_u = 0$.

[3] Refer Appendix B.2

Probability of Inclusion

Fix a vertex v. The probability that v is included in S is:

$$\Pr[v \in S] = p(1-p)^{d(v)}$$

Therefore:

$$\mathbb{E}[|S|] = \sum_{v \in V} p(1-p)^{d(v)}$$

Guarantee on Expected Size

We analyze the randomized construction where each vertex is selected independently with probability p, and a vertex v is included in the independent set S if it is selected and none of its neighbors are selected. Then:

$$\mathbb{E}[|S|] = \sum_{v \in V} p(1-p)^{d(v)}.$$

Using convexity of $f(x) = (1-p)^x$ and Jensen's inequality, we obtain:

$$\sum_{v \in V} (1-p)^{d(v)} \geq n(1-p)^d,$$

where $d = \frac{1}{n}\sum_{v \in V} d(v)$ is the average degree. Hence:

$$\mathbb{E}[|S|] \geq np(1-p)^d.$$

We now optimize the function:

$$f(p) = p(1-p)^d.$$

Differentiating:

$$f'(p) = (1-p)^d - pd(1-p)^{d-1} = (1-p)^{d-1}\big((1-p) - pd\big).$$

Setting $f'(p) = 0$, we obtain:

$$1 - p - pd = 0 \quad \Rightarrow \quad p = \frac{1}{d+1}.$$

Substituting this optimal value gives:

$$\mathbb{E}[|S|] \geq n \cdot \frac{1}{d+1}\left(1 - \frac{1}{d+1}\right)^d = \frac{n}{d+1}\left(\frac{d}{d+1}\right)^d.$$

Using the standard bound $\left(1 - \frac{1}{d+1}\right)^d \geq e^{-d/(d+1)}$, we obtain:

$$\mathbb{E}[|S|] \geq \frac{n}{d+1} \cdot e^{-d/(d+1)}. \geq \frac{n}{e(d+1)}.$$

This is a slightly weaker bound than the previous method, but it has the advantage of being local and easily parallelizable, as each vertex makes its decision independently based on local information.

Recall from the chapter on data structures that in Skip Lists we let each element locally decide how many levels to appear in. The reason for that was updation was quick since we did not have to change everybody's decision just because of the addition of a single element.

A similar reasoning can be applied here to justify the utility of a local decision algorithm.

10.4 Bipartite Subgraph Revisited

Recall the problem of finding a bipartition of the vertices that maximizes the number of bichromatic edges. We showed that a random coloring achieves at least $m/2$ bichromatic edges in expectation, and with some probability. Now we refine our approach by showing that we can always find a coloring with at least $m/2$ bichromatic edges via a self-correction procedure.

10.4.1 Self-Correction

Start with a random coloring of the vertices. If it has at least $m/2$ bichromatic edges, we are done.

Otherwise, let X be the number of bichromatic edges. We have $X < m/2$, so there are more monochromatic edges than bichromatic edges. Call a vertex v *bad* if it has more monochromatic incident edges than bichromatic incident edges. Otherwise it is *good*.

Claim: If the current cut has fewer than $m/2$ crossing edges, then a bad vertex must exist.

Proof idea: Each monochromatic edge contributes equally to two endpoints. So total number of monochromatic edges is half the sum of monochromatic degrees of vertices. If globally there are more monochromatic edges than crossing ones, by pigeonhole principle[4], some vertex must be incident to more monochromatic than bichromatic edges.

Now consider flipping the color of a bad vertex v. Let $d_h(v)$ be the number of monochromatic (same-color) edges incident to v and $d_c(v)$ the number of bichromatic edges. Since $d_h(v) > d_c(v)$, flipping v increases X by

$$\Delta X = d_h(v) - d_c(v) > 0.$$

Thus, each flip strictly increases the number of bichromatic edges. This means that the number of steps is upper bounded by $m/2$.

```
initialize random coloring of vertices
compute X

while exists bad vertex v:
    flip color of v
    update X
```

Complexity analysis: Each flip increases X by at least 1, and $X \leq m$, so there are at most m flips.

If we maintain for each vertex the counts of monochromatic and bichromatic incident edges, each flip can be updated in $\Theta(\deg(v))$. To find a bad vertex we need atmost $\Theta(n)$ time. Over all flips, total time is atmost

$$O\left(\sum_{v \text{ flipped}} (\deg(v) + n)\right) = \Theta(mn + m).$$

Hence the full procedure runs in atmost[5] $\Theta(mn + m)$ after initialization.

[4] You may also prove this by contradiction – if all vertices are bad, then the total number of monochromatic edges is more than the total number of bichromatic edges by summing the inequality, contradicting the assumption.

[5] this can be improved to $\Theta(m \log n)$ by using priority queues with lazy updates – its a fun competitive programming exercise, I encourage you to try it!

Chapter 11

Game Theoretic Algorithms

Game Theory is an *idealisation* of real life games where under different axioms of the game and assumptions about the players' abilities we seek to find their optimal choices.

11.1 Definitions

In a game each player has some *actions* which they can play in the current *state* when it is their turn. A state is a complete description of the current game situation. A *history* is the sequence of states until (and including) the current state.

A *strategy* for a player is a function that maps every possible state to an action which is legal to play. Simply stated, a strategy is a plan for what to play in what situation. A strategy may be as simple as "pick any random legal move" or as complicated as an entire mapping of every state to a carefully chosen legal move. A strategy profile is the collection of all strategies for all players.

A *rational* player is a player who has the "intelligence" to find and choose a strategy that maximises their own utility.

A fact is *common knowledge* in a set of players means that every player knows the fact, and every player knows that every player knows the fact and so on till infinity. This is an interesting definition but for our purposes the layman meaning will be sufficient.

11.2 Nash Equilibria

A Nash equilibrium is a situation where deviation from the chosen strategy by a player (keeping the strategies of the other players fixed) does not strictly increase the utility of the player. Such a deviation[1] is called a *unilateral deviation*.

We will use the idea of *Nash Equilibrium* to predict the optimal action of a player under the following assumptions (which will be referred to as the *standard rationality assumptions*):

- All players are rational.
- The fact that all players are rational is common knowledge among all players.

In case of multiple Nash equilibria, technically all of them are valid answers to the question – "what will happen in the game under the standard rationality assumptions?"

[1] In some sense, a Nash equilibrium is like the equilibrium of a body in physics where the energy is a function of multiple variables. To check equilibrium, a valid way would be to keep all variables except one fixed and analyse with respect to variation of that variable.

11.2.1 Kingdom's Dilemma

The game is very simple with only one state and two actions for both players. The usual example is a symmetric game set between two kingdoms. Two kingdoms must independently choose between PEACE and WAR. They are not allowed to communicate or offer anything in exchange for cooperation or surrender - it is as if the only purpose of these two players is to maximise their utility and they care about nothing else. Let S_A and S_B be the strategies of Kingdom A and Kingdom B respectively.

S_B \ S_A	PEACE	WAR
PEACE	(4,4)	(0,5)
WAR	(5,0)	(1,1)

Each entry in the above table is of the form (U_A, U_B), where U_A and U_B denote the utilities of Kingdom A and Kingdom B respectively. Usually this table is a strategy-utility matrix but here the strategy is simply to choose an action so we wrote it as a action-utility matrix. This matrix is in general called the *payoff matrix*.

The first coordinate denotes the utility of Kingdom A and the second denotes the utility of Kingdom B for this set of actions. So $(S_A, S_B) = (PEACE, PEACE)$ would give both 4 utility, $(S_A, S_B) = (PEACE, WAR)$ would give A utility 0 and B utility 5, $(S_A, S_B) = (WAR, PEACE)$ would give A utility 5 and B utility 0 and $(S_A, S_B) = (WAR, WAR)$ would give both 1 utility.

Looking at the table, we observe that it would of course be preferable for both to choose PEACE since that gives both of them 5 utility. But a deviation from this would be beneficial – for example, if Kingdom B chose WAR instead, they would gain 1 extra utility. So clearly the strategy profile $(S_A, S_B) = (PEACE, PEACE)$ is not a Nash equilibrium. Looking at each of the four strategy profiles, it is clear that $(S_A, S_B) = (WAR, WAR)$ is a Nash equilibrium – since any kingdom's change of strategy to PEACE would not increase their utility (in fact in this case it strictly decrease their utility).

This analysis reveals that a profile with better utilities for all players may not be the rational outcome! (PEACE,PEACE) was better for both players but was not an equilibrium...

A side point to be noted here is that a decrease of the utility of Kingdom A does not matter to Kingdom B. If B had a choice to go from $(U_A, U_B) = (3, 5)$ to $(U_A, U_B) = (0, 4)$ he would not take it since it decreases his own utility – the common idea that another's loss is our gain is not true in general. The class of two-player games where this is true is called a *constant-sum game*, because the sum of utilities of both players is constant for any strategy profile so maximisation of own utility automatically means minimisation of the opponent's. A common case of constant-sum game is the zero-sum game.

11.2.2 Penalty Kick

The shooter and the keeper are playing a game of penalty kick. The shooter is allowed to kick the ball in two sides – left(L) and right(R). The keeper is allowed to choose between two actions – jump to left(L) or jump to right (R). If both choose same side, the keeper wins. If they choose different sides, the shooter wins.

Let A be the keeper and B be the shooter. The payoff matrix is as follows:

S_B \ S_A	L	R
L	(-1,1)	(1,-1)
R	(1,-1)	(-1,1)

Upon careful inspection, we notice that no square of the payoff matrix is a Nash equilibrium. This is because there exists an obvious deviation for the losing player that would strictly increase their utility – just switch action to the other.

This should make us uncomfortable – our planning stage would never end because there would be no settled conclusion of what strategy to have. So would a rational player just not play the game? Or would there be no point in choosing a strategy, just choose any one because does it really matter? This question itself suggests the introduction of randomisation as part of the strategy.

11.3 Mixed Strategies

A pure strategy is a deterministic map of states to actions. A mixed strategy is a mapping from states to a probability distribution over actions – basically a particular probability of taking any action. The failure of pure strategies suggests that perhaps randomisation would settle the game into equilibrium. In mixed strategies, a rational player seeks to maximise the *expected* utility instead of the usual maximisation of utility.

The idea of a mixed strategy should feel familiar – a deterministic algorithm can be exploited by an adversarial input and randomisation in the algorithm can help us deal with such adversaries, like in `QuickSort`.

11.4 Mixed Strategy Nash Equilibria

The Nash equilibrium we saw earlier is called a Pure Strategy Nash equilibrium. But with mixed strategies, the unilateral deviation is a player changing the probability distribution over actions. Note that the distribution can take any real values as long as the sum of probabilities is 1.

Let us consider the penalty kick game again.

S_B \ S_A	L	R
L	(-1,1)	(1,-1)
R	(1,-1)	(-1,1)

Let $S_A = (p : L\ ,\ 1-p : R)$ be the distribution over L and R for player A. Similarly $S_B = (q : L\ ,\ 1-q : R)$ be the distribution over L and R for player B. The expected utility of player A can be written in terms of p and the conditional expected utilities.

$$\mathbb{E}[U_A] = p \cdot (E[U_A|S_A = L]) + (1-p) \cdot E[U_A|S_A = R]$$

$$\mathbb{E}[U_A|S_A = L] = q(E[U_A|S_A = L, S_B = L]) + (1-q)E[U_A|S_A = L, S_B = R] = q(-1) + (1-q)(1) = 1-2q$$

$$\mathbb{E}[U_A|S_A = R] = q(E[U_A|S_A = R, S_B = L]) + (1-q)E[U_A|S_A = R, S_B = R] = q(1) + (1-q)(-1) = 2q-1$$

$$\implies \mathbb{E}[U_A] = p(1-2q) + (1-p)(2q-1) = (2p-1)(2q-1)$$

Now to ensure unilateral deviation does not increase the expected utility of either player, we must have partial derivatives of the expected utility of player A with respect to p and q equal to zero.

$$\frac{\partial \mathbb{E}[U_A]}{\partial p} = 0 \implies 2(2q-1) = 0 \implies q = \frac{1}{2}$$

$$\frac{\partial \mathbb{E}[U_A]}{\partial q} = 0 \implies 2(2p-1) = 0 \implies p = \frac{1}{2}$$

So both actions are chosen with equal probability for both players. This is not surprising because of the symmetries of the payoff matrix.

Now you may have noticed there was an easier way to find $E[U_A]$ by using the normal definition of expected value as a weighted average of the four cases {LL,LR,RL,RR}. But this way illustrates the fact that the partial derivative with respect to p gave the value of q and vice versa. This is not a coincidence, in fact it is rather obvious from the equation we saw earlier.

$$\mathbb{E}[U_A] = p \cdot (E[U_A|S_A = L]) + (1-p) \cdot E[U_A|S_A = R]$$

Here the conditional probabilities do not depend on p so upon differentiation we get that both conditional expectations must be equal which upon solving yields the value of q. Applying the same argument on the corresponding equation written using q would yield the value of p.

$$\mathbb{E}[U_A] = q \cdot (E[U_A|S_B = L]) + (1-q) \cdot E[U_A|S_B = R]$$

11.5 Indifference Theorem

The generalisation of the observation in the previous section is the *indifference theorem*, where for a general set of probabilities $p_1, p_2, \ldots, p_n$ (whose sum must obviously be 1) corresponding to n strategies of player A, a Mixed Strategy Nash Equilibrium implies all the conditional expected utilities are equal. This makes player A indifferent to which strategy he chooses since all give same expected utility. If one of the conditional expected utilities is higher than the others, then player A will simply choose that strategy with probability 1. This may be viewed as B being forced to make all choices for A equal. Of course, the same thing applies with A being forced to make all choices for B equal.

$$\mathbb{E}[U_A] = \sum_{i=1}^{n} p_i \cdot E[U_A|S_A = i] \text{ and MSNE} \implies E[U_A|S_A = 1] = E[U_A|S_A = 2] = \cdots = E[U_A|S_A = n]$$

An important question is – when can mixed strategy Nash equilibria exist? When can both players enforce indifference for each other (or in general, multiple players)?

11.6 The Minimax Principle

We state without proof that for every finite two-player zero-sum game, there exists a mixed strategy Nash equilibrium. A pure strategy Nash equilibrium is considered to be a special case of the general mixed strategy Nash equilibrium.

Philosophical Note

The analysis leading to Nash equilibrium assumes that when a player considers changing strategy, the strategies of all other players remain fixed. Some philosophers have argued that in perfectly symmetric situations between identical rational agents, this may not be the correct thing to check. If both agents reason in exactly the same way, they will expect both their planning stages to reach identical conclusions. Using this idea in the Kingdom's dilemma, one will be led to conclude that both players should choose PEACE. Personally, this seems much more appropriate to me than the Nash perspective. This alternative viewpoint leads to ideas such as superrationality and other decision theories. Classical game theory, however, adopts the Nash perspective and studies unilateral deviations and this is the commonly accepted viewpoint.

Interestingly enough, the fact that most people who study game theory are prone to accepting the Nash perspective is one of the reasons why it is important for us to also adopt the Nash perspective! One of the ways I like to convince myself of the Nash perspective is that a Nash equilibrium is secure against spies – if a spy in your kingdom reveals your strategy to the other kingdom before the actual simultaneous big "reveal", it won't matter as long as you chose the Nash answer since deviation by the other kingdom won't affect you. In general, as long as only one kingdom gets this leaked info, Nash equilibria are secure.

A side note is that Nash equilibrium is simply a mathematical definiton and is not prescriptive in the sense of "what should players do". Under certain assumptions we can conclude that Nash equilibrium is the optimal choice.

11.7 Problems

1. Derive the mixed strategy equilibrium of Rock–Paper–Scissors.

Chapter 12

Zero-Knowledge Proofs

12.1 Definitions

Proofs

Intuitively, a *proof* for a statement X is a set of true statements $S_1, S_2, \ldots$ which ultimately derive from the axioms of the underlying theory and show the statement X to be true. The one who gives the proof is said to be the *prover* and the one who "reads" the proof and verifies that it indeed does show the desired result is called a *verifier*. We may think of the prover and the verifier as two algorithms which take some input and give some output. Here the prover is allowed any amount of computation power but usually the constraint is to have the verifier work in polynomial time or perhaps expected polynomial time.

Schemes

The proof protocol or *scheme* is the entire "interaction" of the prover and the verifier. This includes the "behaviour" of the prover and the verifier and any randomness used in the entire process. A proof scheme is said to be *complete* if a correct proof gets accepted by the verifier with high probability. The counterpart of this property is that a proof scheme is said to be *sound* if an incorrect proof gets rejected by the verifier with high probability. Ideally, we would like the "high probability" to be 1 but relaxing that criteria is incredibly useful because it allows innovative proof schemes which reduce things like memory and time usage!

Interactive Proofs

An *interactive* proof is a back and forth interaction between the prover and the verifier. The verifier asks a question, the prover sends the answer, and the verifier checks if the answer is correct. This may be repeated as many times as needed (while keeping the overall verifier run time polynomial in its input size). The utility of the interactive proof is that the verifier can ask random questions which must be answered "on the fly" – this is not possible in the one-way method of proof in which prover sends a proof to the verifier and verifier can only read the proof.

Zero-Knowledge

Usually, a proof for a statement reveals some additional information beyond the validity of the statement. For example, proving that a number is composite can be given by revealing a factor of the number not equal to 1 or the number itself. In this proof, the verifier gains *knowledge*

by reading the proof, namely a factor of the number which is extra information as compared to only stating compositeness. The concept of gaining knowledge can be rigorously defined as not changing the probability distribution over something for the verifier but for our purposes the intuitive meaning is enough!

Suppose the prover possesses some secret information and wants to prove to the verifier that they indeed possess it without revealing the actual information. A scheme which accomplishes this while being sound and complete is called a *zero-knowledge proof*. The term "zero-knowledge" comes from the fact that because the verifier gains no knowledge beyond the fact that the prover possesses the secret. This has important applications – if such a proof method exists, it enables two people to verify that they indeed share a secret without revealing the secret in case the one of them is faking it and should not know the secret. To make these ideas more concrete, we will see some examples.

12.2 Card Color

Given the standard deck of 52 cards, a card from the deck has three properties – color, suit and rank. Suppose the prover draws a card from the deck without showing it to anyone. The prover claims that they are holding a red card. How can he prove this without revealing the other 2 properties of the card, namely suit and rank? Clearly, showing the card to the verifier is not an option since that would reveal the other two properties.

So we wish to design a scheme where the prover can prove that they are holding a red card without revealing the secret. So at the end of the proof, the verifier should have no idea what card it is among the 26 red cards in a standard deck.

12.2.1 The Scheme

The prover keeps their card face down separate from the deck. The prover then goes through the remaining deck without showing any card to the verifier. The prover puts all cards which are black face up, revealing them to the verifier and puts all remaining cards (red) face down, not revealing them to the verifier. The verifier accepts the proof if the number of black cards revealed is 26 otherwise the proof is rejected.

12.2.2 Completeness and Soundness

If the prover was indeed holding a red card, then he will be able to reveal 26 black cards. If the prover was lying, they will be able to produce at most 25 black cards no matter what. The scheme is **complete** because a correct proof will be accepted by the verifier with probability 1. The scheme is **sound** because a incorrect proof will be rejected by the verifier with probability 1.

12.3 Graph Isomorphism

Given two graphs G_1 and G_2, the *graph isomorphism problem* is to determine if they are isomorphic. This decision problem is **NP** because the answer `YES` has a polynomially verifiable certificate – the permutation σ which converts G_1 to G_2! The problem is believed to not be in **P** since the best algorithm for it is slower than polynomial time.

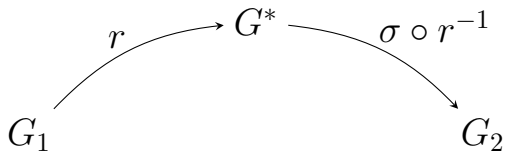
Let us say there exists an open problem of determining whether two publicly known graphs G_1 and G_2 are isomorphic or not. Now a prover claims to have done the requisite computation and found the isomorphism σ, proving that the graphs are isomorphic. To avoid someone stealing

the credit for his hard work, he does not want to share the isomorphism with the verifier but wants to show that he indeed possesses the secret isomorphism.

12.3.1 The Scheme

The prover generates a random permutation r and applies it to the vertex labeling of G_1. This produces a graph G^* which is isomorphic to G_1 and of course, is also isomorphic to G_2. The prover sends G^* to the verifier. The verifier can ask only one of the two following things:

1. Q_1: What is the permutation that would convert G_1 to G^*? (r)
2. Q_2: What is the permutation that would convert G^* to G_2? $(\sigma \circ r^{-1})$



The verifier asks Q_1 and Q_2 with equal probability and verify the answer. If answer is correct, the verifier accepts otherwise declines.

12.3.2 Soundness and Completeness

If the prover truly knows the secret, he will be able to answer any one of Q_1 and Q_2 correctly because he knows r and σ so he will be able to calculate $\sigma \circ r^{-1}$. But if he is a faker and he created G^* by doing $G_1 \circ r$, then he will only be able to answer Q_1 correctly and fail to answer Q_2 since he doesn't know σ. Similarly if he fakes G^* by doing a random $G_2 \circ r$ then he won't be able to tell how to permute G^* to get G_1.

For an actual prover, we will run this scheme k times and since the answer will be correct, we will accept with probability 1. Thus the scheme is **sound** because we accept a correct proof with high probability.

For a faker, the probability of being able to answer a random question among Q_1 and Q_2 correctly is at most $\frac{1}{2} + \frac{1}{2} \cdot \frac{1}{|V|!-1}$. This is because there is a $\frac{1}{2}$ probability of getting the question which he "prepared" for and if he gets the other question, then he can answer a random permutation which is not the inverse of the one he prepared (because $G_1 \neq G_2$) – there are $|V|! - 1$ such permutations. Since the second term is quite small for even $|V| \geq 10$ also, it doesn't really matter so usually it is dropped and we say that the faker always gets caught when it is the other question.

Thus a faker will be rejected with probability $1 - \frac{1}{2^k}$ for k independent runs of this scheme. Since in polynomial time we can run this scheme any k times, we can achieve as high a probability of rejection as we want. Thus the scheme is **sound**!

In general for soundness we can put the condition that if input size is n then rejection of incorrect proof should be done with probability at least $\frac{1}{n^c}$, where $c > 0$ because we can run the scheme independently a polynomial number of times to make it as high a probability of rejection as we want.

Interestingly, this scheme can be viewed intuitively as a $\mathbf{RP_{co}}$ decision problem from the perspective of the verifier – the verifier says `YES` if the proof is correct and `NO` if the proof is incorrect. The verifier always says `YES` when the actual answer is `YES` and says `NO` when the actual answer is `NO` with probability at least $\frac{1}{n^c}$. Indeed, the definition has $\frac{1}{n^c}$ as a lower bound because of the exact same reason as here – we must be able to achieve any high probability of being correct by running this scheme a polynomial number of times.

12.3.3 Game Theory Perspective

Let $|V| = n$ be the number of vertices in the graph G_1. An interesting perspective on this scheme is looking at it as a two-player game between the faker and the verifier.

The faker has two actions A_1 and A_2 – he can fake G^* by doing A_1 which is $G_1 \circ r$ or A_2 which is $G_2 \circ r$. The verifier has two actions Q_1 and Q_2 – the two questions. The game has both of them choosing only one of the actions. The game is a zero-sum game with payoff matrix is as follows:

S_V \ S_F	A_1	A_2
Q_1	(-1,1)	(1,-1)
Q_2	(1,-1)	(-1,1)

Notice this is the exact same game as the penalty-kick game we saw in the chapter on game theory. So we recall there existed a mixed strategy Nash equilibrium with both players choosing each action with equal probability and the expected payoff is 0. Since the optimal play from verifier is to ask Q_1 and Q_2 with equal probability, we can guarantee a payoff of at least 0 in our scheme since the verifier plays optimally. Let p_{catch} be the probability of catching the faker.

$$\mathbb{E}[U_V] = p_{\text{catch}} \cdot (1) + (1 - p_{\text{catch}}) \cdot (-1) \geq 0$$

$$\implies 2p_{\text{catch}} - 1 \geq 0 \implies p_{\text{catch}} \geq \frac{1}{2}$$

Chapter 13

Derandomisation

13.1 Introduction

Randomised algorithms often achieve remarkable simplicity and efficiency. However, they raise a natural question – how much randomness is really necessary? Recall that we stated that for our purposes, pseudo-randomness is true randomness. In this chapter, we will finally differentiate between the two! Since true randomness is often expensive or unavailable, it is natural to ask whether one can reduce, or even eliminate, the amount of randomness required. The study of such questions forms the subject of *derandomisation*.

13.2 Independence

13.2.1 Mutual Independence

A set of random variables $S = \{X_1, X_2, \ldots, X_n\}$ are said to be *mutually independent* if for every subset $s \subseteq S$, we can write the probability of a particular assignment of values as the product of the individual probabilities.

$$\{X_{i_1}, X_{i_2}, \ldots, X_{i_t}\} \subseteq \{X_1, X_2, \ldots, X_n\}$$

$$\implies \Pr[X_{i_1} = a_1, \ldots, X_{i_t} = a_t] = \prod_{j=1}^{t} \Pr[X_{i_j} = a_j]$$

This definition is the strongest possible definition of independence and is the default meaning of the term "independent".

13.2.2 k-Wise Independence

A set of random variables $X_1, X_2, \ldots, X_n$ are said to be *k-wise independent* if every subset of at most k variables is mutually independent.

By definition it follows that $k+1$-wise independence implies k-wise independence. Of course, n-wise independence is the same as mutual independence of the whole set and is thus the strongest version of k-wise independence. The case $k = 2$ is the weakest version of k-wise independence and is called *pairwise independence*.

13.3 Generating Pairwise Independent Bits

Let us say that we wish to generate a pairwise independent set of $n = 2^k$ random bits. Our source of randomness is a set of m truly random bits $R_1, R_2, \ldots, R_m$, where by truly random

what we really mean is mutually independent. Our goal will be to achieve the same n with minimum usage of truly random bits since they are expensive to generate.

13.3.1 Linear Map

Let $n \in \mathbb{K} = \{0, 1, \ldots, 2^k - 1\}$. Consider the finite field $\mathbb{F}_{2^k}$ – recall that in the standard construction of finite fields, the elements are polynomials and the operations are taken modulo a particular polynomial called the irreducible polynomial. We assign indices to the elements of $\mathbb{F}_{2^k}$ as $0, 1, \ldots, 2^k - 1$ (in any order) for our convenience. Hence the elements of $\mathbb{F}_{2^k}$ are $\{p_0, p_1, \ldots, p_{2^k-1}\}$.

Choose uniformly random values $a, b \in \mathbb{F}_{2^k}$ and define the function $T : \mathbb{K} \to \{0, 1\}$ as $T(i) = \text{const}(a \cdot p_i + b)$. The function T basically uses a linear mapping to convert the element p_i to a random element and then extracts the constant term of the resulting polynomial. It is quite straightforward to prove that $a \cdot p_i + b$ for a given i is uniformly distributed over $\mathbb{F}_{2^k}$. Consequently, $T(i)$ is uniformly distributed over $\{0, 1\}$.

$$\implies \Pr(T(i) = 0) = \Pr(T(i) = 1) = \frac{1}{2}$$

Now consider the random bits generated by T as the set of random variables $\{T(0), T(1), \ldots, T(2^k - 1)\}$. With some thought, it can be proved that this set is pairwise independent.

Cost

To generate independent and uniformly distributed a and b, we need $2k$ truly random bits. This is because each of a and b can take 2^k values and we can identify a particular value in the finite field $\mathbb{F}_{2^k}$ using the binary representation of the index of the element. In terms of $n = 2^k$, we needed $2 \log_2 n$ truly random bits to generate n pairwise independent random bits. This is already quite efficient in terms of the number of truly random bits used, but can we do better?

13.3.2 XOR Construction

Choose m truly random bits $R_1, R_2, \ldots, R_m$. For every non-empty subset $S \subseteq \{1, 2, \ldots, m\}$, define the random bit

$$X_S = \bigoplus_{i \in S} R_i,$$

where $\oplus$ denotes the XOR operation. It is fairly straightforward to prove that X_S is equally likely to be 0 or 1.

$$\Pr(X_S = 0) = \Pr(X_S = 1) = \frac{1}{2}$$

To prove pairwise independence, consider two distinct subsets S and T. Since $S \neq T$, there exists a bit R_z belonging to exactly one of them. Without loss of generality, let $z \in S - T$. Fix all random bits except R_z. Flipping R_z changes X_S but leaves X_T unchanged. So for every fixed value of X_T, the bit X_S is equally likely to be 0 or 1.

$$\Pr(X_S = v \mid X_T = w) = \frac{1}{2} = \Pr(X_S = v)$$

$$\implies \frac{\Pr((X_S = v) \cap (X_T = w))}{\Pr(X_T = w)} = \Pr(X_S = v)$$

$$\implies \Pr((X_S = v) \cap (X_T = w)) = \Pr(X_S = v) \cdot \Pr(X_T = w)$$

Cost

Since we generated $n = 2^m - 1$ pairwise independent random bits using m truly random bits, the cost for a fixed n is $m = \log_2(n+1)$. This again begs the question, can we do better? Turns out, we really can not!

13.3.3 Minimum Cost

Let us assume the existence of a set of n pairwise independent random bits generated using m truly random bits. Without loss of generality, consider the generator to be a function $F : \{0,1\}^m \to \{0,1\}^n$ from every instance of the m truly random bits to a particular instance of the n pairwise independent random bits. For our convenience, we represent an instance of the n pairwise independent random bits as a row matrix of dimension $1 \times n$.

Since there are 2^m possible instances of the m truly random bits, we can consider the matrix M of dimension $2^m \times n$ where the i-th row of M is the output of F for the i-th instance of the m truly random bits, indexed arbitrarily (say, lexicographically). For $n = 4$ and $m = 3$ and representing the i-th instance of the m truly random bits as R_i, the matrix M is given by

$$M = \begin{bmatrix} F(R_0 = (0,0,0)) \\ F(R_1 = (0,0,1)) \\ F(R_2 = (0,1,0)) \\ F(R_3 = (0,1,1)) \\ F(R_4 = (1,0,0)) \\ F(R_5 = (1,0,1)) \\ F(R_6 = (1,1,0)) \\ F(R_7 = (1,1,1)) \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Interestingly, the function F in the above example is indeed a valid generator for $n = 4$ pairwise independent random bits using $m = 3$ truly random bits. This can be verified by checking that the pairs $(0,0), (0,1), (1,0)$ and $(1,1)$ occur equal number of times if we consider any two columns side-by-side since it is equivalent to verifying that the probability of a particular pair of bits is $1/4$ which is the exact probability required for pairwise independence. Of course, one may question the need for a matrix representation of this? The reason why we formulated this in terms of matrices is because we will exploit the properties of the matrix operator **rank**.

Consider M for some m and n. Replace every entry of M which is 0 with -1. From our earlier discussion, we know that this means that for every pair of columns side-by-side, the number of occurences of $(1,1)$, $(1,-1)$, $(-1,1)$, and $(-1,-1)$ will be equal.

$$M' = \begin{bmatrix} -1 & -1 & -1 & -1 \\ -1 & -1 & 1 & 1 \\ -1 & 1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

So if we take the dot product of those two columns considering them as vectors, the resulting value will be 0.

$$C_i \cdot C_j = \sum_{k=1}^{n} C_{ik} \cdot C_{jk} = 2^{m-2} \cdot (1 \times 1) + 2^{m-2} \cdot (1 \times -1) + 2^{m-2} \cdot (-1 \times 1) + 2^{m-2} \cdot (-1 \times -1) = 0$$

So any two columns of M are orthogonal, implying that the rank of M is the number of columns of M which is n. Since the rank of a matrix is at most the minimum of the number of rows and columns,

$$\text{rank}(M) \leq \min(n, 2^m) \leq 2^m$$

$$\text{rank}(M) = n \implies n \leq 2^m \implies m \geq \log_2(n)$$

Is this the best bound? With our clever XOR construction, we had achieved something slightly worse than this. It turns out that with a slight modification to the proof, we can achieve the stricter bound whose equality occurs at the XOR construction. Consider prepending a column of all ones to the matrix M'.

$$M'' = \begin{bmatrix} 1 & -1 & -1 & -1 & -1 \\ 1 & -1 & -1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & 1 & -1 \\ 1 & 1 & -1 & -1 & 1 \\ 1 & 1 & -1 & 1 & -1 \\ 1 & 1 & 1 & -1 & -1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

The condition that any two columns of M' are orthogonal still holds trivially because each of the original columns have equal number of 1 and -1 entries. In this version of the proof, we conclude that the rank of M' is $n + 1$.

$$\text{rank}(M) \leq \min(n + 1, 2^m) \leq 2^m$$

$$\text{rank}(M) = n + 1 \implies n + 1 \leq 2^m \implies m \geq \log_2(n + 1)$$

13.4 Bipartite Subgraph Yet Again

Recall the probabilistic algorithm of repeatedly generating a random 2-coloring of the vertices and checking the number of bichromatic edges. The expected number of bichromatic edges was shown to be at least $m/2$, implying the existence of a coloring with at least $m/2$ bichromatic edges. However, if we carefully examine the proof, we notice that we never actually used the mutual independence of the random bits. For every edge (u, v), the only property we required was that the colors assigned to u and v were pairwise independent. This ensured that

$$\Pr(u \text{ and } v \text{ receive different colors}) = \frac{1}{2},$$

which in turn implied that the expected number of bichromatic edges is at least $m/2$. Therefore, instead of assigning colors using $|V|$ mutually independent random bits, we may use our pairwise independent bit generator. From the previous section, we know that if we consider the smallest k such that $2^k \geq |V| + 1$, this requires only k truly random bits. Explicitly, k is equal to $\lceil \log_2(|V| + 1) \rceil$. So instead of proving the existence of a valid coloring in the complete set of colorings of size $2^{|V|}$, we proved the existence of a valid coloring in the set of colorings of size $2^{\lceil \log_2(|V|+1) \rceil} \leq 2(|V| + 1)$. This is a much stronger statement than the original one!

This observation leads us to a completely derandomised and thus completely deterministic algorithm. Since our search space is polynomial in size, we can simply enumerate all possible colorings and we are guaranteed to find a coloring satisfying our requirement of at least $m/2$ bichromatic edges.

13.4.1 The Algorithm

```
n=|V|;
k=ceil(log(n+1.base=2));
vector bestColoring(n);
bestSize=0;
vector all_seeds=GenerateAllSeeds(k);

for (seed in all_seeds)
{
    coloring=GenerateColoring(s)
    cutSize=CountBichromaticEdges(coloring)

    if (cutSize>bestSize)
    {
        bestSize=cutSize
        bestColoring=coloring
    }
}
print(bestColoring);
```

13.4.2 Time Complexity

`GenerateAllSeeds` takes $\Theta(n = |V|)$ time. In each iteration, `GenerateColoring` takes $\Theta(n)$ time and `CountBichromaticEdges` takes $\Theta(m = |E|)$ time. Since there are n such iterations, overall time complexity is $\Theta(n(n + m))$.

13.4.3 Method Of Conditional Expectation

Another way to derandomise the algorithm is to use the method of conditional expectation. Instead of making random choices, we assign vertices one by one smartly, trying to maximise the number of bichromatic edges. Suppose some vertices have already been assigned and the remaining vertices are still unassigned. Let X be the random variable denoting the number of bichromatic edges. Let L and R be the bipartitions.

Denote the expected value of X given the first i assignments by:

$$\mathbb{E}[X \mid i]$$

For the $i+1$-th assignment of vertex v, we have two choices:

- place v in L,
- place v in R.

The current conditional expectation is the average of the expectations obtained from these two choices. Therefore, at least one choice must yield a conditional expectation that is at least as large as the current one.

$$\max\left(\mathbb{E}[X \mid \text{i, v in L}], \mathbb{E}[X \mid \text{i, v in R}]\right) >= \frac{1}{2}\left(\mathbb{E}[X \mid \text{i, v in L}] + \mathbb{E}[X \mid \text{i, v in R}]\right) = \mathbb{E}[X \mid i]$$

We simply assign the next vertex such that it would add the most bichromatic edges in this step – that choice yields a better expectation for the future.

$$\text{Since I have ensured that } \mathbb{E}[X \mid i+1] \geq \mathbb{E}[X \mid i]$$

$$\implies \mathbb{E}[X \mid \text{n assignments}] \geq \mathbb{E}[X \mid 0 \text{ assignments}]$$

$$\implies X_{final} \geq \frac{m}{2}$$

Why did that work?

Initially $\mathbb{E}[X] = \frac{m}{2}$. At every step we choose an assignment that does not decrease the conditional expectation[1]. Hence the conditional expectation never drops below $m/2$!

After all vertices have been assigned, no randomness remains. The conditional expectation is then equal[2] to the actual number of crossing edges produced by the algorithm.

$$\implies X_{final} \geq \frac{m}{2}.$$

Thus we obtain a deterministic algorithm that always constructs a bipartite subgraph containing at least half of the edges.

Why does this not work for all problems?

The properties of this particular problem we needed for this to work were –

[1] This is by no means a proof for optimality – it does not maximise the number of bichromatic edges.

[2] If you are feeling tricked somehow, I relate fully! But it is true, we designed this deterministic algorithm and simultaneously proved that it produces at least $m/2$ crossing edges using conditional expectation!

1. $E[X \mid 1] = m/2$, regardless of which assignment that was – symmetry between the left and right sides.

2. It was obvious which assignment would maximise $E[X \mid i+1]$ given $E[X \mid i]$.

Both these properties do not hold, for example, in the Dominating Set problem. Depending on your first choice (to put in the dominating set or dominated set), you might get a very good dominating set in expectation or a very bad one. An extreme counter example is a star shaped[3] graph, where if you put the central node in dominated set you will get a dominating set of size $n-1$, but if you put it in dominating set you will get a dominating set of size 1. The second property also fails because in further choices it is not clear how to choose which set to put the node in to maximise the conditional expectation.

[3] A star shaped graph is a graph with one central node and $n-1$ nodes connected to it and only it.

13.5 Problems

1 Consider the polynomial map $P : \mathbb{F}_{2^n} \to \mathbb{F}_{2^n}$ defined by $P(i) = a_{k-1} \cdot p_i^{k-1} + a_{k-2} \cdot p_i^{k-2} + \cdots + a_1 \cdot p_i + a_0$. For a random initialisation of $a_0, a_1, \ldots, a_{k-1}$, consider the set of random variables $\mathbb{G} = \{P(0), P(1), \ldots, P(2^n - 1)\}$.

(a) Show that for any i, the random variable $T(i)$ is uniformly distributed over $\mathbb{F}_{2^n}$

(b) Construct a system of equations and show existence and uniqueness of the k-tuple of coefficients $a_0, a_1, \ldots, a_{k-1}$ given a particular assignment of values to any k-sized subset of $\mathbb{G}$. Show that $\mathbb{G}$ is k-wise independent by showing that the probability of a particular assignment of values to any k-sized subset of $\mathbb{G}$ is thus $\frac{1}{2^k}$.

$$\Pr\left(P(i_1) = v_1, \ldots, P(i_k) = v_k\right) = \frac{1}{2^k} = \prod_{j=1}^{k} \Pr\left(T(i_j) = v_j\right)$$

(c) Show that $\mathbb{G}$ is not $(k+1)$-wise independent by considering an arbitrary subset of $\mathbb{G}$ of size $k+1$ and constructing a non-trivial formula for the $(k+1)$-th element in terms of the other k elements, showing that the probability of a particular assignment of values to the $(k+1)$-th element can be 0 in case the formula is not satisified, disproving $(k+1)$-wise independence.

2 Recall that a set of n pairwise independent bits generated from m truly random bits satisfies $m \geq \log_2(n+1)$, obtained by showing that the columns of the associated ± 1 matrix M' (together with a prepended all-ones column) are pairwise orthogonal and hence linearly independent. In this problem, we wish to establish a general bound for k-wise independence. Let F, M, M', M'' be defined as in the text.

Let $\mathcal{S} = \{S \subseteq \{1, \ldots, n\} : |S| \leq k/2\}$ be set of all sets containing at most $k/2$ (including $S = \emptyset$). Define the *XORed matrix* N to be the $2^m \times |\mathcal{S}|$ matrix with one column for each $S \in \mathcal{S}$ constructed by taking XOR of the columns element-wise. The XOR of an empty set is taken as 1 for our convenience. Formally,

$$N_{r,S} = \bigoplus_{i \in S} M_{r,i}, \quad N_{r,\phi} = 1$$

Let N' be the corresponding ± 1-valued matrix obtained from N by replacing 0's with -1's and define C_S be the column generated by the set S.

(a) Show that the number of columns in N' is $|\mathcal{S}| = \sum_{i=0}^{\lfloor k/2 \rfloor} \binom{n}{i}$

(b) For any two distinct subsets $S, T \in \mathcal{S}$, both of size at most $k/2$, show that the dot product of the columns C_S and C_T of M' is zero.

$$C_S \cdot C_T = 0$$

(Hint: consider the symmetric difference $S \triangle T$)

(c) Using the result above, show that

$$\text{rank}(N') = |\mathcal{S}| = \sum_{i=0}^{\lfloor k/2 \rfloor} \binom{n}{i}$$

(d) Use the result above and an inequality on the rank of a matrix to show that

$$m \geq \log_2 \left(\sum_{i=0}^{\lfloor k/2 \rfloor} \binom{n}{i} \right)$$

(e) Show that for $k = 2$, N=M and N'=M" by noticing that our convention of taking empty set XOR as 1 automatically created a column of all 1's in N'. Additionally, verify that our new result matches our original conclusion for $k = 2$.

$$k = 2 \implies m \geq \log_2(n+1)$$

Appendices

Appendix A

Probability and Expectation

A.1 Linearity of Expectation

For any random variables $X_1, X_2, \ldots, X_n$ (not necessarily independent),

$$\mathbb{E}\left[\sum_{i=1}^{n} X_i\right] = \sum_{i=1}^{n} \mathbb{E}[X_i]$$

A.2 Tail Sum Formula

For a non-negative integer-valued random variable X,

$$\mathbb{E}[X] = \sum_{k=1}^{\infty} \Pr(X \geq k) = \sum_{k=0}^{\infty} \Pr(X > k)$$

For a non-negative continuous random variable X,

$$\mathbb{E}[X] = \int_0^{\infty} \Pr(X \geq t)\, dt$$

A.3 Union Bound

For any events $A_1, A_2, \ldots, A_n$,

$$\Pr\left(\bigcup_{i=1}^{n} A_i\right) \leq \sum_{i=1}^{n} \Pr(A_i)$$

This is often used to upper bound the probability that at least one "bad event" occurs.

A.4 Independence of Expectation

If X and Y are independent random variables, then

$$\mathbb{E}[XY] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$$

A.5 Variance

For a random variable X, define the variance of X as

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

For any two random variables X and Y,

$$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y) + 2\,\mathrm{Cov}(X,Y) \quad (\text{where } \mathrm{Cov}(X,Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y])$$

If X and Y are independent, then $\mathrm{Cov}(X,Y) = 0$, so

$$\mathrm{Var}(X+Y) = \mathrm{Var}(X) + \mathrm{Var}(Y)$$

More generally, for any collection $\{X_1, \ldots, X_n\}$,

$$\mathrm{Var}\left(\sum_{i=1}^{n} X_i\right) = \sum_{i=1}^{n} \mathrm{Var}(X_i) + 2 \sum_{1 \le i < j \le n} \mathrm{Cov}(X_i, X_j)$$

A.6 Indicator Variables

For an event A, define the indicator random variable

$$I_A = \begin{cases} 1 & \text{if } A \text{ occurs,} \\ 0 & \text{otherwise.} \end{cases}$$

$$\implies \mathbb{E}[I_A] = \Pr(A)$$

A.7 Law of Total Expectation

Let X and Y be random variables. Then

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X \mid Y]]$$

provided the expectations exist. Intuitively, the theorem states that we may first compute the expected value of X under every possible circumstance described by Y, and then average those conditional expectations according to the probability of each Y.

For a series of random variables $X_0, X_1, \ldots,$

$$\mathbb{E}[X_{i+1}] = \mathbb{E}[\mathbb{E}[X_{i+1} \mid X_i]]$$

A.8 Wald's Equation

Definitions

Let $X_1, X_2, \ldots, X_N$ be i.i.d. random variables with finite expected value μ.

$$\mathbb{E}[X_1] = \mathbb{E}[X_2] = \cdots = \mathbb{E}[X_N] = \mu$$

Now let the number of X's in the series (which is of course, N) also be a random variable with expected value n.

Valid Stopping Time

For N to be a valid stopping time for a series of random variables $X_1, X_2, \ldots$ the decision of whether to stop at i (implying $N = i$) must only depend on the information gathered until X_i. This is fairly intuitive since the decision to stop can not be based of off future information.

For example, a rule like "$N = i$ if $X_i < \mu$" is valid but the rule "$N = i$ if $X_{i+1} > \mu$" is not.

The Equation

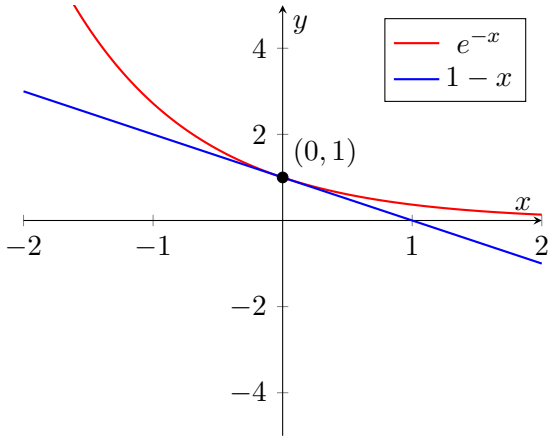
Given that N is a valid stopping time for the series of random variables $X_1, X_2, \ldots$ the Wald's Equation states that the expected sum equals the expected number of terms multiplied by the expected value of each term.

$$\mathbb{E}\left[\sum_{i=1}^{N} X_i\right] = \mathbb{E}[N] \cdot \mathbb{E}[X] = n \cdot \mu$$

Appendix B

Inequalities

B.1 Exponential Inequality



$$1 - x \leq e^{-x} \ \forall \ x \in \mathbb{R} \implies (1 - x)^k \leq e^{-kx} \ \forall \ k \in \mathbb{N}, x \in \mathbb{R}$$

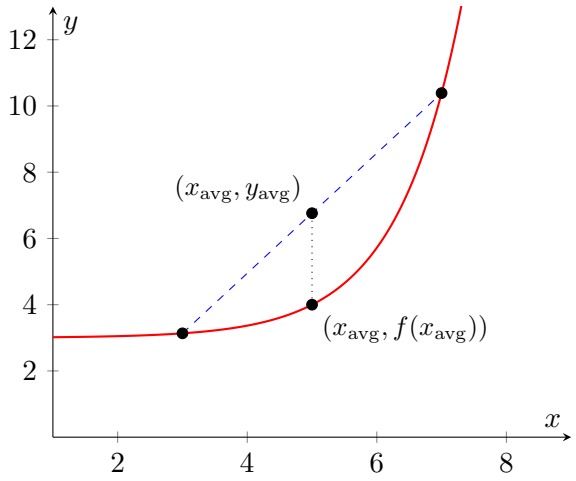
B.2 Jensen's Inequality

For a concave-up function f and random variable X,

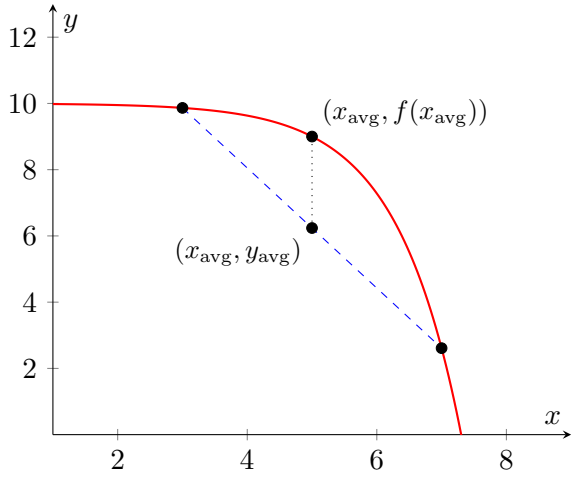
$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$$

Another form of Jensen's inequality is for a convex function f and real numbers $x_1, x_2, \ldots, x_n$,

$$f(\frac{1}{n}\sum_{i=1}^{n} x_i) \leq \frac{1}{n}\sum_{i=1}^{n} f(x_i) \iff f(x_{avg}) \leq y_{avg}$$



For concave-down functions, the inequality is reversed, i.e. $f(x_{avg}) \geq y_{avg}$.



B.3 Markov's Inequality

Let X be a non-negative random variable and let $a > 0$. Then

$$\Pr(X \geq a) \leq \frac{\mathbb{E}[X]}{a}$$

Proof

Define the indicator variable I as

$$I = \begin{cases} 1 & \text{if } X \geq a, \\ 0 & \text{otherwise.} \end{cases}$$

Then $X \geq aI$ follows by definition! Taking expectations,

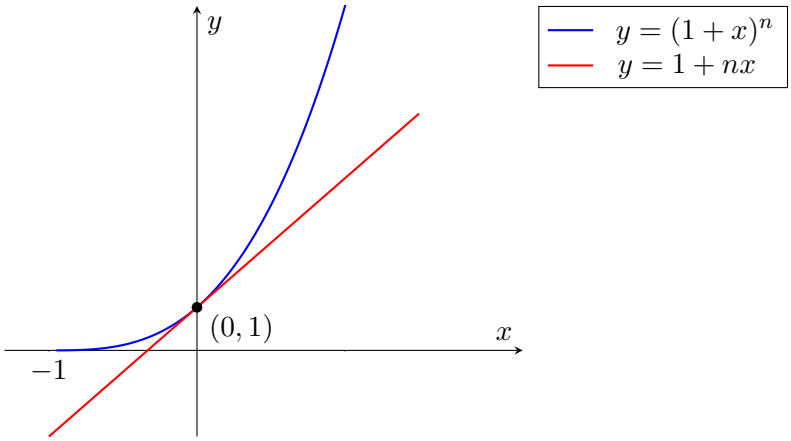
$$\mathbb{E}[X] \geq a \cdot \mathbb{E}[I] = a \cdot \Pr(X \geq a) \implies \Pr(X \geq a) \leq \frac{\mathbb{E}[X]}{a}$$

An equivalent and useful form is $\Pr(X \geq c\,\mathbb{E}[X]) \leq \frac{1}{c}$

B.4 Bernoulli's Inequality

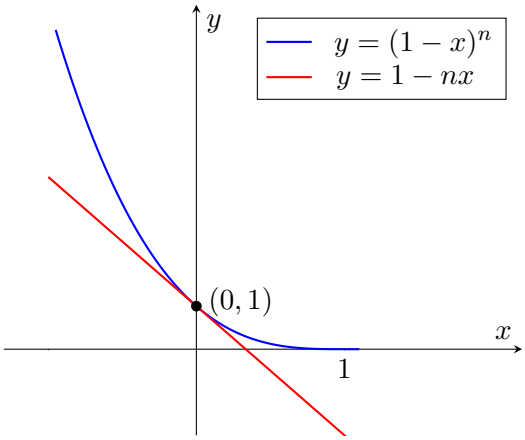
Let $x \geq -1$ be a real number and $n \geq 1$ be an integer. Then $(1+x)^n \geq 1+nx$.

$$x \geq -1 \text{ and } n \geq 1 \implies (1+x)^n \geq 1+nx$$



A commonly used equivalent form is obtained by substituting $x \to -x$.

$$x \leq 1 \text{ and } n \geq 1 \implies (1-x)^n \geq 1-nx$$



Appendix C

Convergence and Limit Theorems

It is quite difficult to properly grasp what these theorems say if you do not know the precise definitions of the terms used to state them. That is why I decided to be more elaborate in this appendix because there a lot of common misconceptions which I struggled to get clarity on until I took a rigorous and exhaustive approach.

C.1 Identical and Independently Distributed Random Variables

This is also denoted by i.i.d. for short. If it is stated that $X_1, X_2, \ldots$ are i.i.d, then they are independent and have the same probability distribution. They are essentially copies of each other, but the value of one does not affect the others.

C.2 Almost Surely

When we say an event occurs "almost surely" we mean that it is true with probability one. The phrase "almost surely" can be used within a sentence like "X is almost surely equal to Y" which means "X is equal to Y" almost surely. We can also say that an event being almost sure is equivalent to the event having "almost every" outcome in its set.

This does not necessarily mean that every outcome is part of the event, it just means the "measure" of the set of outcomes in the event is equal to the total measure of the sample space. We will not be rigorously defining measure here but let me demonstrate via an example.

A random real number chosen from $[0, 1]$ is almost surely not $\frac{1}{2}$. This is because the measure of the set $[0, 1] - \{\frac{1}{2}\}$ is equal to the measure of the sample space.

C.3 Almost Never

When we say an event occurs "almost never" we mean that it is true with probability zero. Again, this does not mean that the event has no outcomes in its set, but rather that the measure of that set is zero as compared to the measure of the sample space. The complementary event of an almost never event is an almost sure event.

The example we used earlier can be restated as "a random real number chosen from $[0, 1]$ is almost never $\frac{1}{2}$".

C.4 $\lim_{n\to\infty} X_n = Y$

The event $\lim_{n\to\infty} X_n = Y$ is defined (where all X_n and Y are random variables) as the set of all outcomes ω such that $\lim_{n\to\infty} X_n(\omega) = Y(\omega)$. Since the sequence $X_n(\omega)$ is a real number sequence, the usual definition of limit applies.

In case Y is a constant a then it is the set of all outcomes ω such that $\lim_{n\to\infty} X_n(\omega) = a$. I personally do not like this notation since we are trying to define the notion of convergence and hence limits for random variables and in the definition we are using the notation $\lim_{n\to\infty} X_n$ which makes it seem circular when it really is not. The notation means to talk about an event defined using the limit of sequences of real numbers and not a sequence of random variables.

C.5 Convergence in Probability

A sequence of random variables X_n "converges in probability" to a constant a if for every $\varepsilon > 0$, the probability of X_n deviating from a by more than ε tends to zero.

$$X_n \xrightarrow{\text{P}} a \iff \lim_{n\to\infty} \Pr(|X_n - a| > \varepsilon) = 0 \ \ \forall\, \varepsilon > 0$$

Note that we can **not** in general, swap the limit and the probability.

C.6 The Weak Law of Large Numbers

Let $X_1, X_2, \ldots, X_n$ be i.i.d random variables with $\mathbb{E}[X_i] = \mu$ and finite variance σ^2. Define the random variable Z_n to be average of the random variables $X_1, X_2, \ldots, X_n$.

$$Z_n = \frac{X_1 + X_2 + \cdots + X_n}{n}$$

The Weak Law of Large Numbers (WLLN) states that Z_n converges in probability to μ.

$$n \to \infty \implies Z_n \xrightarrow{P} \mu$$

Proof

For independent random variables, the variance of their sum is the sum of the variances.

$$\text{Var}(X_1 + \cdots + X_n) = n\sigma^2$$

Dividing by n^2 yields the variance of the average –

$$\text{Var}(Z_n) = \frac{\sigma^2}{n}$$

Applying Chebyshev's inequality (only valid if the variance is finite) gives us the probability of a deviation of the average from its expected value –

$$\Pr(|\overline{Z}_n - \mu| > \varepsilon) \leq \frac{\sigma^2}{n\varepsilon^2} \implies \lim_{n\to\infty} \Pr(|Z_n - \mu| > \varepsilon) \leq \lim_{n\to\infty} \frac{\sigma^2}{n\varepsilon^2} = 0$$

So the probability that Z_n deviates from μ by more than ε tends to zero for all $\varepsilon > 0$

$$\lim_{n\to\infty} \Pr(|Z_n - \mu| > \varepsilon) = 0$$

This is equivalent to Z_n converging in probability to μ, hence the Weak Law of Large Numbers is proved.

C.7 Almost Sure Convergence

A sequence of random variables X_n "almost surely converges" to a constant a if the event $\lim_{n\to\infty} X_n = a$ occurs almost surely. This means for almost every outcome ω, $\lim_{n\to\infty} X_n(\omega) = a$.

$$X_n \xrightarrow{\text{a.s.}} a \iff \Pr\Big(\lim_{n\to\infty} X_n = a\Big) = 1$$

Example

It is not yet intuitively obvious what is the difference between almost sure convergence and convergence in probability. Consider the sequence of independent random variables $X_1, X_2, \ldots$ such that

$$X_n = \begin{cases} 1 & \text{with probability } \frac{1}{n} \\ 0 & \text{otherwise} \end{cases}$$

Let us consider the convergence in probability of X_n to 0. For $\varepsilon \geq 1$, $Pr(|X_n - 0| > \varepsilon) = 0$ since X_n is at most 1 thus $|X_n - 0| \leq 1$. For $\varepsilon < 1$, $Pr(|X_n - 0| > \varepsilon) = \frac{1}{n}$. In both cases, as $n \to \infty$, $Pr(|X_n - 0| > \varepsilon) \to 0$. Hence X_n converges in probability to 0.

Now consider the almost sure convergence to 0 – the probability that the event $(X_n \neq 0)$ occurs is $\frac{1}{n}$ for all n. Applying the second Borel–Cantelli lemma on the event $(X_n \neq 0)$ we get $Pr(X_n \neq 0 \text{ i.o.}) = 1$.

$$\sum_{n=1}^{\infty} \frac{1}{n} = \infty \implies Pr(X_n \neq 0 \text{ i.o.}) = 1$$

Thus the sequence $X_n(\omega)$ almost surely has infinitely many n where $X_n(\omega) \neq 0$. From the concept of limits of real number sequences, a sequence of 0's and 1's has infinitely many 1's iff it can not converge to 0. Since it almost surely has infinitely many 1's, it almost never converges to 0.

So this properly disproves the almost sure convergence of the sequence X_n to 0. In fact, it is almost never convergent to 0, yet it is convergent in probability to 0. Hence the two notions are not the same.

With some thought, it can be shown that almost sure convergence implies convergence in probability, but as we saw, not necessarily the other way around.

C.8 A_n infinitely often

Let $A_1, A_2, A_3, \ldots$ be a sequence of events. Then the event "A_n infinitely often" or "A_n i.o." is defined as there are infinite $n \in 1, 2, 3, \ldots$ such that A_n occurs. Thus the event "A_n i.o." occurs iff for all N, at least one of the events $A_N, A_{N+1}, A_{N+2}, \ldots$ occur.

$$A_n \text{ i.o.} = \bigcap_{N=1}^{\infty} \bigcup_{n=N}^{\infty} A_n$$

C.9 The Borel–Cantelli Lemmas

The **first** Borel–Cantelli lemma states that if the sum of $Pr(A_n)$ is finite, then the event (A_n i.o.) occurs almost never.

$$\sum_{n=1}^{\infty} \Pr(A_n) < \infty \implies \Pr(A_n \text{ i.o.}) = 0$$

The **second** Borel–Cantelli lemma states that if all A_n are independent and the sum of $Pr(A_n)$ diverges, then the event (A_n i.o.) occurs almost surely.

$$\text{All } A_n \text{ independent and } \sum_{n=1}^{\infty} \Pr(A_n) = \infty \implies \Pr(A_n \text{ i.o.}) = 1$$

C.10 The Strong Law of Large Numbers

Let $X_1, X_2, \ldots, X_n$ be i.i.d random variables with $\mathbb{E}[X_i] = \mu$ and finite variance σ^2. Define the random variable Z_n to be average of the random variables $X_1, X_2, \ldots, X_n$.

$$Z_n = \frac{X_1 + X_2 + \cdots + X_n}{n}$$

The Strong Law of Large Numbers (SLLN) states that Z_n converges almost surely to μ.

$$X_n \xrightarrow{\text{a.s.}} \mu$$

The proof of the SLLN uses the Borel–Cantelli lemmas and we will not state it here as the proof itself does not hold much instructive value.

Application of SLLN

If I want to find the expected value of a random variable X and the only thing I can do is sample the variable X ... then I can approximate the expected value really well by averaging over all the samples. This is because sampling a random variable X is equivalent to creating a random variable X_1 and then evaluating it at a random outcome ω. Sampling it n times is equivalent to creating n i.i.d. random variables $X_1, X_2, \ldots, X_n$ and then evaluating them at a random outcome ω.

Taking the average on the n samples would mean evaluating the average random variable Z_n at ω –

$$Z_n(\omega) = \frac{X_1(\omega) + X_2(\omega) + \cdots + X_n(\omega)}{n}$$

Assuming n really is very large, we can use the Strong Law of Large Numbers to state that Z_n converges almost surely to the expected value of X_1. This is equivalent to saying for almost every outcome ω, $Z_n(\omega)$ converges to $\mathbb{E}[X_1]$. In practice for well-behaved random variables this does work quite well if the variance is not too large. It is usually used in physics for experimental data.

If we want more guarantees about the distribution of the average Z_n, we can use the Central Limit Theorem.

C.11 Convergence in Distribution

Let X_n be a sequence of random variables with distribution functions $F_n(x) = \Pr(X_n \leq x)$. We say that X_n converges in distribution to a random variable X, written as

$$X_n \xrightarrow{d} X,$$

if the distribution functions satisfy

$$F_n(x) \to F(x)$$

This is the weakest form of convergence. Convergence in probability implies convergence in distribution.

C.12 The Central Limit Theorem

If $X_1, X_2, \ldots$ are i.i.d. with expected value μ and finite variance σ^2 and Z_n be their average, then

$$\frac{\sqrt{n}}{\sigma}(Z_n - \mu) \xrightarrow{d} \mathcal{N}(0,1)$$

This means that the distribution of the "normalized" average approaches the standard normal distribution as $n \to \infty$, even though the average itself approaches μ. Here the normalisation factor is $\frac{\sqrt{n}}{\sigma}$. Interestingly, the Central Limit Theorem is enough to prove the Weak Law of Large Numbers.

What can we conclude from the Central Limit Theorem? We can say that the average of a large number of samples have fluctuations from the mean of order $\frac{\sigma}{\sqrt{n}}$. In other words the error in the average is $\Theta(\frac{1}{\sqrt{n}})$. Since we know the approximate distribution of the average, we can find things like the value of the probability of it deviating from the expected value by some ε instead of using weak general upper bounds like Chebyshev.

Appendix D

Discrete Mathematics

D.1 Pigeonhole Principle

Let $a_1, a_2, \ldots, a_n$ be non-negative real numbers and let

$$S = \sum_{i=1}^{n} a_i, \text{ then there } \textit{exists} \text{ an } i \text{ such that } a_i \geq \frac{S}{n}$$

This simply states that at least one value is at least the average value.

Proof by Contradiction

If every $a_i < \frac{S}{n}$, then summing over all i gives

$$\sum_{i=1}^{n} a_i < S$$

which is a contradiction.

D.2 Relations

A relation $R : A \to B$ is defined as the set of $(a, b) \in A \times B$. The presence of an element (a, b) in R is denoted by $a \; R \; b$.

D.2.1 Partial Orders

A relation $\preceq$ on a set S is called a *partial order* if it satisfies the following properties for all $a, b, c \in S$:

1. **Reflexivity:** $a \preceq a$
2. **Antisymmetry:** $a \preceq b$ and $b \preceq a \implies a = b$
3. **Transitivity:** $a \preceq b$ and $b \preceq c \implies a \preceq c$

Posets

A *poset*, is a pair $(S, \preceq)$ consisting of a set and a partial order on it.

D.2.2 Equivalence Relations

A relation $\equiv$ on a set S is called an *equivalence relation* if it satisfies the following properties for all $a, b, c \in S$:

1. **Reflexivity:** $a \equiv a$
2. **Symmetry:** $a \equiv b \implies b \equiv a$
3. **Transitivity:** $a \equiv b$ and $b \equiv c \implies a \equiv c$

A very common construction of an equivalence relation is from a partial order as follows –

$$aRb \text{ iff } a \preceq b \text{ and } b \preceq a$$

Equivalence Classes

Given an equivalence relation $\equiv$ on S, the *equivalence class* of an element $a \in S$ is all the elements related to it.

$$[a] = \{x \in S : x \equiv a\}$$

D.3 Graph Theory

A graph G is an ordered pair (V, E) where V is a set of vertices and $E \subseteq \{\{u, v\} \mid u, v \in V,\ u \neq v\}$ is a set of edges. $|E|$ is usually denoted by m and $|V|$ is usually denoted by n.

D.3.1 Degree

For a vertex $v \in V$, the degree $d(v)$ is the number of edges which have v as an endpoint. Equivalently, $d(v)$ is the number of neighbours of v.

Given n is the number of vertices, the average degree of a graph is

$$d = \frac{1}{n}\left(\sum_{v \in V} d(v)\right) = \frac{2m}{n}$$

D.3.2 Bipartite Graph

A graph $G = (V, E)$ is bipartite if the vertex set V can be partitioned into two disjoint sets L and R such that

$$V = L \cup R, \quad L \cap R = \emptyset$$

and every edge connects a vertex in L to a vertex in R. That is, there are no edges with both endpoints in the same set.

D.3.3 Subgraph

A subgraph of a graph (V, E) is a pair of sets (V', E') where $V' \subseteq V$ and $E' \subseteq E$ with the additional condition that the subgraph must be a valid graph!

D.3.4 Clique

A clique in a graph G is a subset chosen from the set of vertices of G such that every pair of vertices in the clique is connected by an edge. The size of the clique is defined as the number of vertices in the clique.

Appendix E

Abstract Algebra

E.1 Rings

A *ring* $(R, +, \cdot)$ is a set R equipped with two binary operations, addition and multiplication where the following properties hold.

1. The operation $+$ must satisfy the following properties for all $a, b, c \in R$:
   (a) Closure: if $a, b \in R$, then $a + b \in R$.
   (b) Associativity: $(a + b) + c = a + (b + c)$
   (c) Identity: there exists $e \in G$ such that $a + e = e + a = a$
   (d) Inverses: for every $a \in R$, there exists $a^{-1} \in R$ such that
   $$a + a^{-1} = a^{-1} + a = e$$
   (e) Commutativity: $a + b = b + a$
2. The operation $\cdot$ must satisfy the following properties on for all $a, b, c \in R$:
   (a) Associative: $(a \cdot b) \cdot c = a \cdot (b \cdot c)$
   (b) Distributive over $+$: $a \cdot (b + c) = a \cdot b + a \cdot c$

Commutative Rings

A ring is said to be *commutative* if $a \cdot b = b \cdot a$ for all $a, b \in R$.

E.2 Fields

A *field* $(F, +, \cdot)$ is a commutative ring with a multiplicative identity such that every element which is not the additive identity possesses a multiplicative inverse. We choose to denote the additive identity as 0, the multiplicative identity as 1, the additive inverse of a as $(-a)$ and the multiplicative inverse of a as a^{-1} or $\frac{1}{a}$.

E.2.1 Division

The operator *division* is the operator / where a/b is defined as $a \cdot b^{-1}$ if $b \neq 0$. If $b = 0$, then a/b is undefined. a/b is also denoted as $\frac{a}{b}$ just like normal divison.

E.2.2 Finite Fields

A field containing finitely many elements is called a *finite field*. A finite field[1] exists if and only if its size is of the form p^k where p is a prime number and k is a positive integer. Finite fields of size p^k are denoted by F_{p^k}.

E.3 Polynomial Rings

Let F be a field. The set of all polynomials with coefficients in F is denoted by $F[x]$.

$$f(x) \in F[x] \iff f(x) = a_n \cdot x^n + a_{n-1} \cdot x^{n-1} + \cdots + a_1 \cdot x + a_0, \text{ all } a_i \in F$$

E.3.1 Degree

The *degree* of a nonzero polynomial is the largest index n for which $a_n \neq 0$. If all coefficients are 0, then the degree is assigned $-\infty$ to satisfy certain theorems about polynomials.

E.3.2 Division Algorithm

Let $f(x), g(x) \in F[x]$ with $g(x) \neq 0$. Then there exist unique polynomials $q(x), r(x) \in F[x]$ such that $f(x) = q(x) \cdot g(x) + r(x)$ where $\deg(r(x)) < \deg(g(x))$. Here, polynomial multiplication is performed in the usual way of convolution using the operators from the field F.

E.3.3 Factor Theorem

An immediate conclusion from the division algorithm is that if $f(a) = 0$ for some $a \in F$, then $f(x)=(x - a) \cdot g(x)$ where $g(x)$ is also a polynomial of degree one less than $f(x)$.

E.3.4 Irreducible Polynomials

A nonconstant polynomial $f(x) \in F[x]$ is said to be *irreducible* if it cannot be written as a product of two polynomials of strictly smaller degree. For example, $x^2 + x + 1$ is an irreducible polynomial over the polynomial ring $\mathbb{F}_2[x]$ – this can be proved easily by simply putting 0 and 1 in the polynomial and verifying that the value isn't 0.

Irreducible polynomials always exist for a polynomial ring over a finite field. Irreducible polynomials over a field play a role analogous to that of prime numbers in the integers.

E.4 Construction of $\mathbb{F}_{2^k}$

Let $p(x) \in \mathbb{F}_2[x]$ be an irreducible polynomial of degree k. Then the set $\mathbb{F}_2[x] \mod (p(x))$ generates all polynomials of degree at most $k - 1$. This set with the operators $+$ and $\cdot \mod p(x)$, (where coefficients are reduced modulo 2) forms a field. Since each coefficient is in $\mathbb{F}_2$, we have two choices for each coefficient (0 and 1) so there are 2^k elements in this field and is a valid construction for $\mathbb{F}_{2^k}$. For example, the field $\mathbb{F}_4$ consists of the elements $\{0, 1, x, x + 1\}$.

A common confusion is to think of $\mathbb{F}_{2^k} = \mathbb{F}_2[x] \mod (p(x))$ as a polynomial ring. This is not the case, it is an actual field with all the properties as expected by the definition. This will become more clear with examples.

[1] Interestingly, all finite fields of the same size are isomorphic which is quite intuitive because all the properties of the operators over the set are the same, only a relabeling of elements can be done.

E.4.1 Addition

Addition is performed coefficient-wise modulo 2. Let us say that $x^3 + x + 1$ and $x^2 + x + 1$ are to be added.

$$(x^3 + x + 1) + (x^2 + x + 1) = x^3 + x^2 + ((1 + 1) \bmod 2)x + ((1 + 1) \bmod 2) = x^3 + x^2$$

E.4.2 Multiplication

Multiply the corresponding polynomials in the usual way and reduce the result modulo the irreducible polynomial $p(x)$. Let us say that $x + 1$ and x are to be multiplied in the field $F_4 = \mathbb{F}_2[x] \mod (x^2 + x + 1)$.

$$(x + 1) \cdot (x) = (x^2 + x) \bmod (x^2 + x + 1) = -1 \bmod (x^2 + x + 1) = 1$$

Appendix F

Linear Algebra

F.1 Vector Spaces

Let $\mathbb{F}$ be a field. A vector space over $\mathbb{F}$ is a set V equipped with vector addition and scalar multiplication satisfying the usual axioms:

- closure under addition and scalar multiplication,
- associativity and commutativity of addition,
- existence of a zero vector,
- existence of additive inverses,
- distributive properties,
- compatibility of scalar multiplication.

An example of a common vector space is the set of n-tuples of elements from $\mathbb{F}$,

$$\mathbb{F}^n = \{(x_1, x_2, \ldots, x_n) : x_i \in \mathbb{F}\}$$

Common choices for the field $\mathbb{F}$ include the real numbers $\mathbb{R}$ or the integers $\mathbb{Z}$ with the usual addition and multiplication, and the finite field $\mathbb{F}_p$ where p is a prime with addition and multiplication modulo p.

F.2 Subspaces

A subset $W \subseteq V$ is called a subspace of the vector space V if

1. $0 \in W$
2. for all $u, v \in W$, we have $u + v \in W$
3. for all $u \in W$ and $\alpha \in \mathbb{F}$, we have $\alpha u \in W$

F.3 Dimension of a Subspace

A collection of vectors $v_1, v_2, \ldots, v_k$ is called a basis of a subspace W if

- the vectors are linearly independent
- any vector in W is a linear combination of the vectors

The number of vectors in any basis of W is called the dimension of W and is denoted by $\dim(W)$.

F.4 The Nullspace

Let $M \in \mathbb{F}^{m\times n}$. The nullspace of M is defined by

$$\mathcal{N}(M) = \{x \in \mathbb{F}^n : Mx = 0\}.$$

The nullspace is also called the *kernel* of M, and we write

$$\ker(M) = \mathcal{N}(M).$$

The nullspace of a matrix is a subspace of $\mathbb{F}^n$.

F.5 Nullity and Rank

Since the nullspace is a subspace, it has a well-defined dimension.

Define $\text{nullity}(M) = \dim(\ker(M))$.

The rank of M, denoted $\text{rank}(M)$ is the dimension of the column space or row space of M, equivalently the maximum number of linearly independent columns or rows of M.

F.6 The Rank–Nullity Theorem

One of the fundamental results of linear algebra is the rank–nullity theorem.

Theorem (Rank–Nullity). For every matrix $M \in \mathbb{F}^{m\times n}$, we have

$$\text{rank}(M) + \text{nullity}(M) = n.$$

An immediate consequence is that if $M \neq 0$, then

$$\text{rank}(M) \geq 1, \implies \text{nullity}(M) \leq n - 1.$$